3.Convolutional neural network

Slide ref:-

PROF. VINEETH B., IITH, NPTEL

Yunzhe Xue

HTTP://CS231N.GITHUB.IO/CONVOLUTIONAL-NETWORKS/

Convolutional Neural Networks

CONVOLUTION OPERATION, MOTIVATION, POOLING, CONVOLUTION AND POOLING AS AN INFINITELY STRONG PRIOR

Review: Convolution Operation

- **Convolution** is a mathematical way of combining two signals to form a third signal
- One of the most important techniques in signal processing
- In case of 2D data (grayscale images), the convolution operation between a filter $W^{k \times k}$ and an image $X^{N1 \times N2}$ can be expressed as:

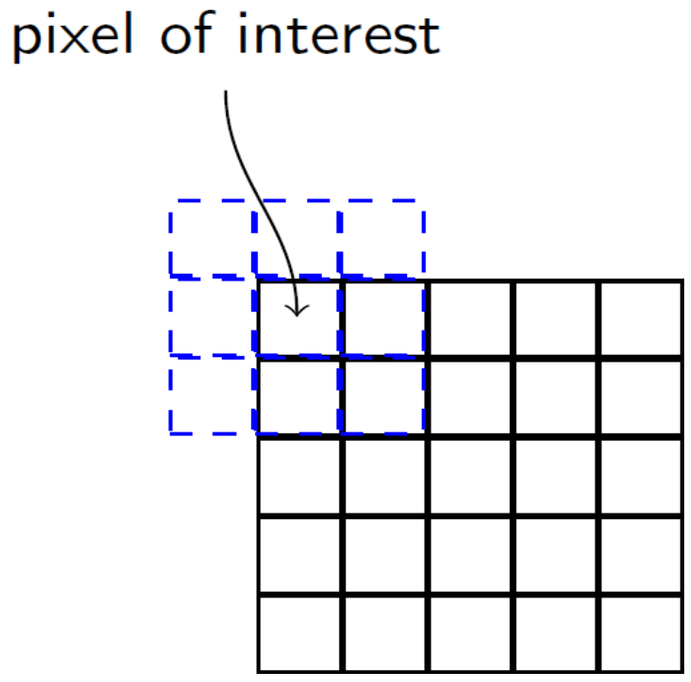
$$Y(i,j) = \sum_{u=-k}^{k} \sum_{v=-k}^{k} W(u,v) X(i-u, j-v)$$

Convolution Operation

More generally, given a $K_1 \times K_2$ filter W, we can write it as:

$$Y(i,j) = \sum_{a=\lfloor -\frac{K_1}{2} \rfloor}^{\lfloor \frac{K_1}{2} \rfloor} \sum_{b=\lfloor -\frac{K_2}{2} \rfloor}^{\lfloor -\frac{K_2}{2} \rfloor} X(i-a, j-b) W\left(\frac{K_1}{2} + a, \frac{K_2}{2} + b\right)$$

This allows kernel to be **centered** on pixel of interest

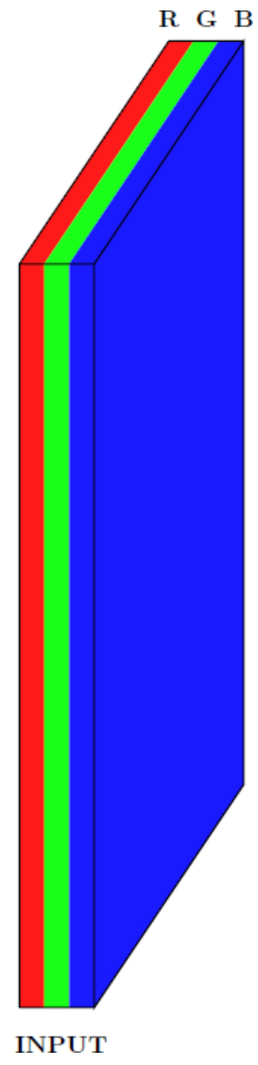


Pause and Ponder

- In the 1D case, we slide a one-dimensional filter over a one-dimensional input
- In the 2D case, we slide a two-dimensional filter over a two-dimensional input
- What would happen in the 3D case where your images are in color (RGB)?

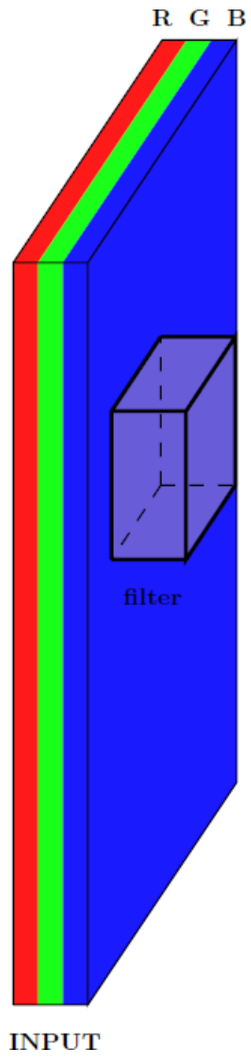
Convolution Operation

- What would a 3D filter look like?



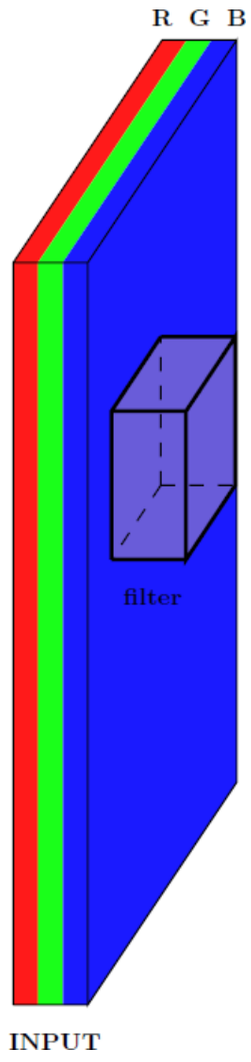
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume



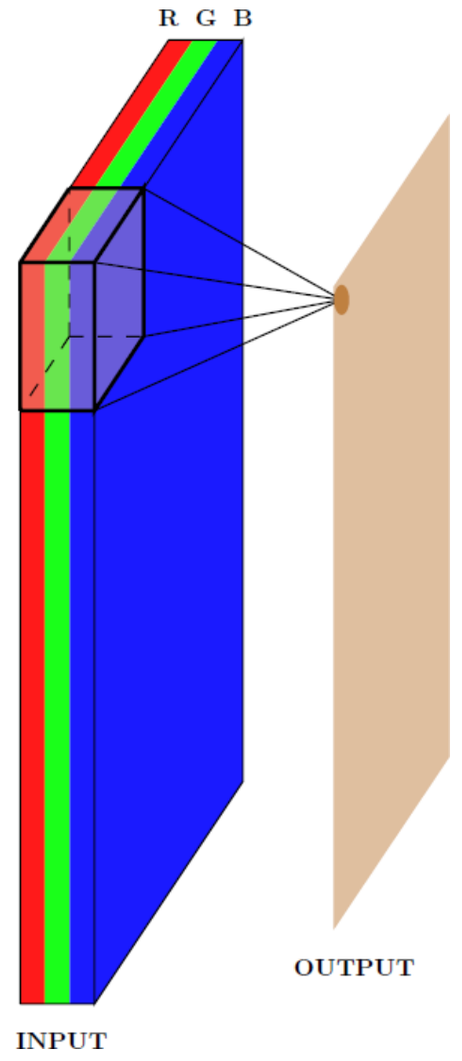
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over 3D input and perform the convolution operation



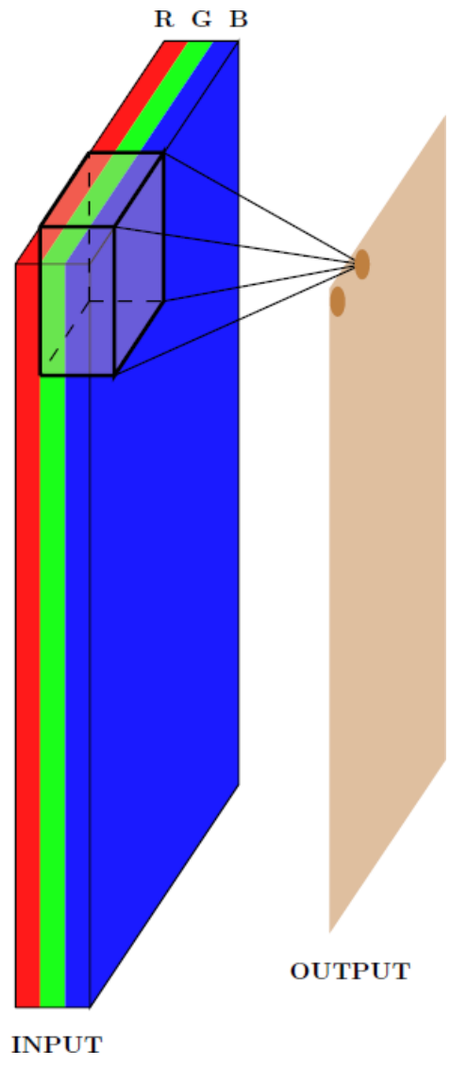
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



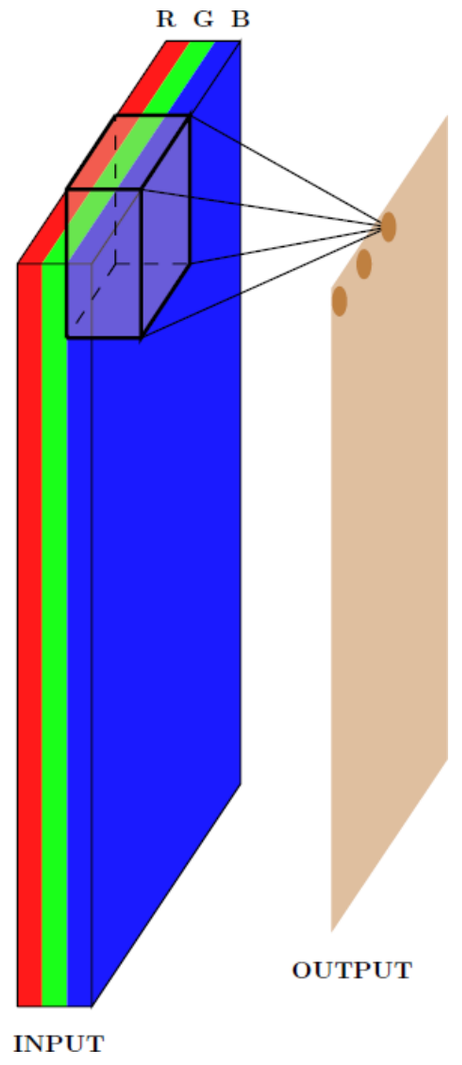
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



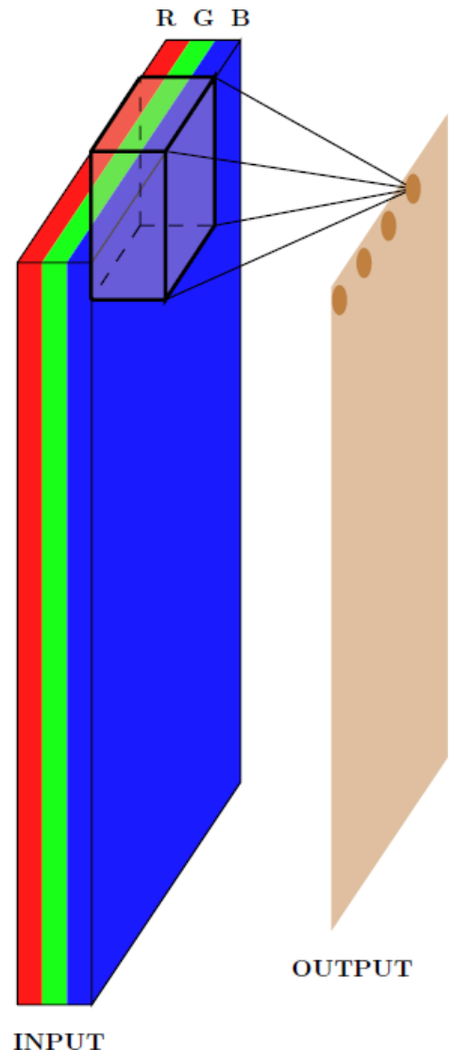
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



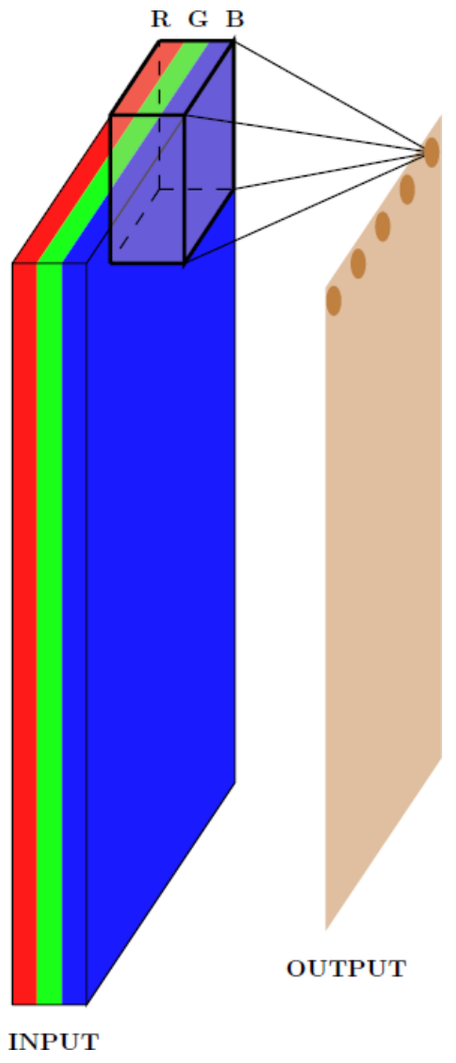
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



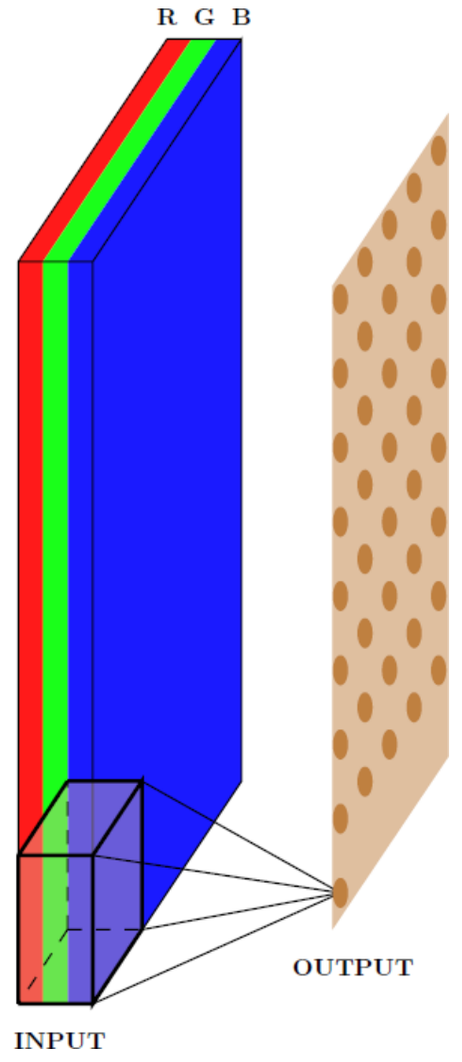
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



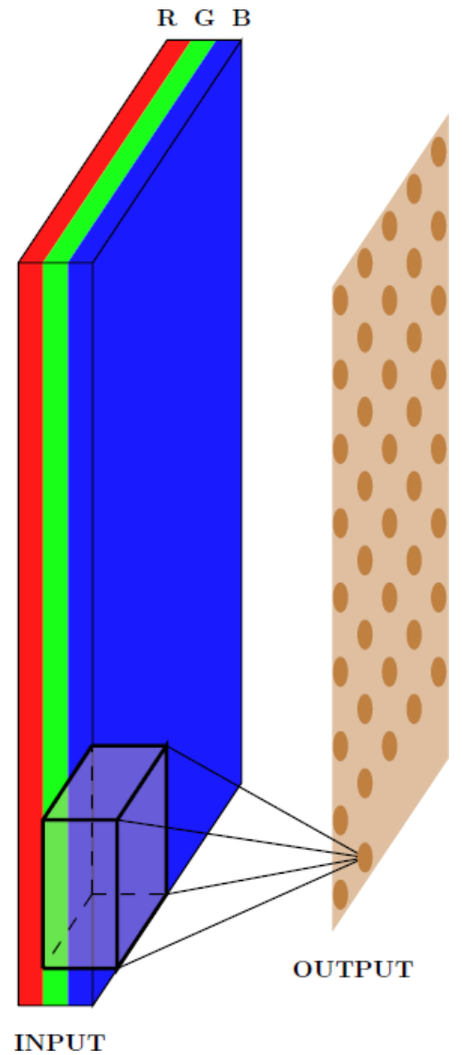
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



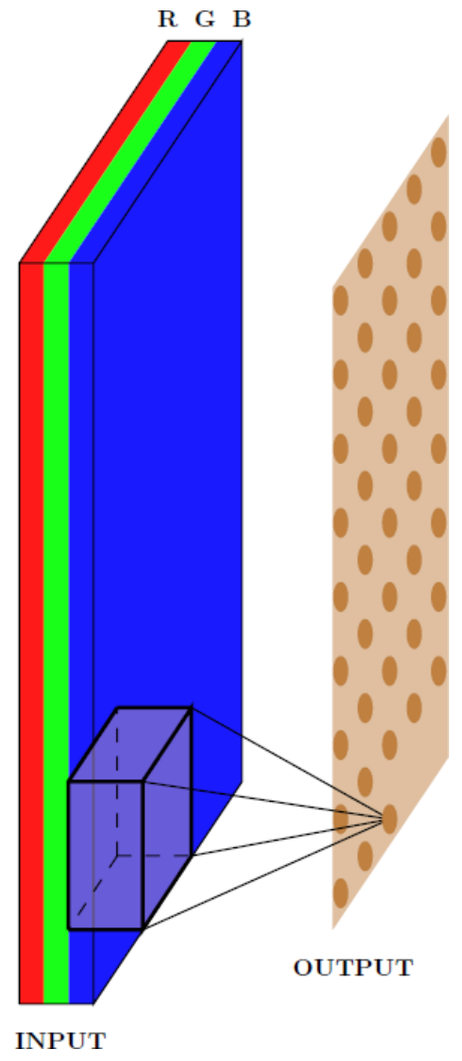
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



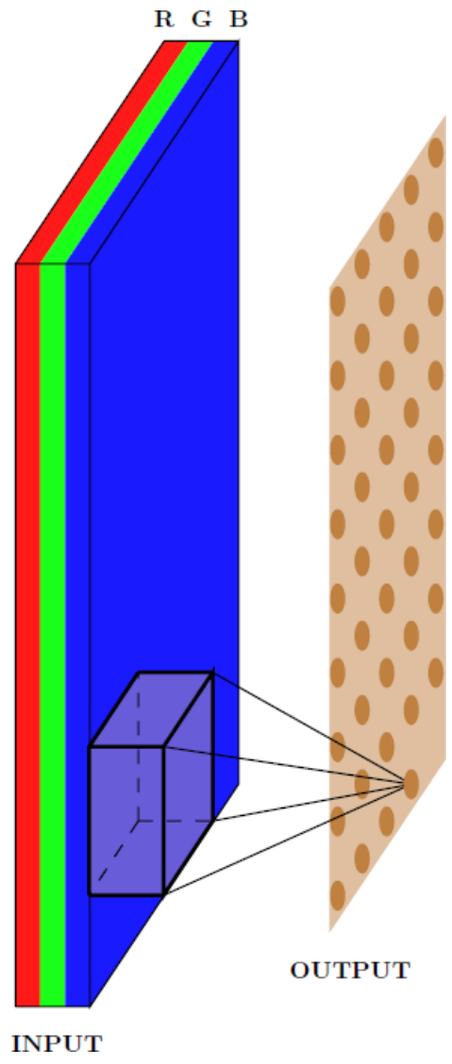
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



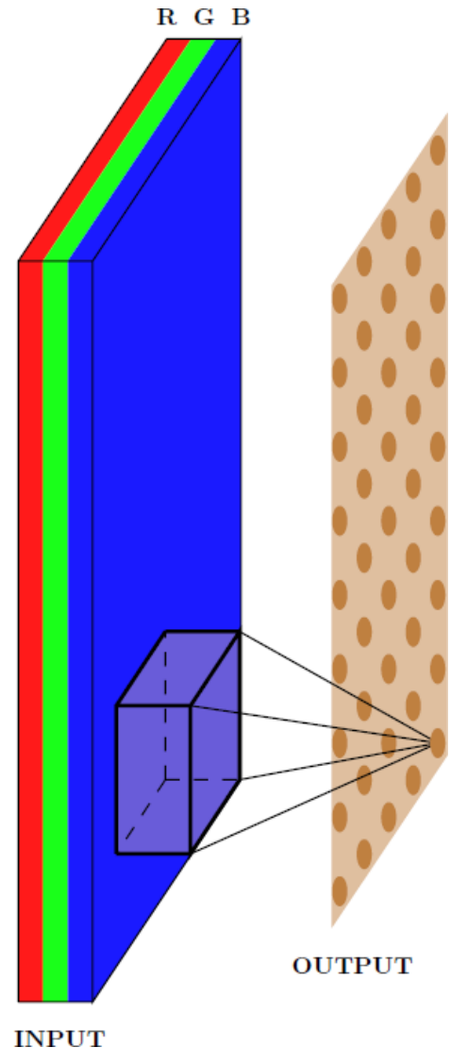
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation



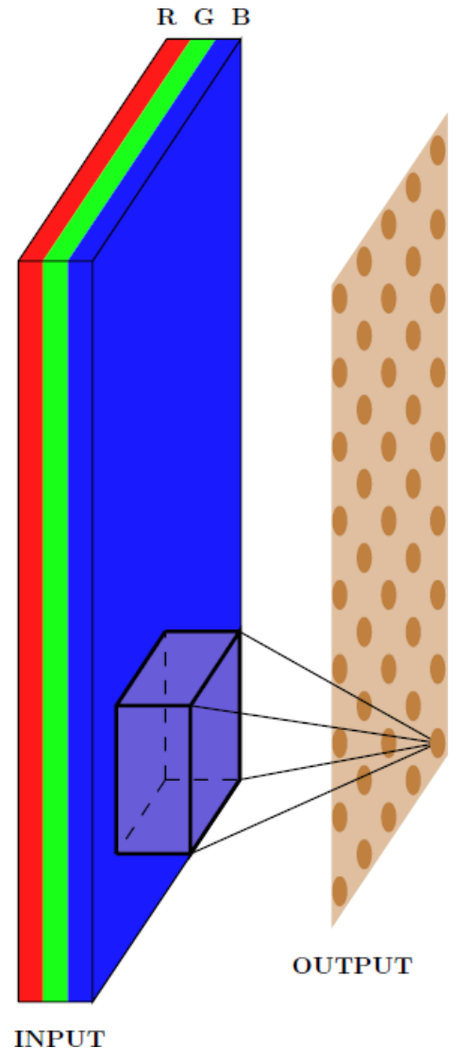
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation
- We assume that the filter always extends to the depth of the image
- In effect, we are doing a 2D convolution operation on a 3D input (because the filter moves along the height and the width but not along the depth)



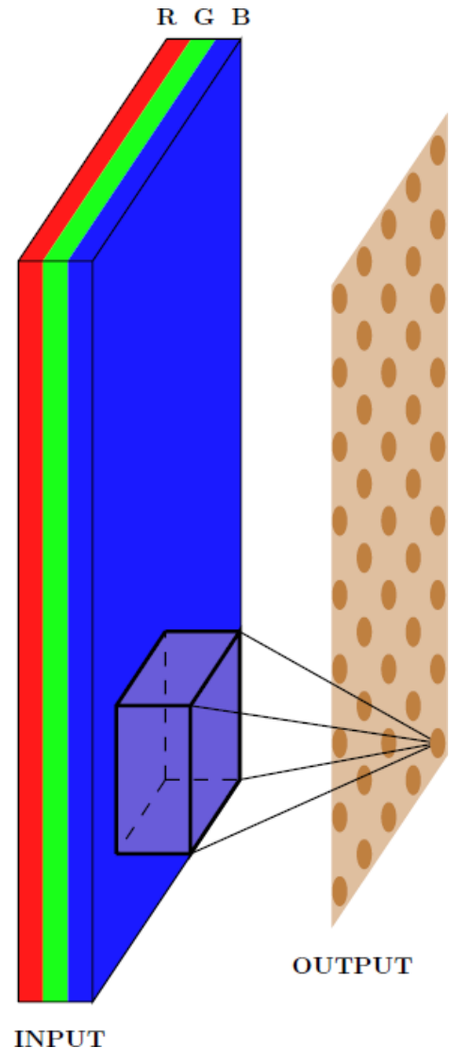
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation
- We assume that the filter always extends to the depth of the image
- In effect, we are doing a 2D convolution operation on a 3D input (because the filter moves along the height and the width but not along the depth)
- As a result the output will be 2D (only width and height, no depth)



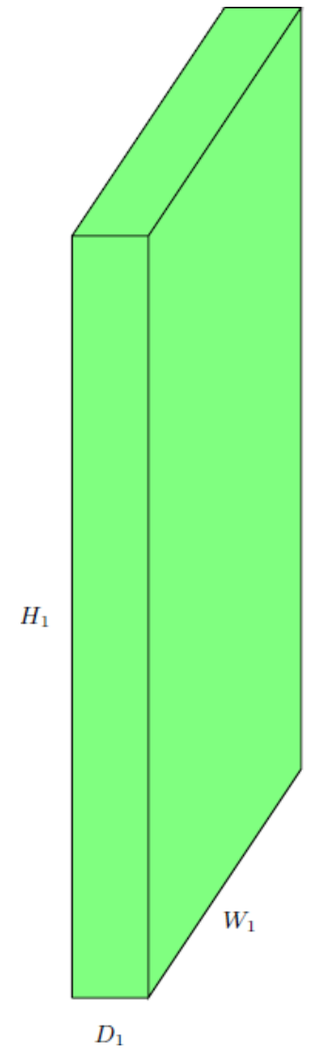
Convolution Operation

- What would a 3D filter look like?
- It will be in 3D too and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and perform the convolution operation
- We assume that the filter always extends to the depth of the image
- In effect, we are doing a 2D convolution operation on a 3D input (because the filter moves along the height and the width but not along the depth)
- As a result the output will be 2D (only width and height, no depth)
- We can apply multiple filters to get multiple feature maps



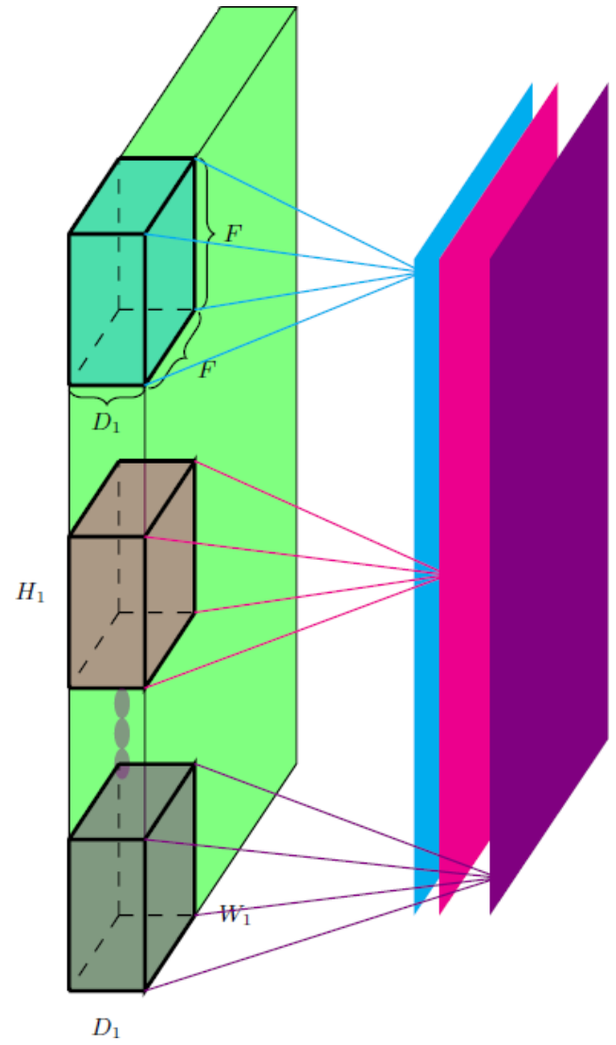
Convolution: Understanding the (Hyper)Parameters

- Input dimensions: Width (W_1) × Height (H_1) × Depth (D_1)



Convolution: Understanding the (Hyper)Parameters

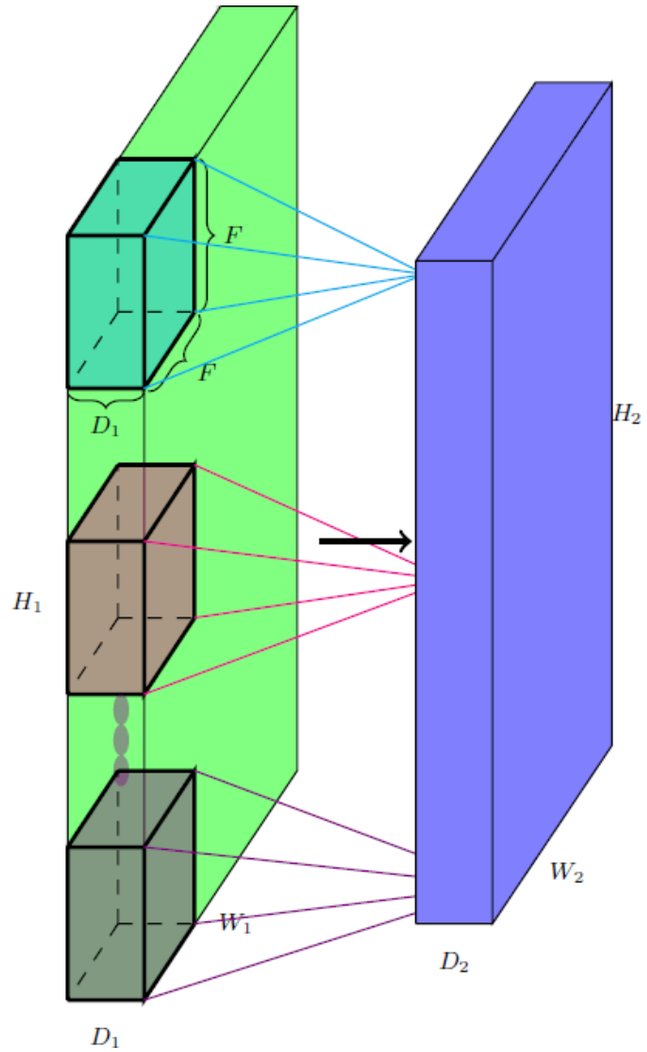
- Input dimensions: Width (W_1) × Height (H_1) × Depth (D_1)
- Spatial extent (F) of each filter (the depth of each filter is same as the depth of input)
- Output dimensions is $W_2 \times H_2 \times D_2$



Convolution: Understanding the (Hyper)Parameters

- Input dimensions: Width (W_1) × Height (H_1) × Depth (D_1)
- Spatial extent (F) of each filter (the depth of each filter is same as the depth of input)
- Output dimensions is $W_2 \times H_2 \times D_2$

- Stride (S) - Number of filters K

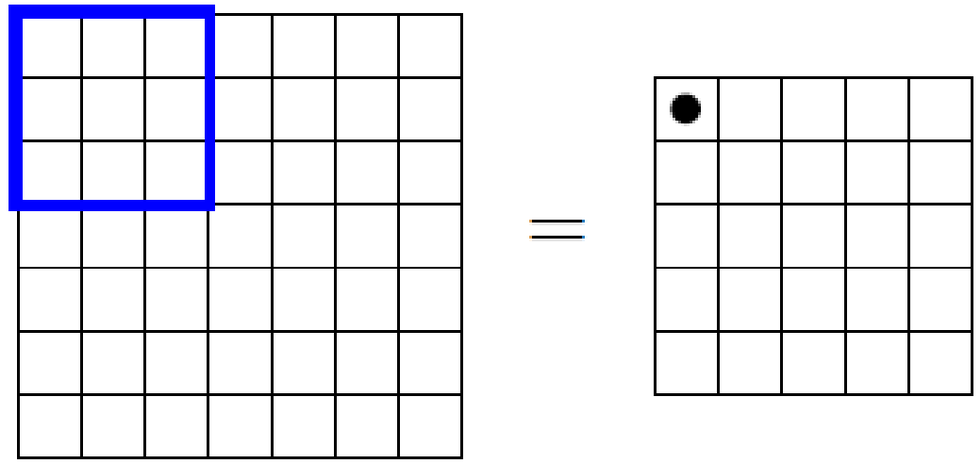


Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output

Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output

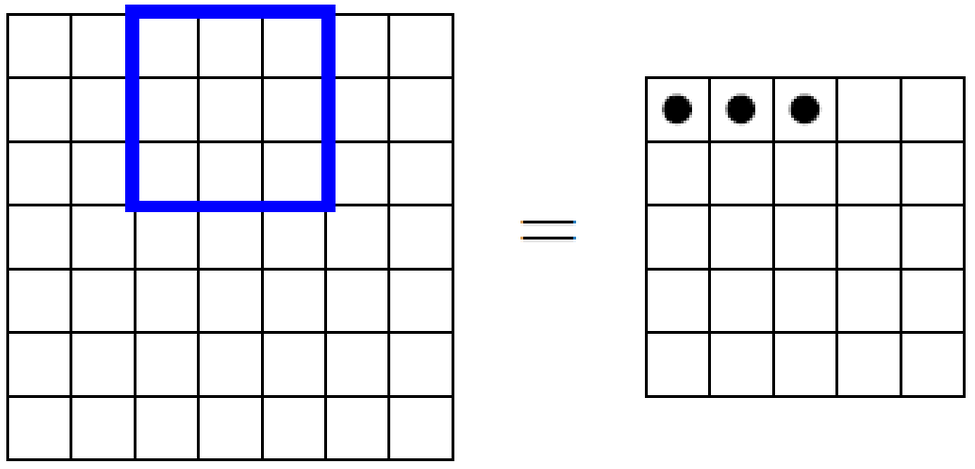


Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output

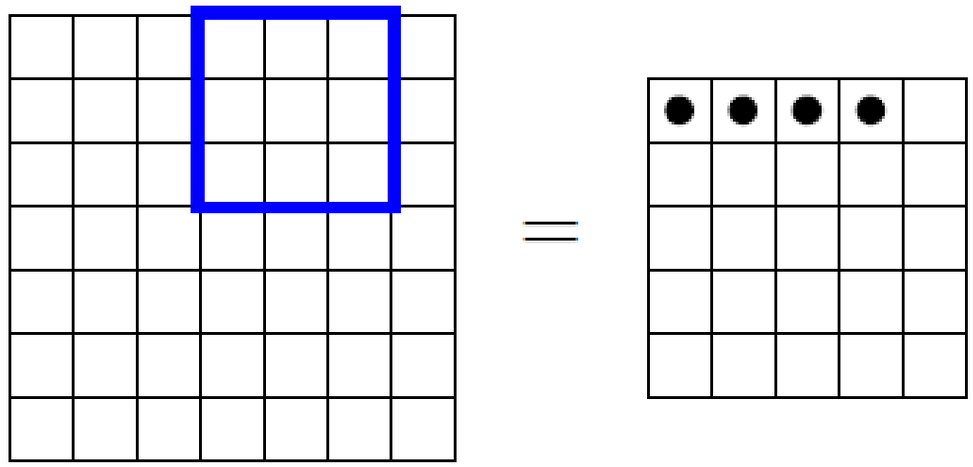
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output



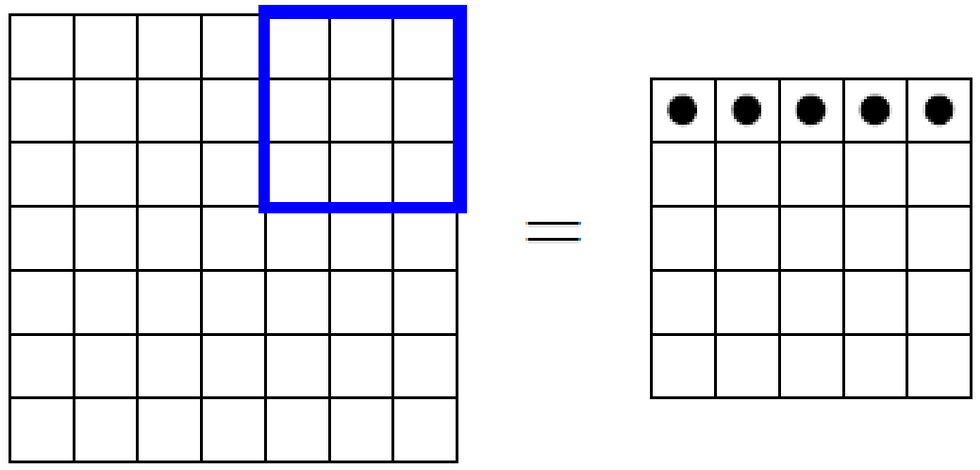
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output



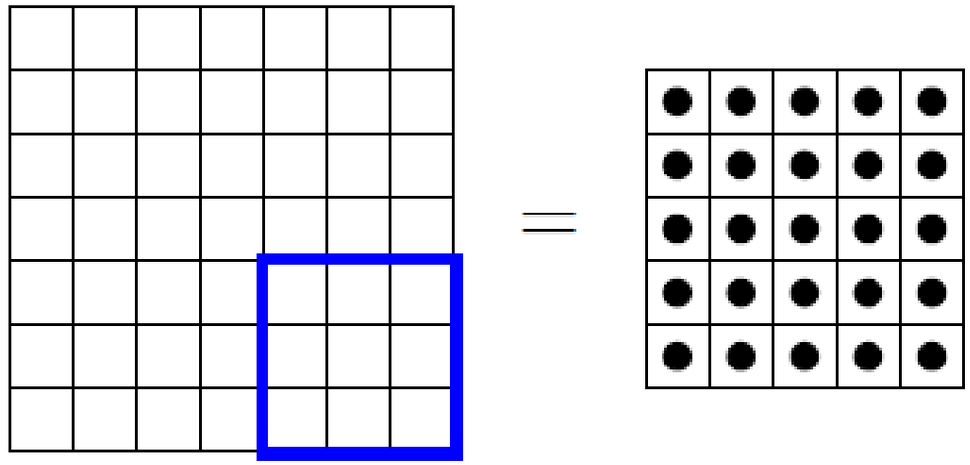
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output



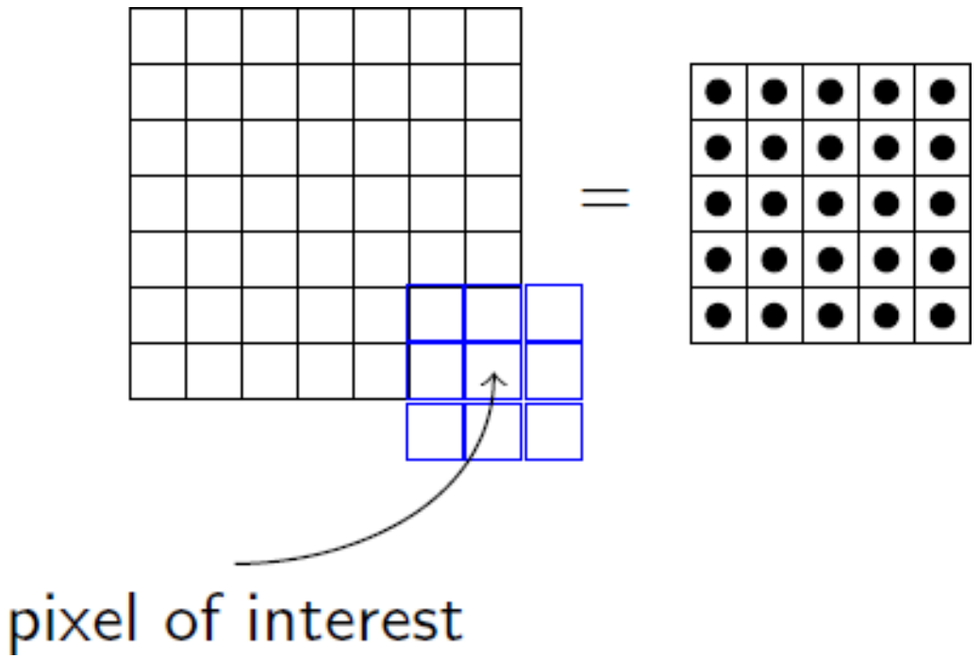
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output



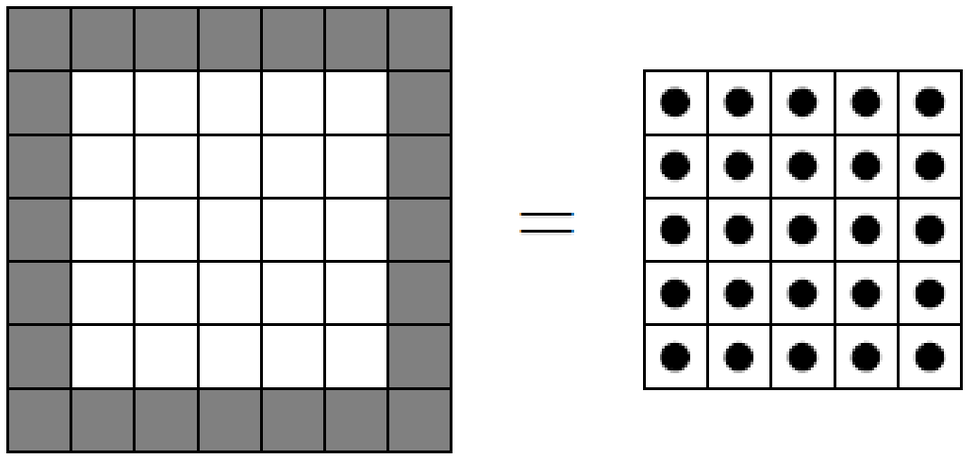
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- we can't place the kernel at corners as it will cross the input boundary



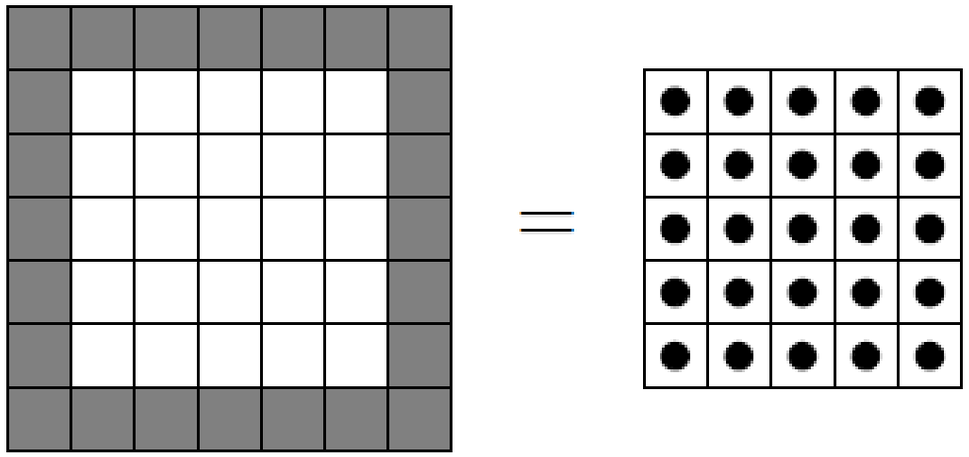
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)



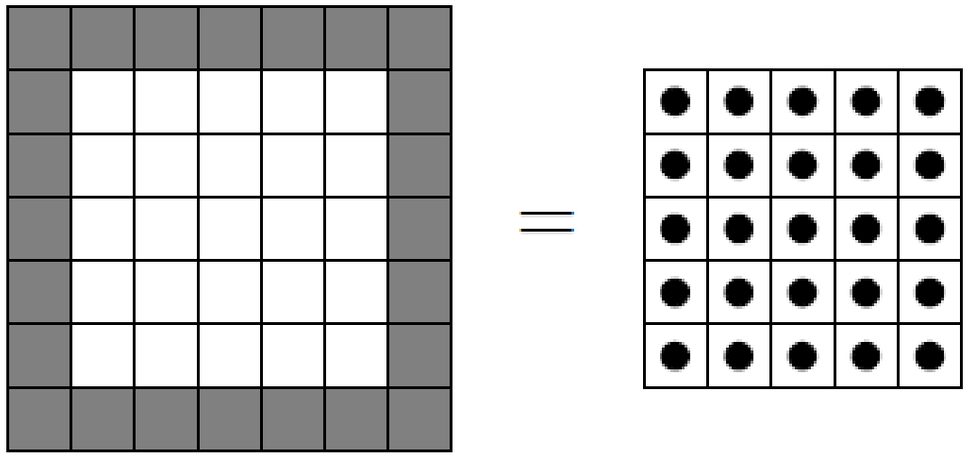
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input



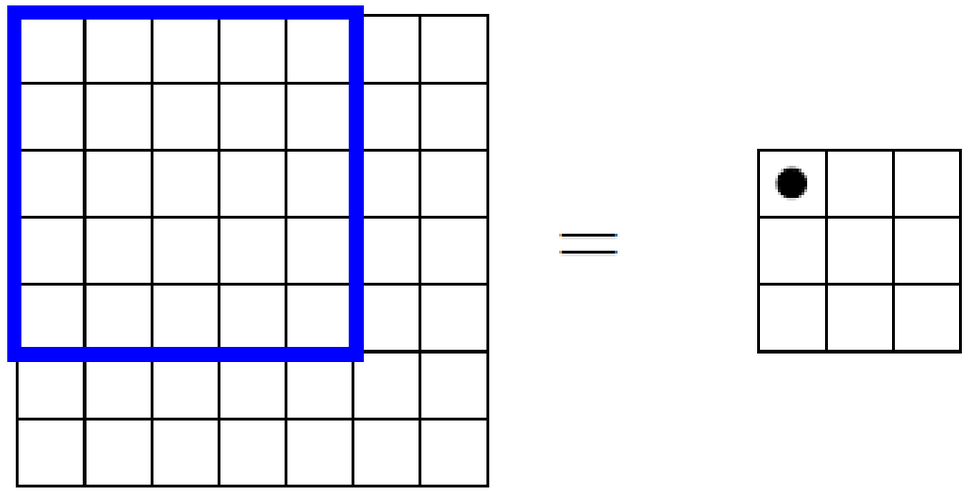
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels



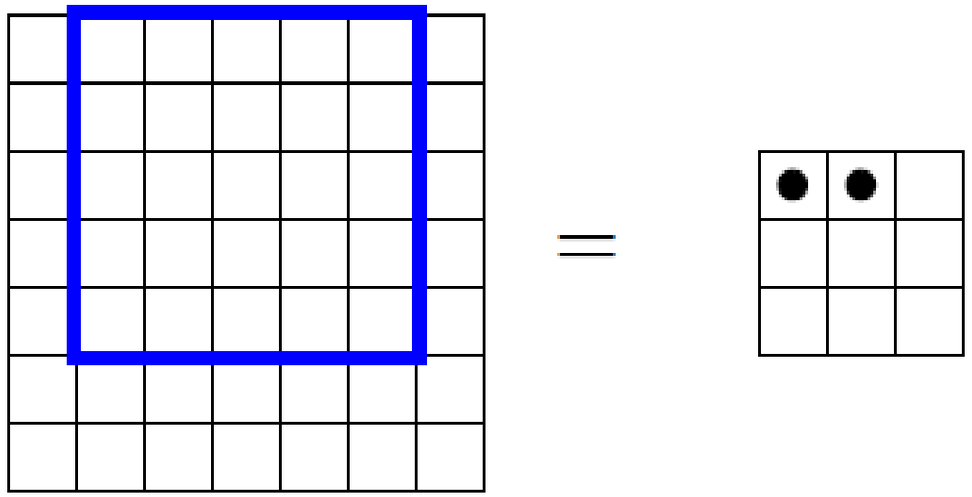
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel



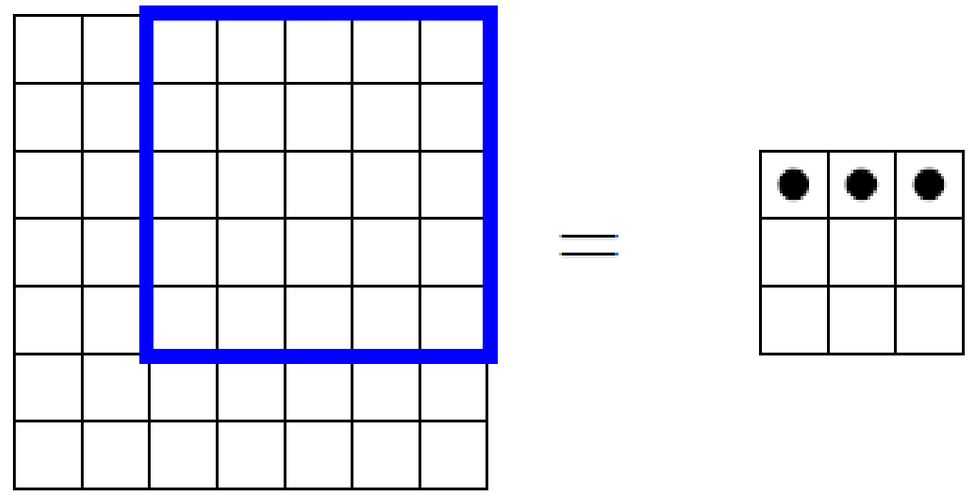
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel
- We have an even smaller output now



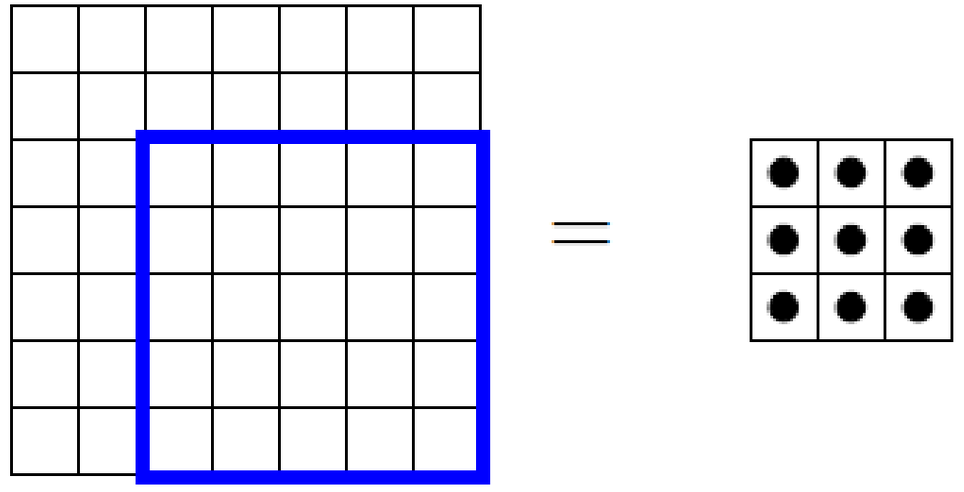
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel
- We have an even smaller output now



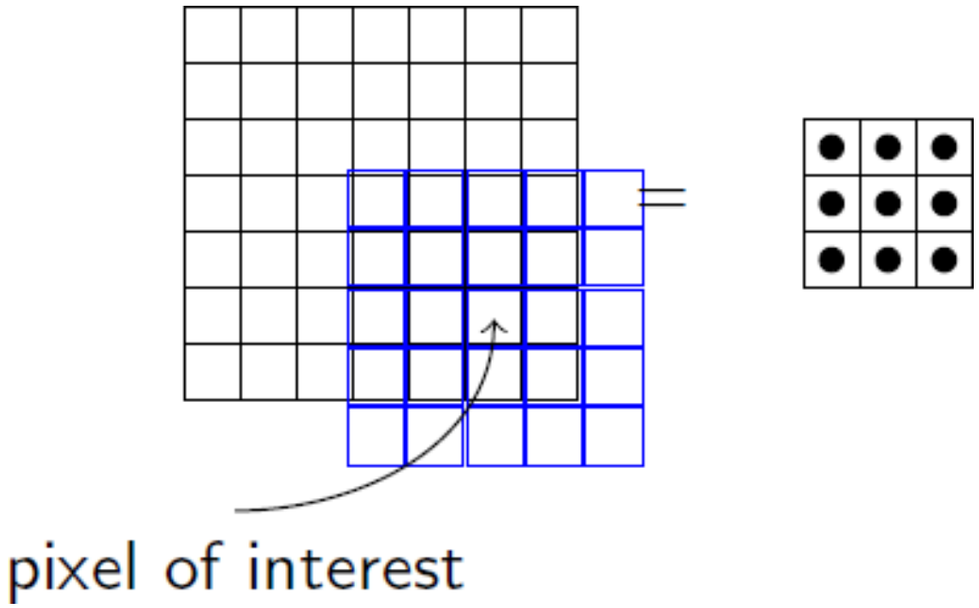
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel
- We have an even smaller output now



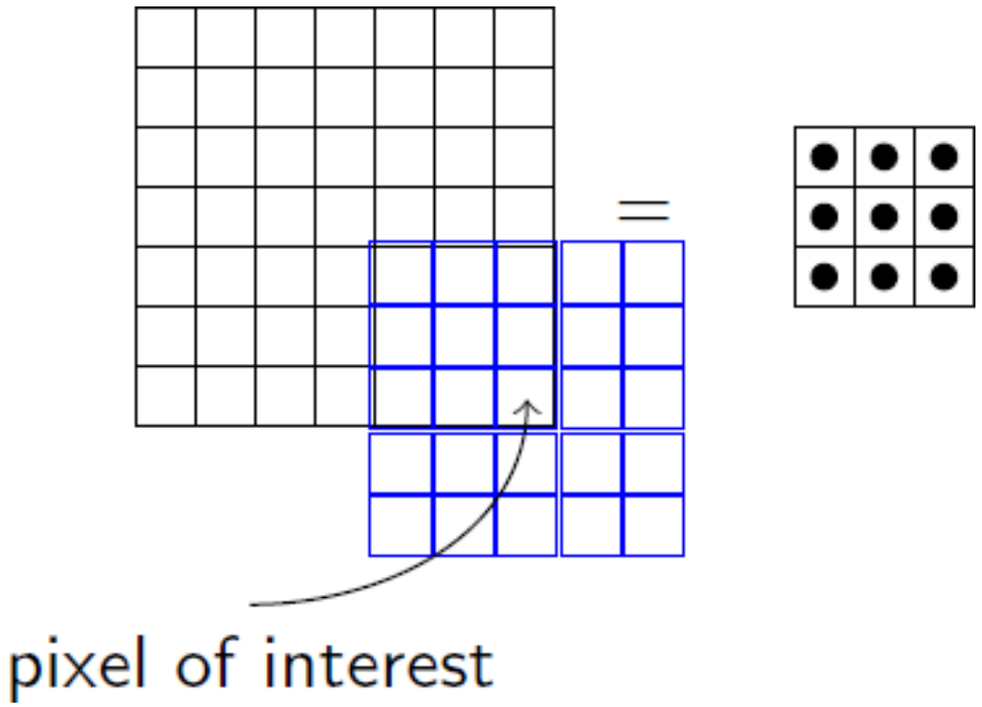
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel
- We have an even smaller output now



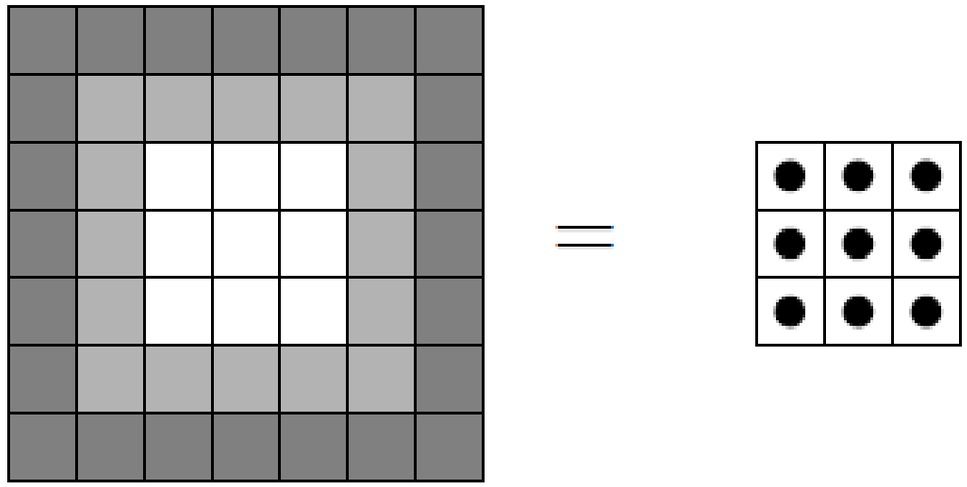
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel
- We have an even smaller output now



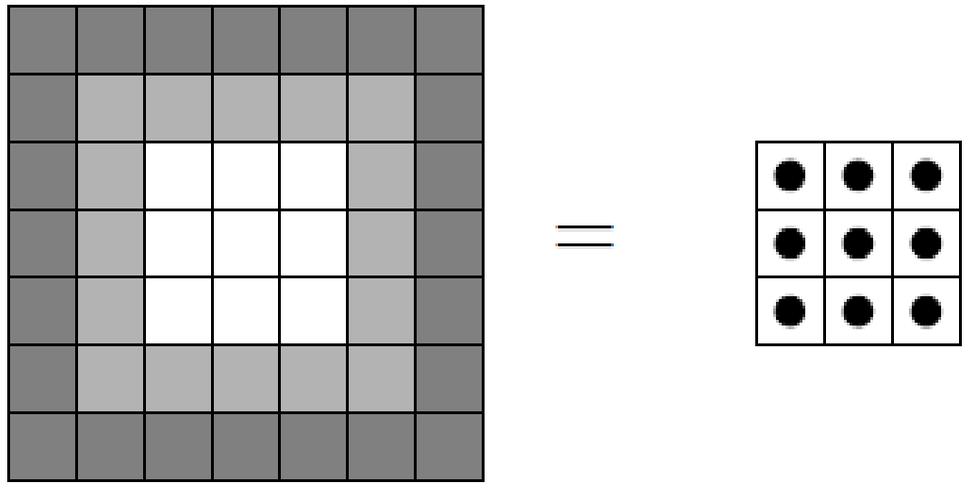
Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel
- We have an even smaller output now



Convolution: Understanding the (Hyper)Parameters

- Let us compute dimensions (W_2, H_2) of output
- Recall that we can't place the kernel at corners as it will cross the input boundary
- This is true for all shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than input
- As size of kernel increases, this becomes true for even more pixels
- For example, let's consider a 5 × 5 kernel
- We have an even smaller output now



In general,

$$W_2 = W_1 - F + 1$$
$$H_2 = H_1 - F + 1$$

We will refine this formula further

Convolution: Understanding the (Hyper)Parameters

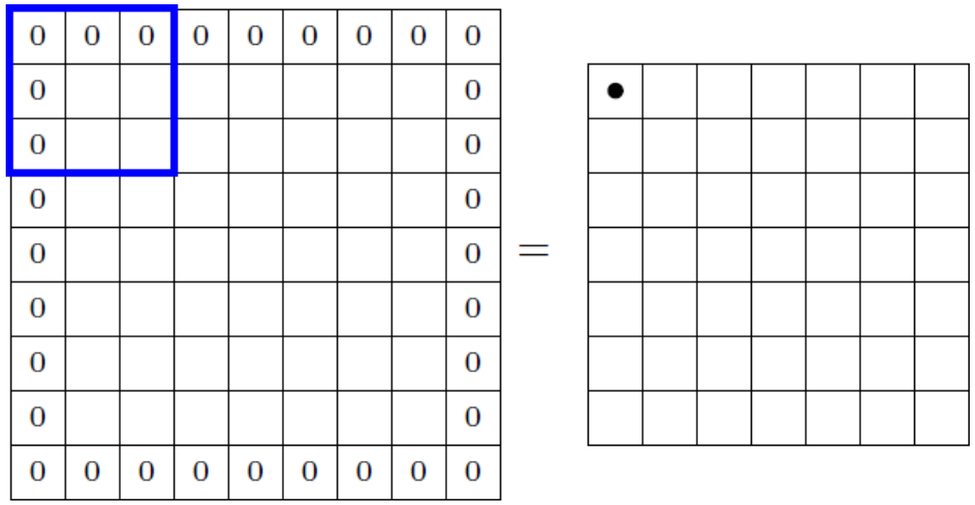
- What if we want output to be of same size as input?

Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners

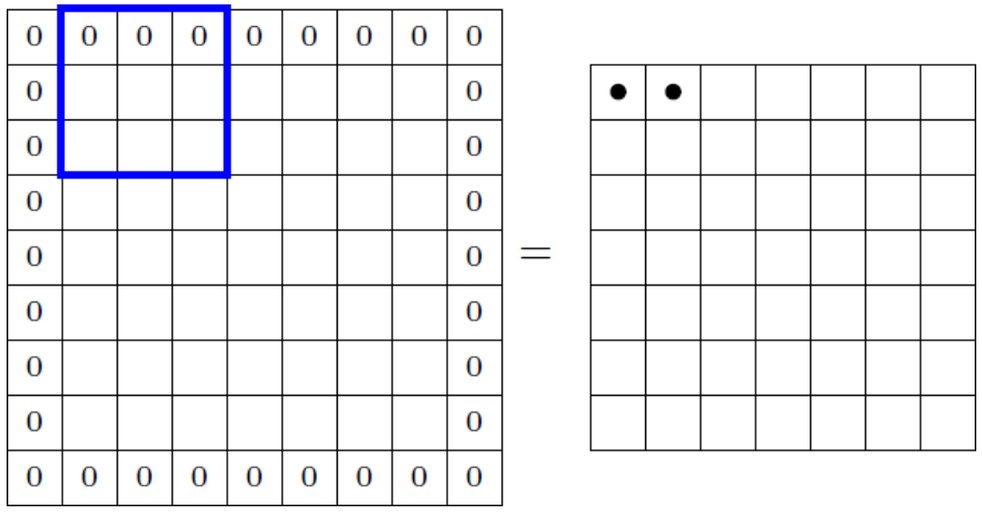
Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- Recall use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners
- Let us use pad P = 1 with a 3 × 3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right;



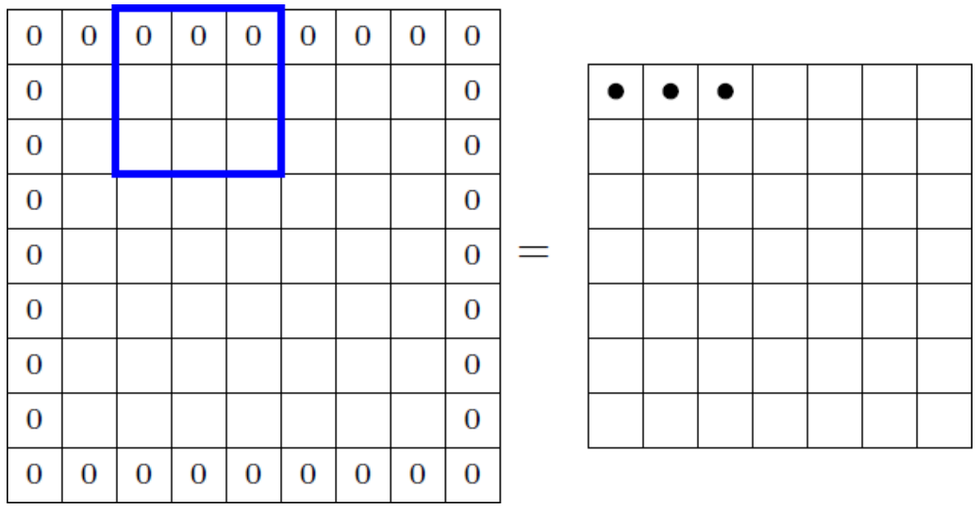
Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- Recall use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners
- Let us use pad P = 1 with a 3 × 3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right;



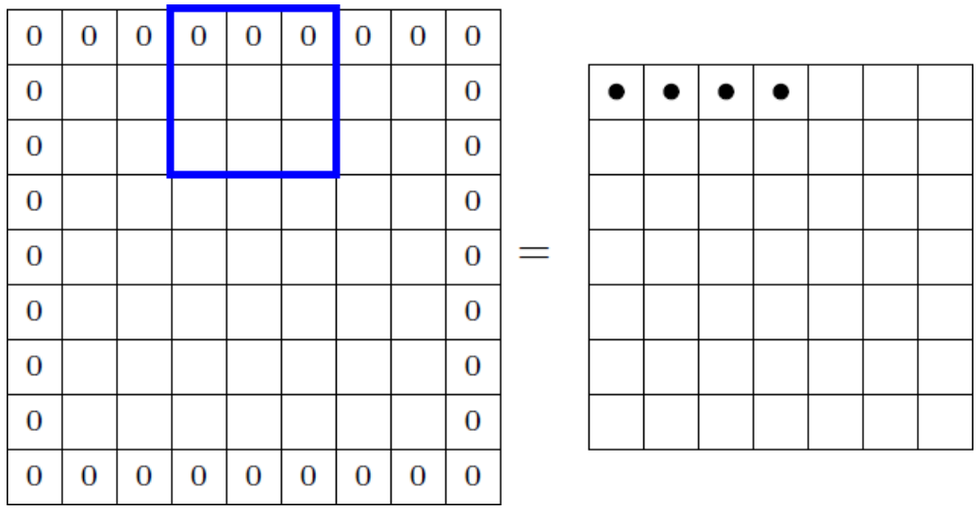
Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- Recall use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners
- Let us use pad P = 1 with a 3 × 3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right;



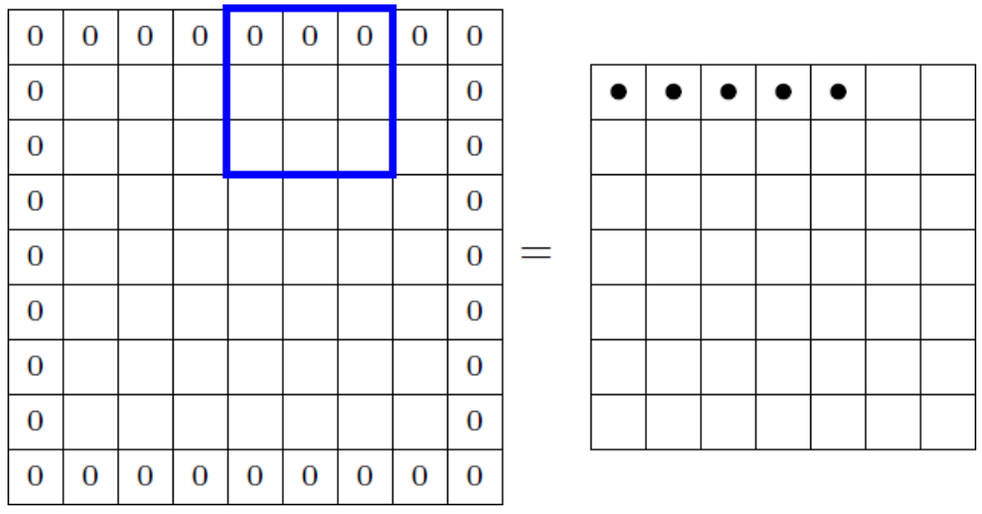
Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- Recall use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners
- Let us use pad P = 1 with a 3 × 3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right;



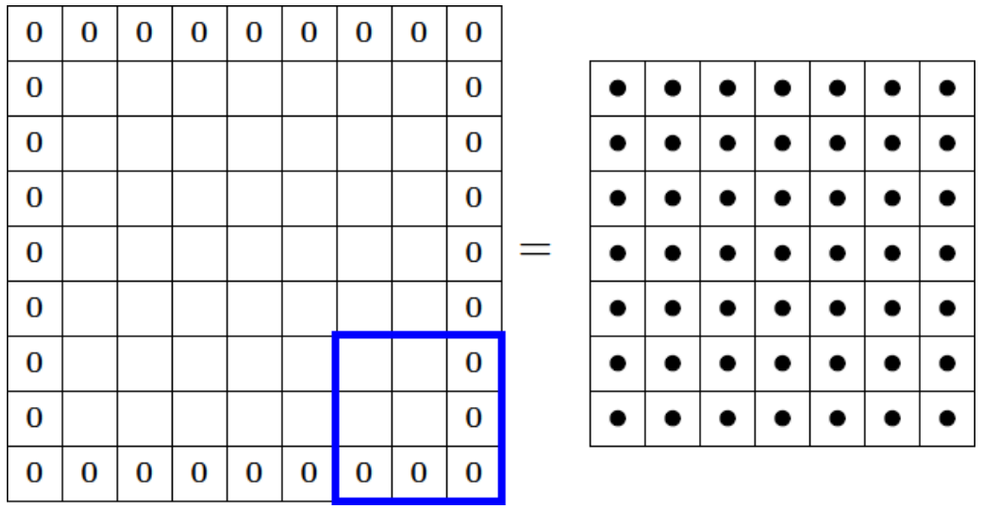
Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- Recall use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners
- Let us use pad P = 1 with a 3 × 3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right;



Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- Recall use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners
- Let us use pad P = 1 with a 3 × 3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right;



Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- Recall use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners
- Let us use pad $P = 1$ with a 3×3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right;

Convolution: Understanding the (Hyper)Parameters

- What if we want output to be of same size as input?
- Recall use of **padding**
- Pad inputs with appropriate number of inputs so you can now apply kernel at corners
- Let us use pad P = 1 with a 3 × 3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right;

We now have:

$$W_2 = W_1 - F + 2P + 1$$
$$H_2 = H_1 - F + 2P + 1$$

We will refine this formula further

Convolution: Understanding the (Hyper)Parameters

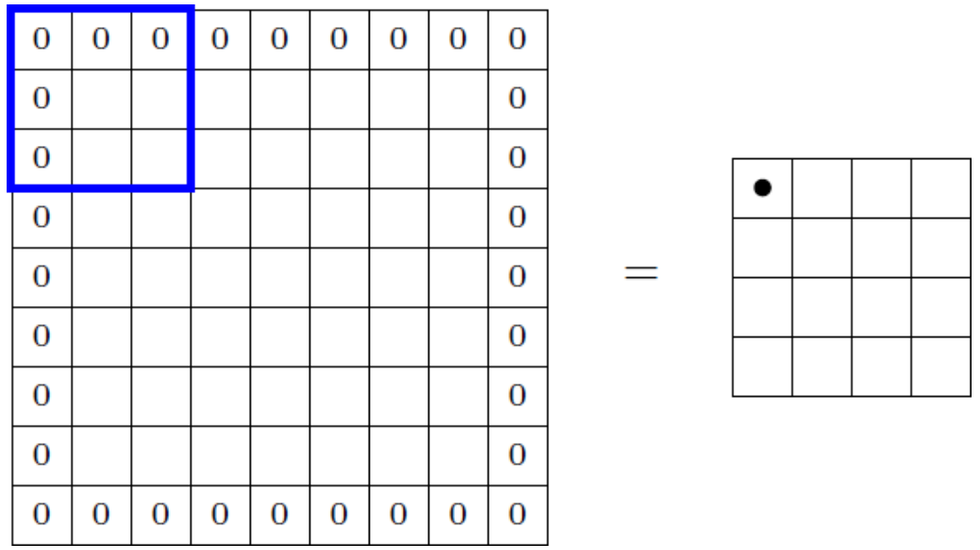
- What does **stride** S do?

Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)

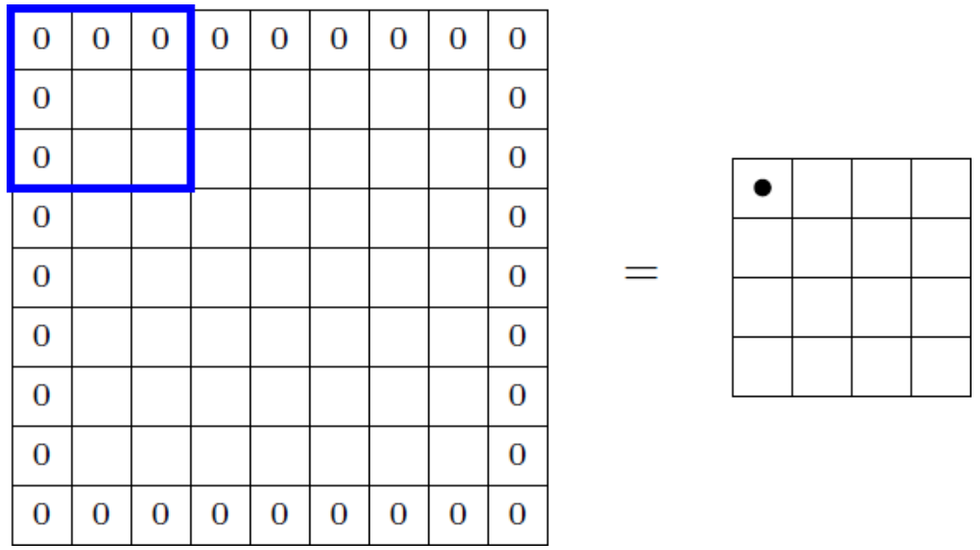
Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



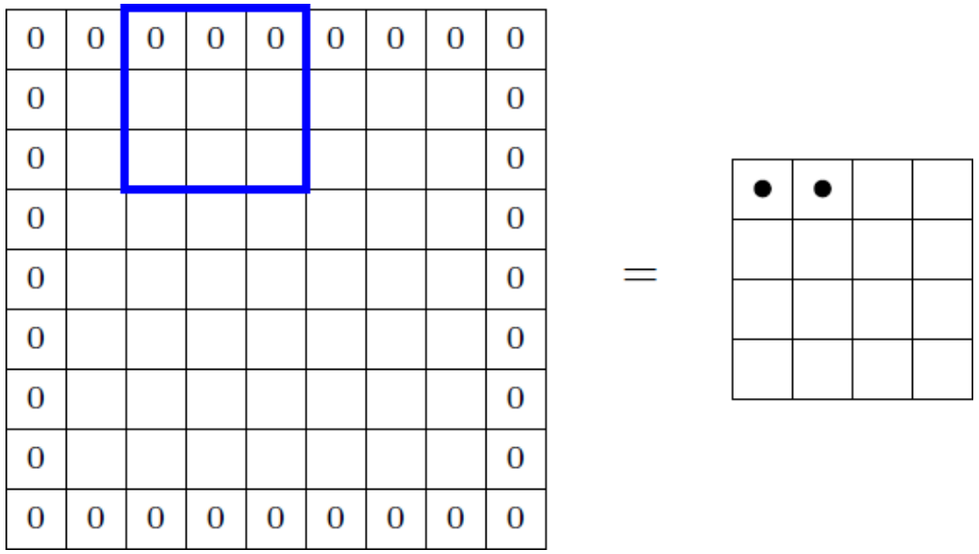
Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



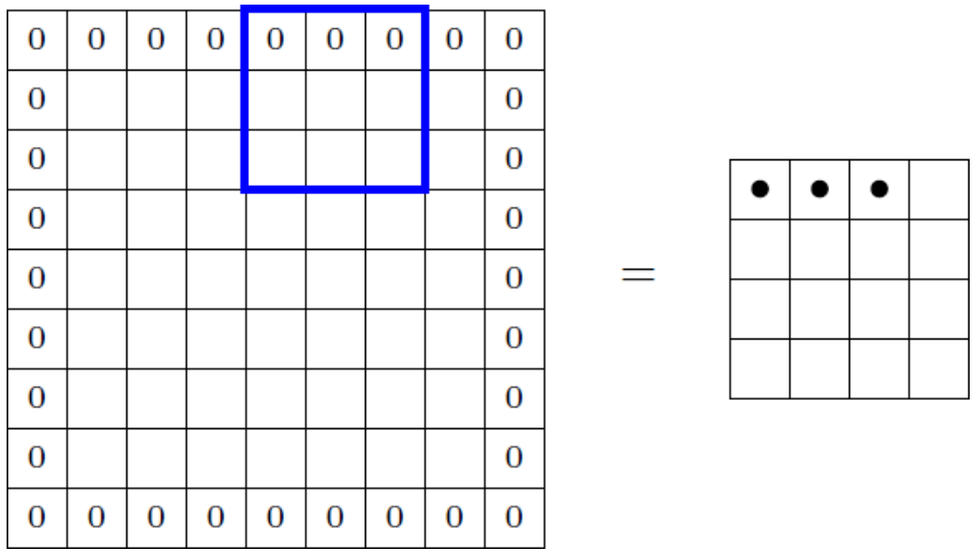
Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



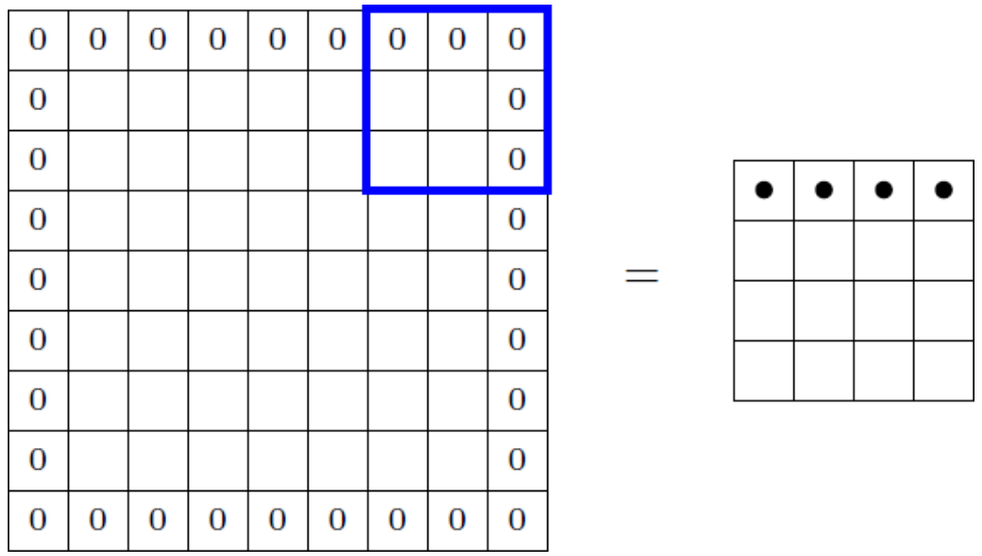
Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



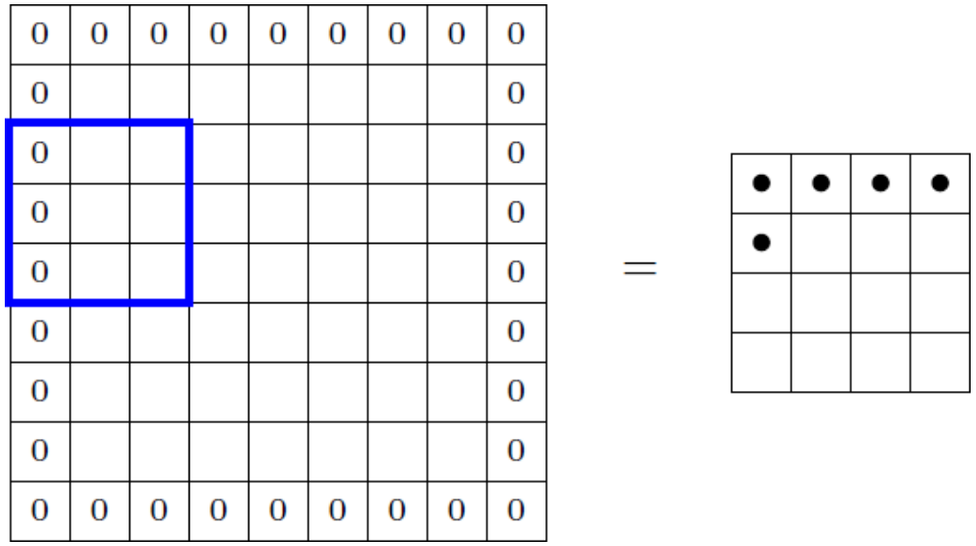
Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



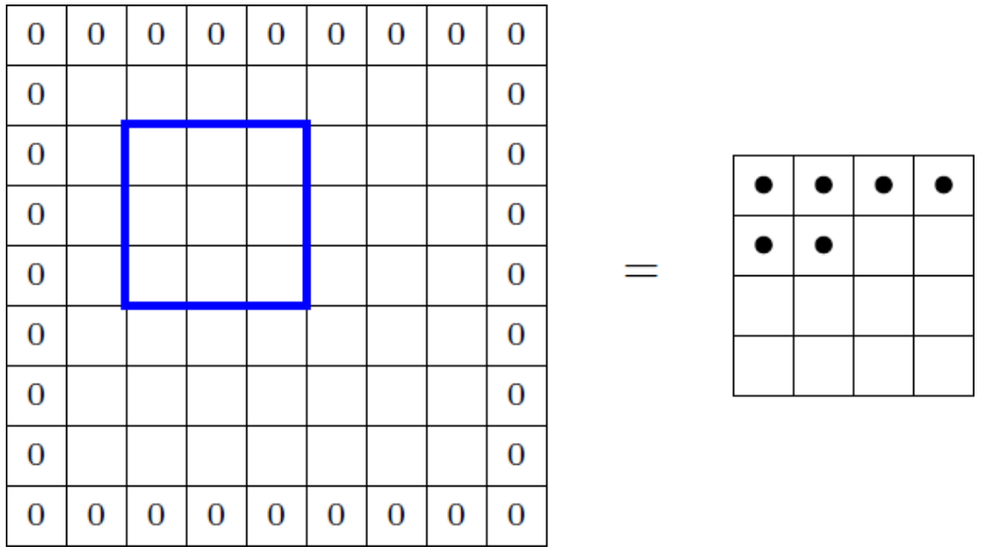
Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



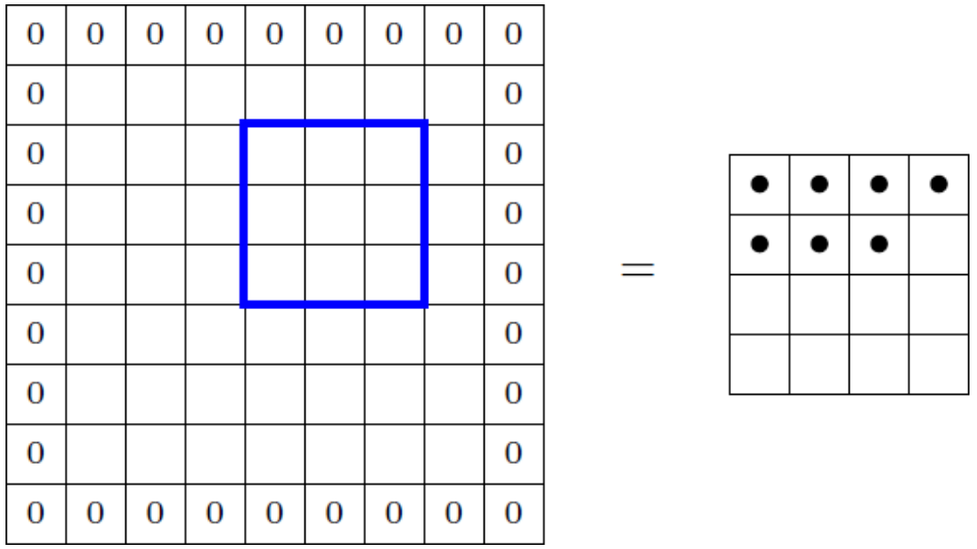
Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



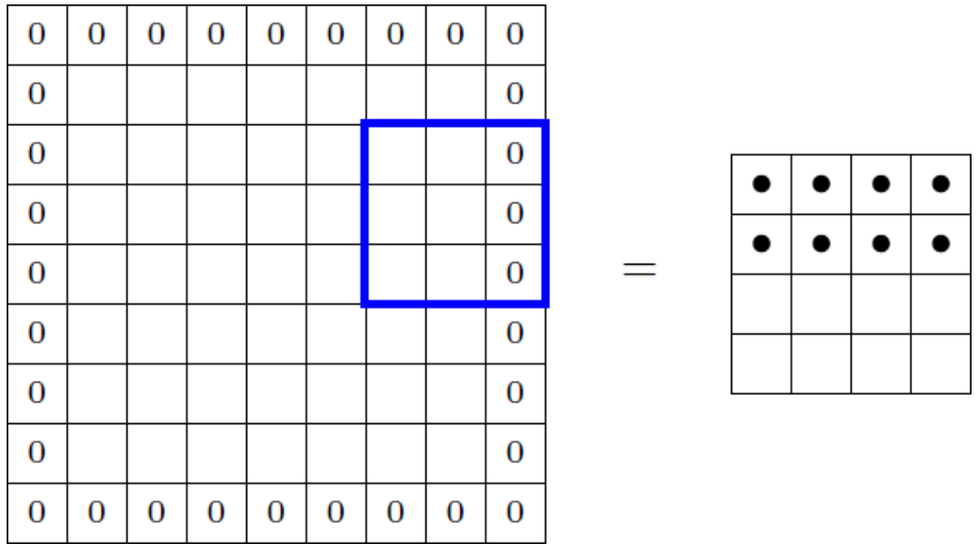
Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions

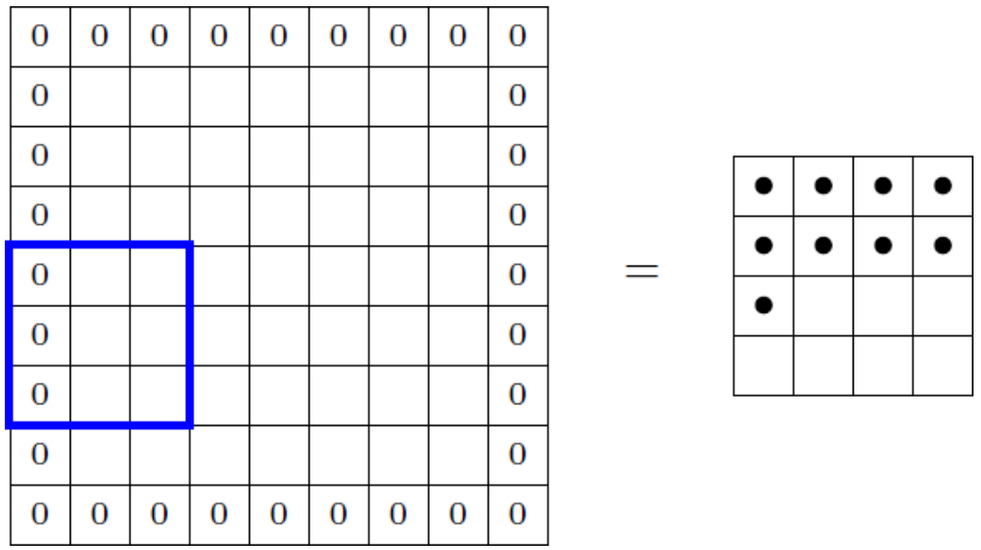


Convolution: Understanding the (Hyper)Parameters

What does **stride** S do?

It defines the intervals at which the filter is applied (here $S = 2$)

Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions

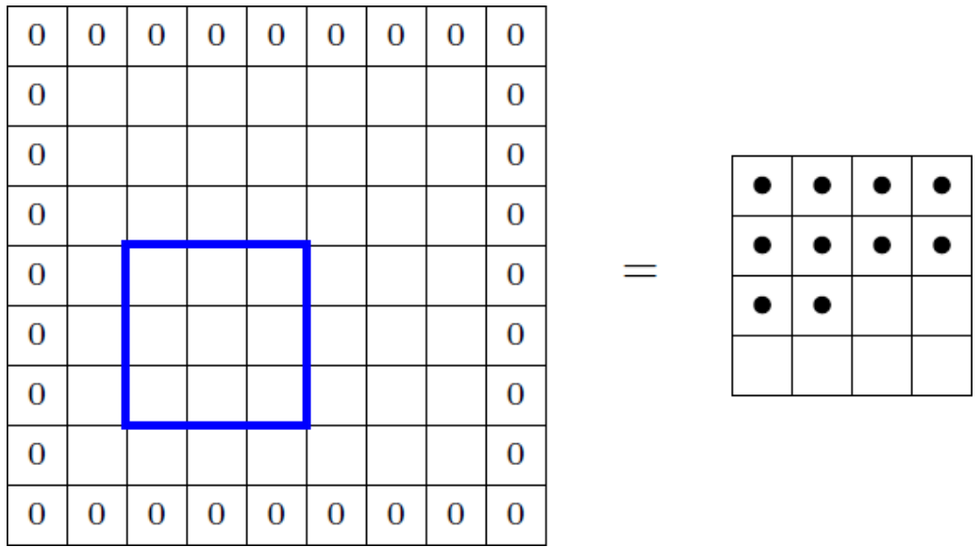


Convolution: Understanding the (Hyper)Parameters

What does **stride** S do?

It defines the intervals at which the filter is applied (here $S = 2$)

Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions

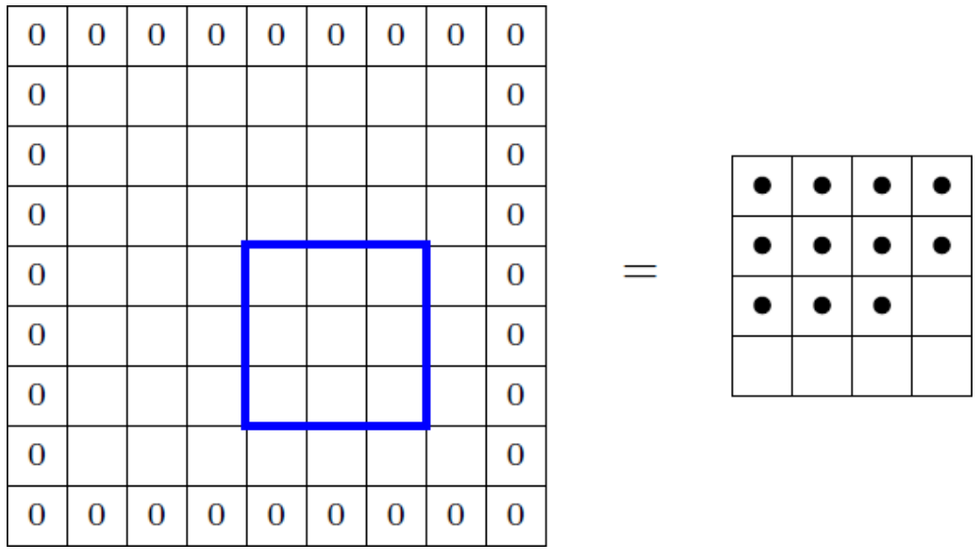


Convolution: Understanding the (Hyper)Parameters

What does **stride** S do?

It defines the intervals at which the filter is applied (here $S = 2$)

Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions

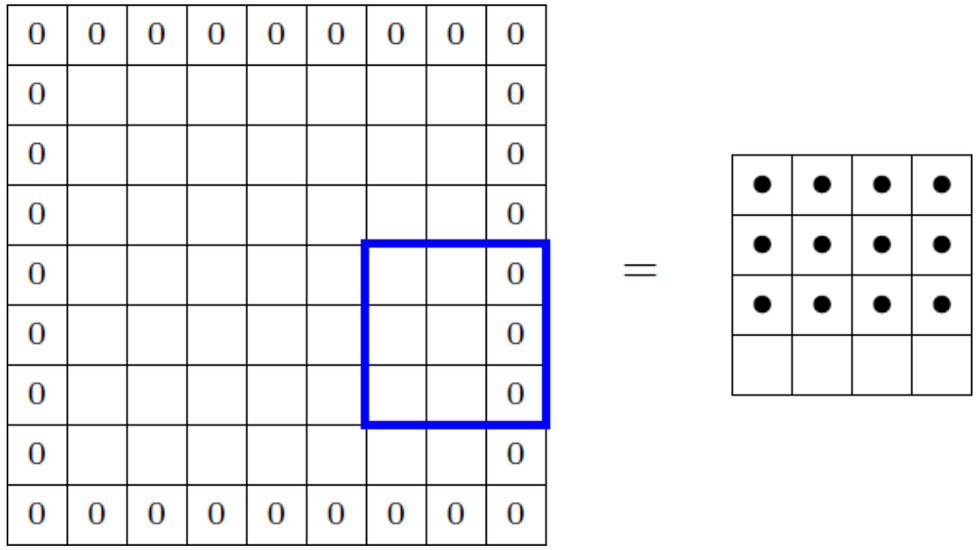


Convolution: Understanding the (Hyper)Parameters

What does **stride** S do?

It defines the intervals at which the filter is applied (here $S = 2$)

Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions

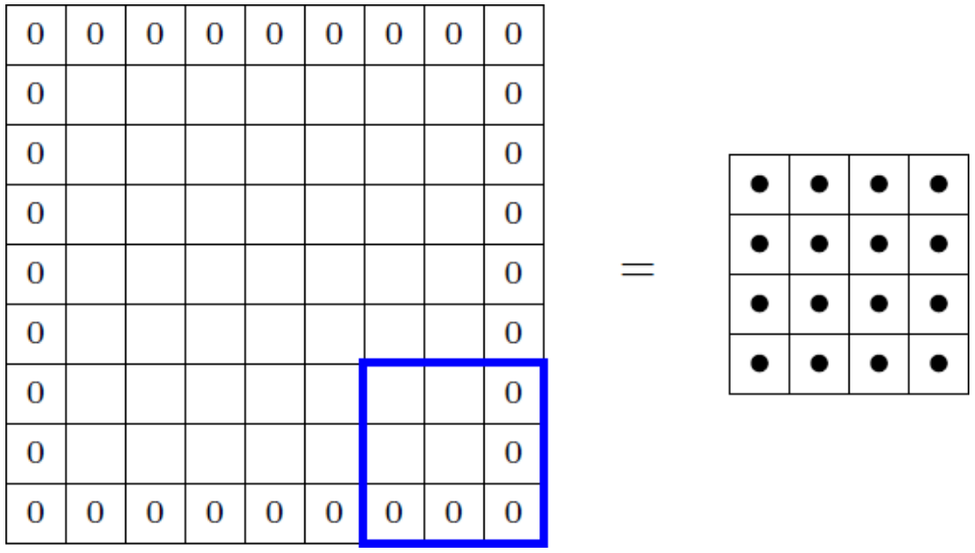


Convolution: Understanding the (Hyper)Parameters

What does **stride** S do?

It defines the intervals at which the filter is applied (here $S = 2$)

Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions



Convolution: Understanding the (Hyper)Parameters

- What does **stride** S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skip every 2nd pixel ($S = 2$) which will result in an output of smaller dimensions

So our final formula should mostly look like,

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

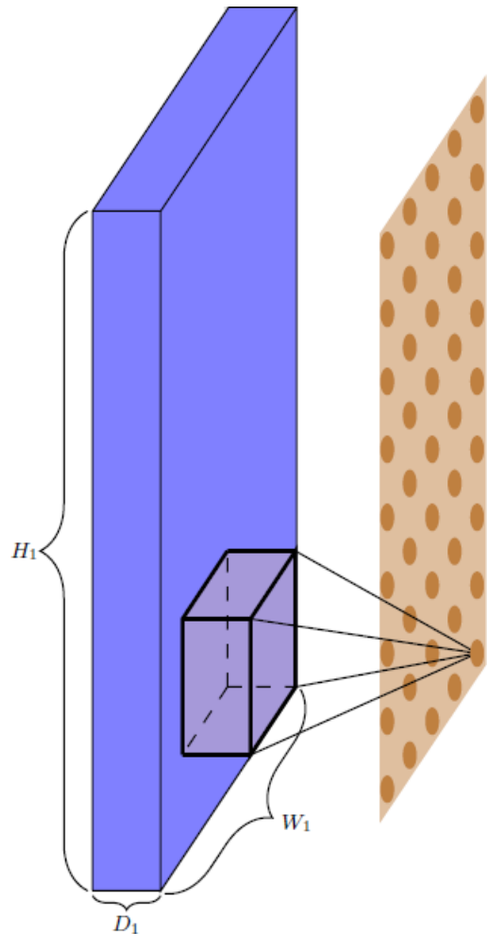
Not done yet, we will refine this formula further!

Convolution: Understanding the (Hyper)Parameters

- Finally, coming to depth of output

Convolution: Understanding the (Hyper)Parameters

- Finally, coming to depth of output
- Each filter gives us one 2D output

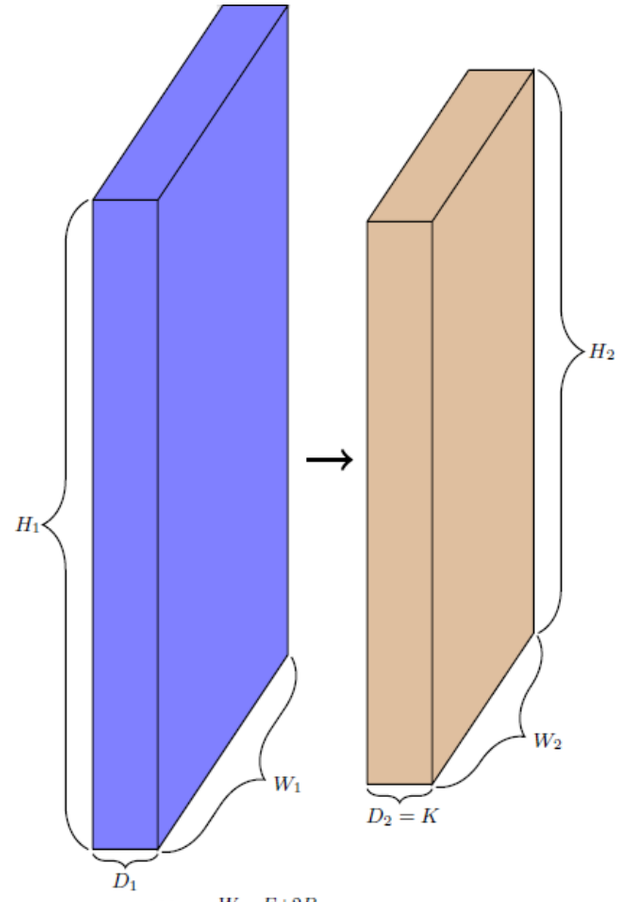


Convolution: Understanding the (Hyper)Parameters

- Finally, coming to depth of output
- Each filter gives us one 2D output

  K filters will give us K such 2D outputs
- We can think of resulting output as

  $K \times W_2 \times H_2$ volume
- Thus $D_2 = K$



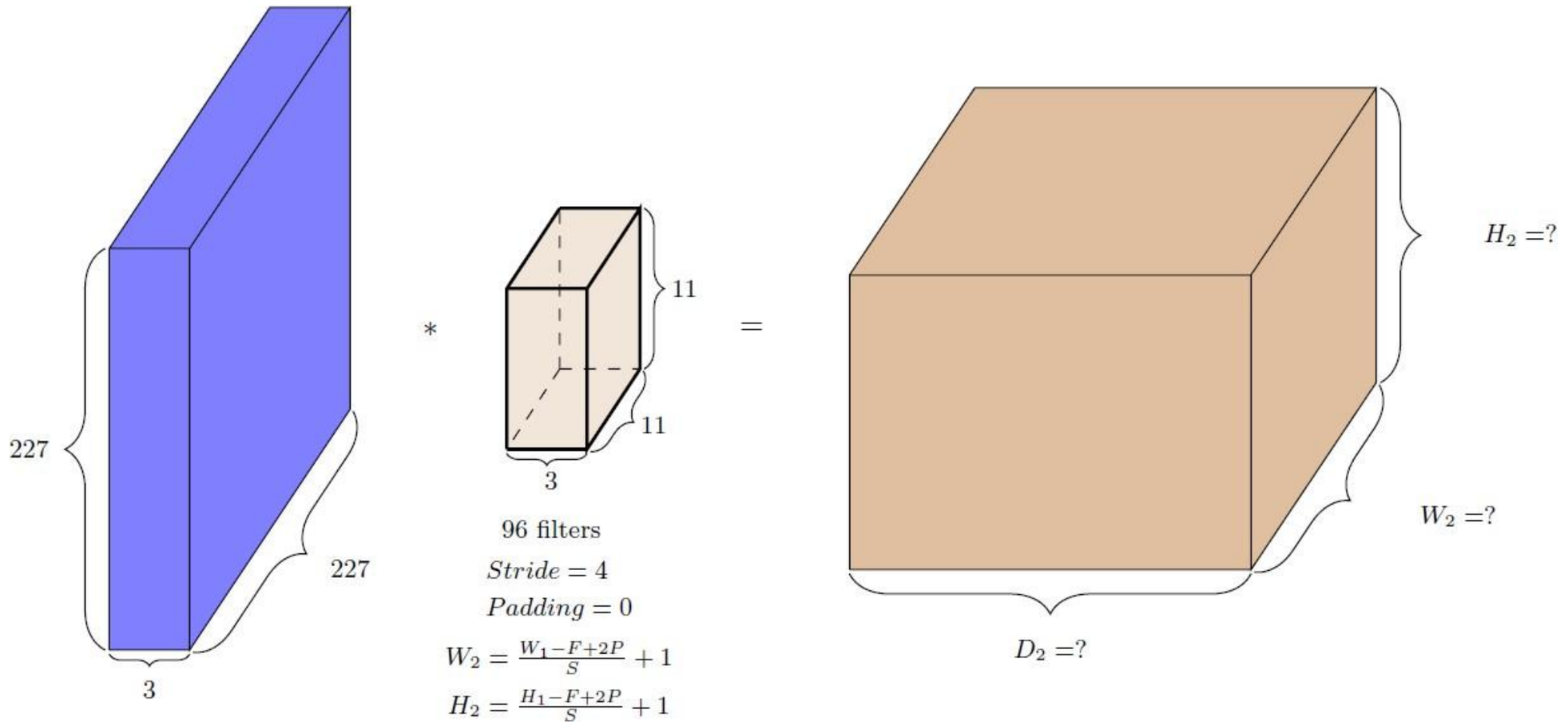
$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

$$D_2 = K$$

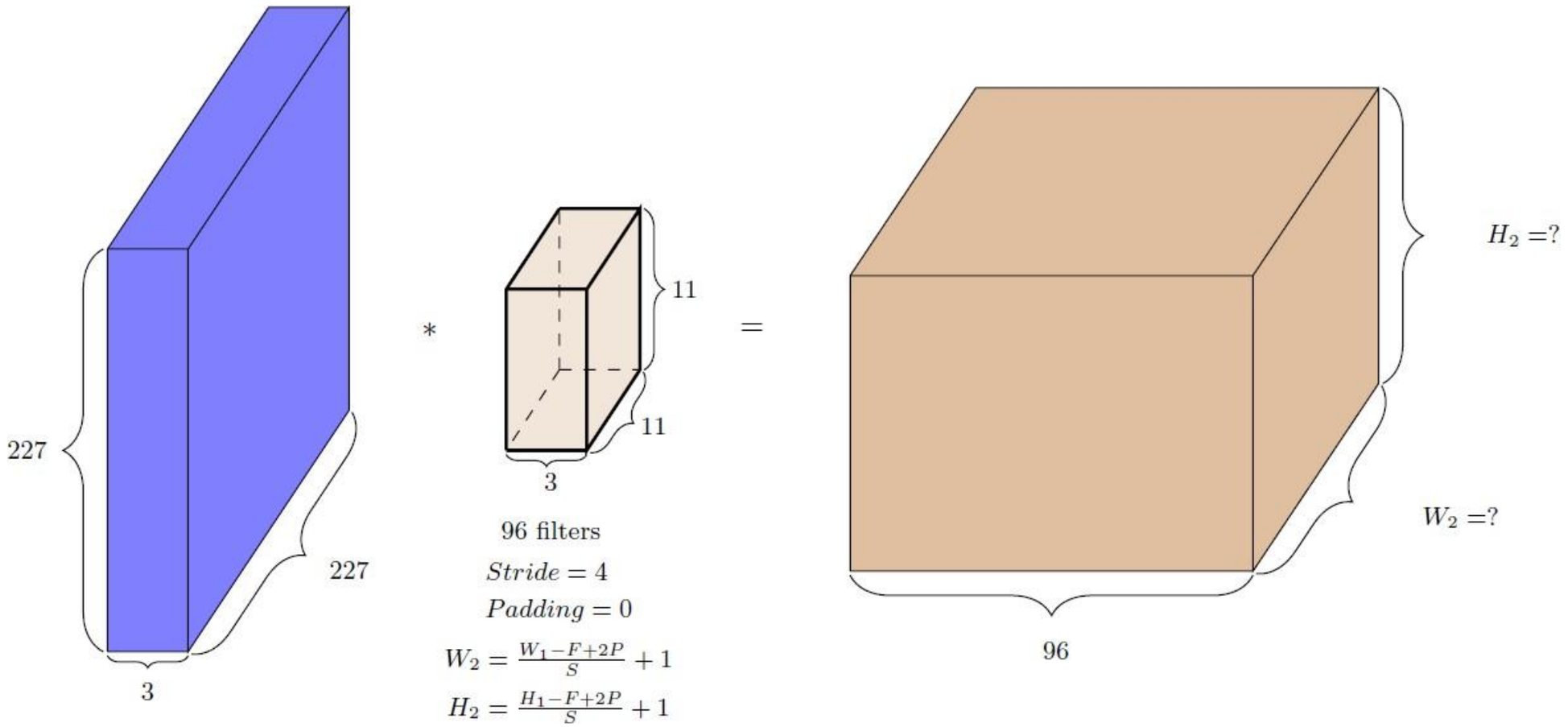
Quick Exercise

Work out output dimensions for the setting below!



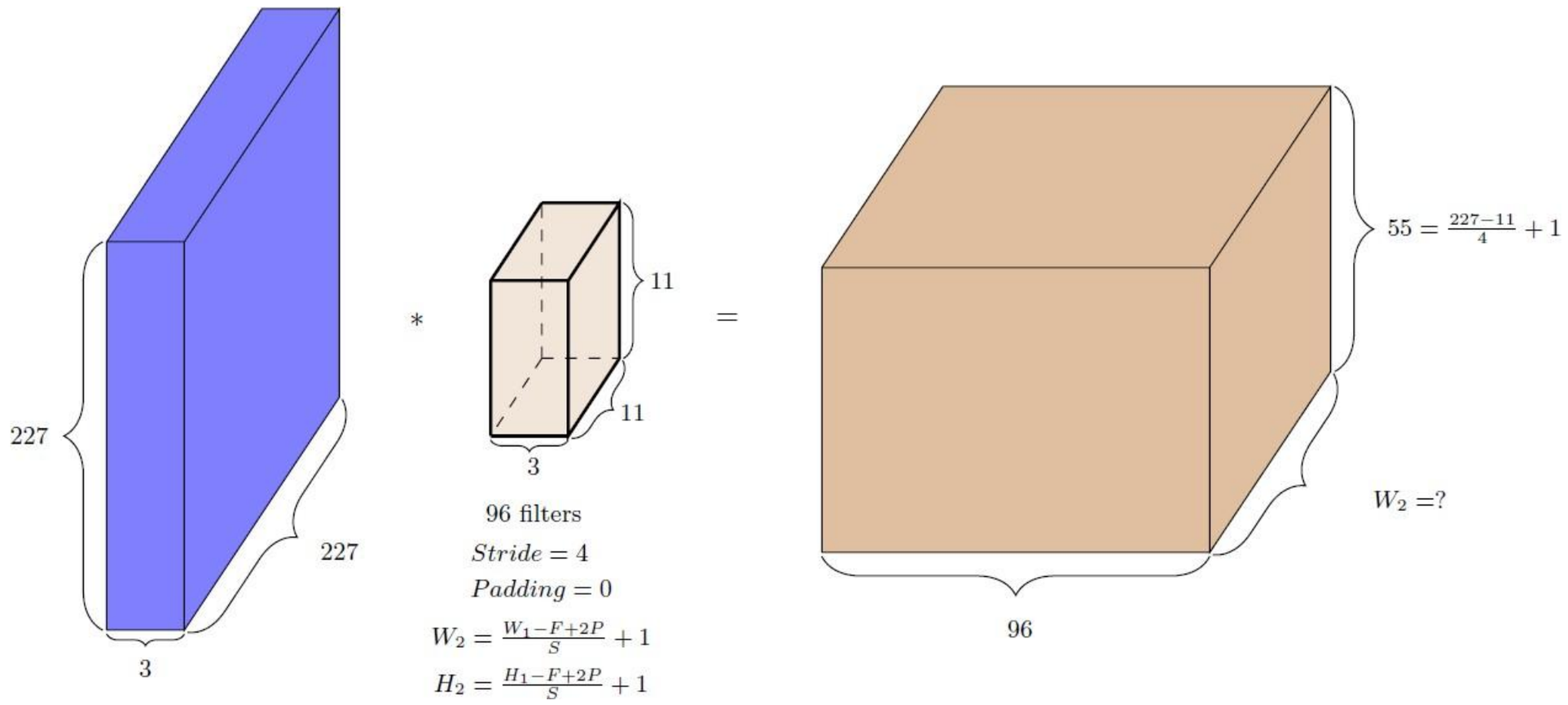
Quick Exercise

Work out output dimensions for the setting below!



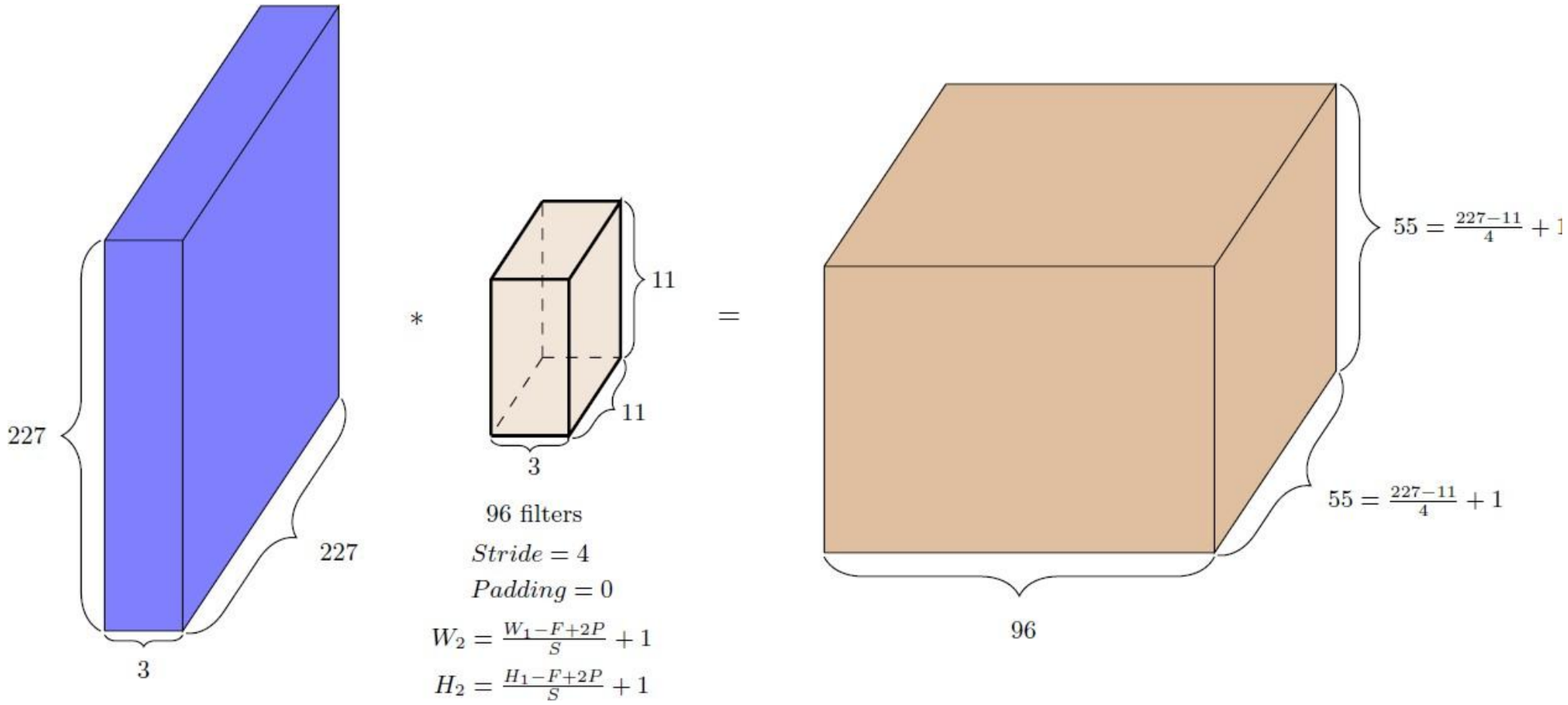
Quick Exercise

Work out output dimensions for the setting below!



Quick Exercise

Work out output dimensions for the setting below!

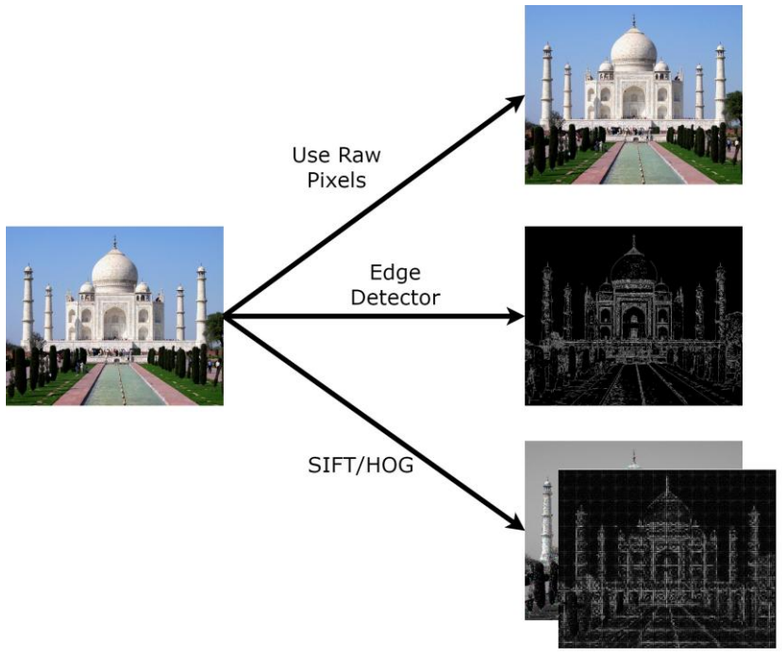


Pause and Ponder

- What is the connection between convolution and neural networks? Won't feedforward neural networks do?
- We will try to understand this by considering the task of "image classification"

Traditional Machine Learning for Vision

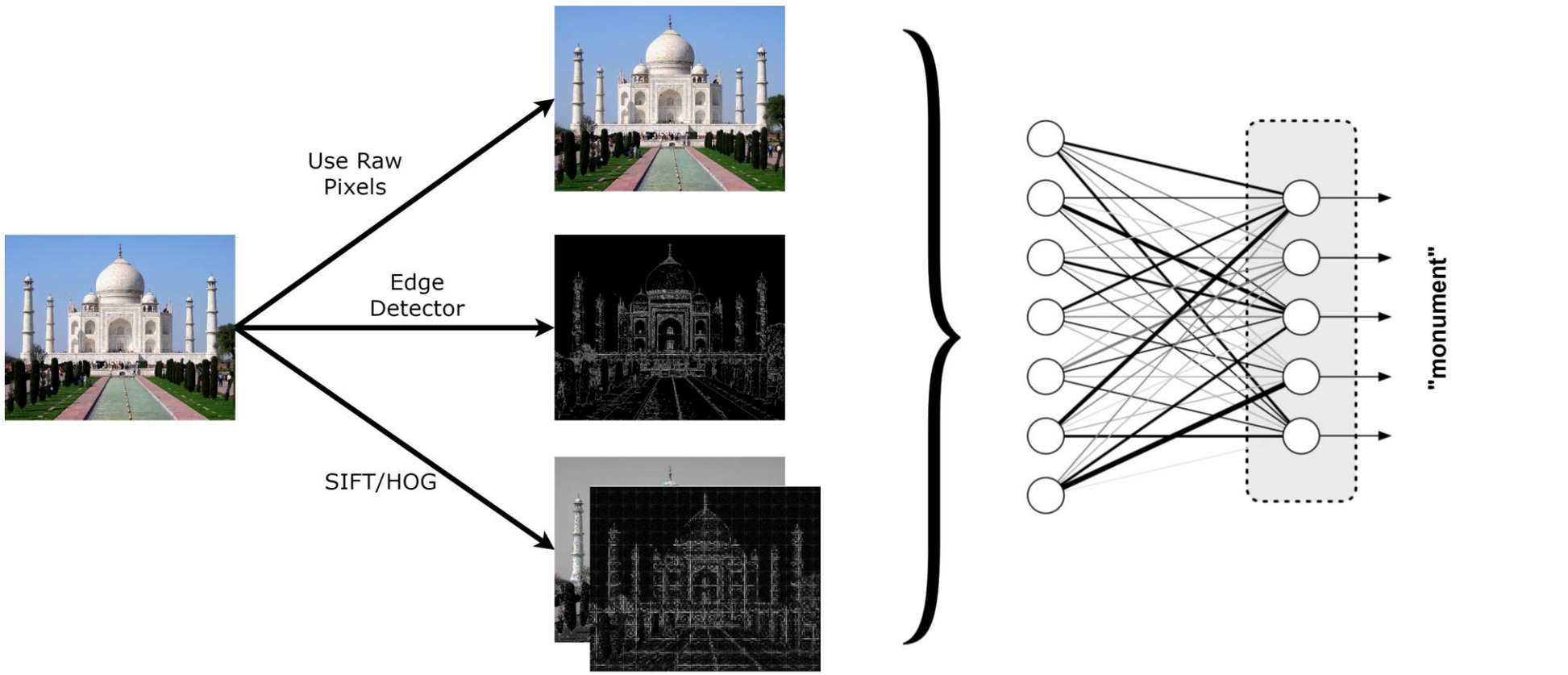
- Traditional ML-based computer vision solutions involve static feature engineering from images (e.g. recall SIFT, LBP, HoG, etc)
- Though effective, static feature engineering was a bottleneck of pre-DL vision solutions



Static Feature Engineering (No Learning)

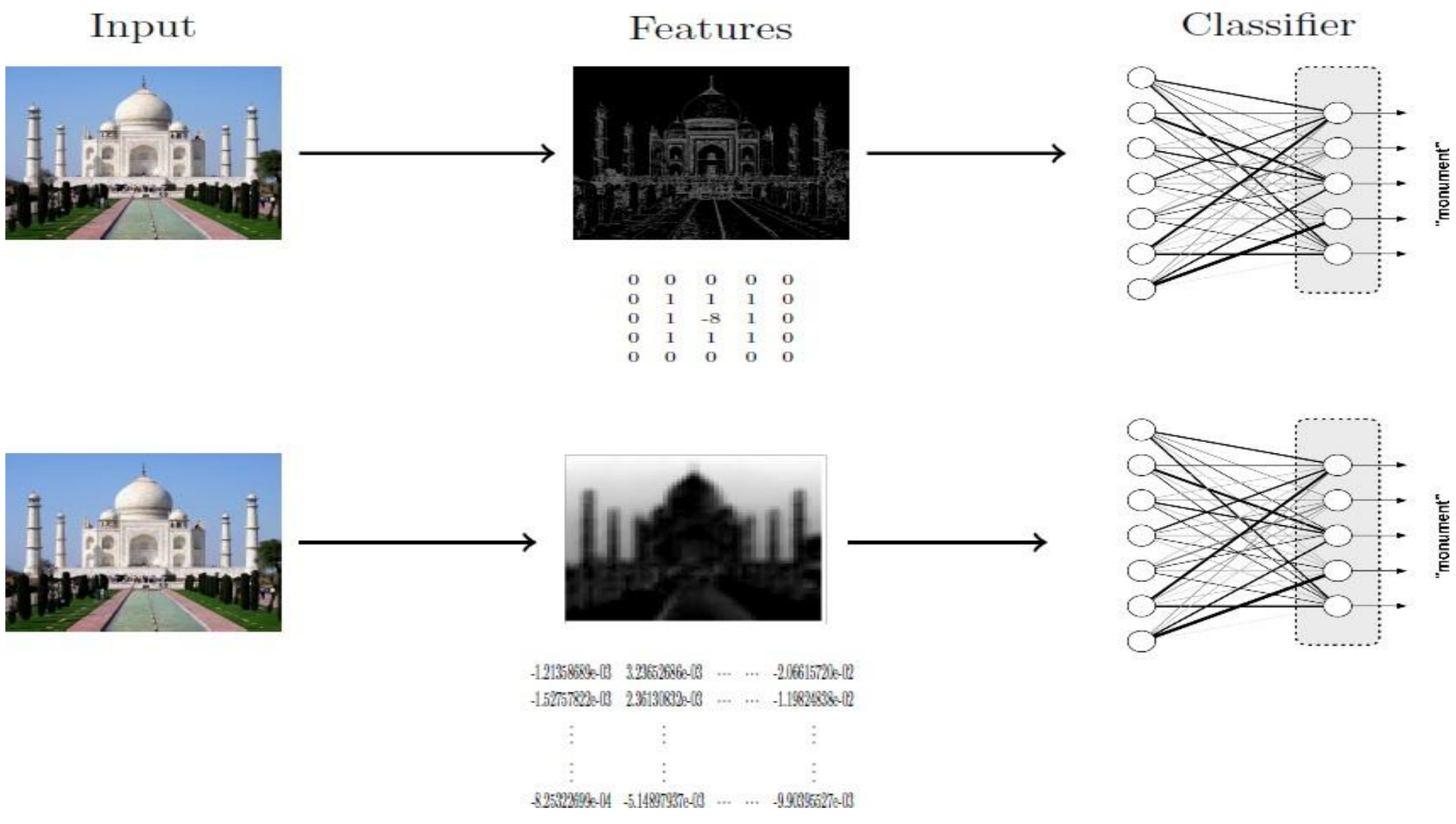
Traditional Machine Learning for Vision

- Traditional ML-based computer vision solutions involve static feature engineering from images (e.g. recall SIFT, LBP, HoG, etc)
- Though effective, static feature engineering was a bottleneck of pre-DL vision solutions



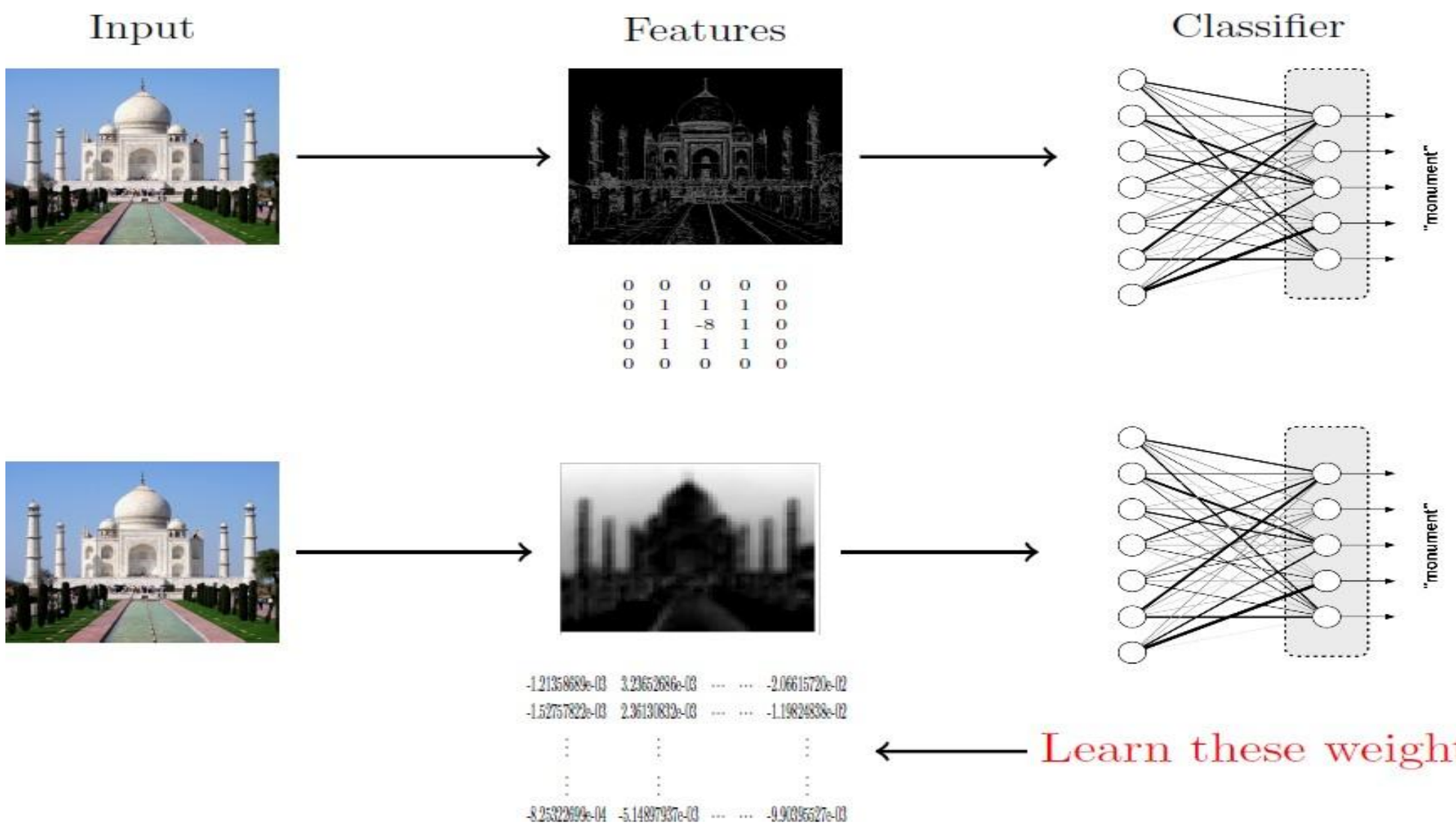
Beyond Static Feature Engineering

- Instead of using handcrafted kernels such as edge detectors can we **learn** **meaningful kernels/filters** in addition to learning the weights of the classifier?



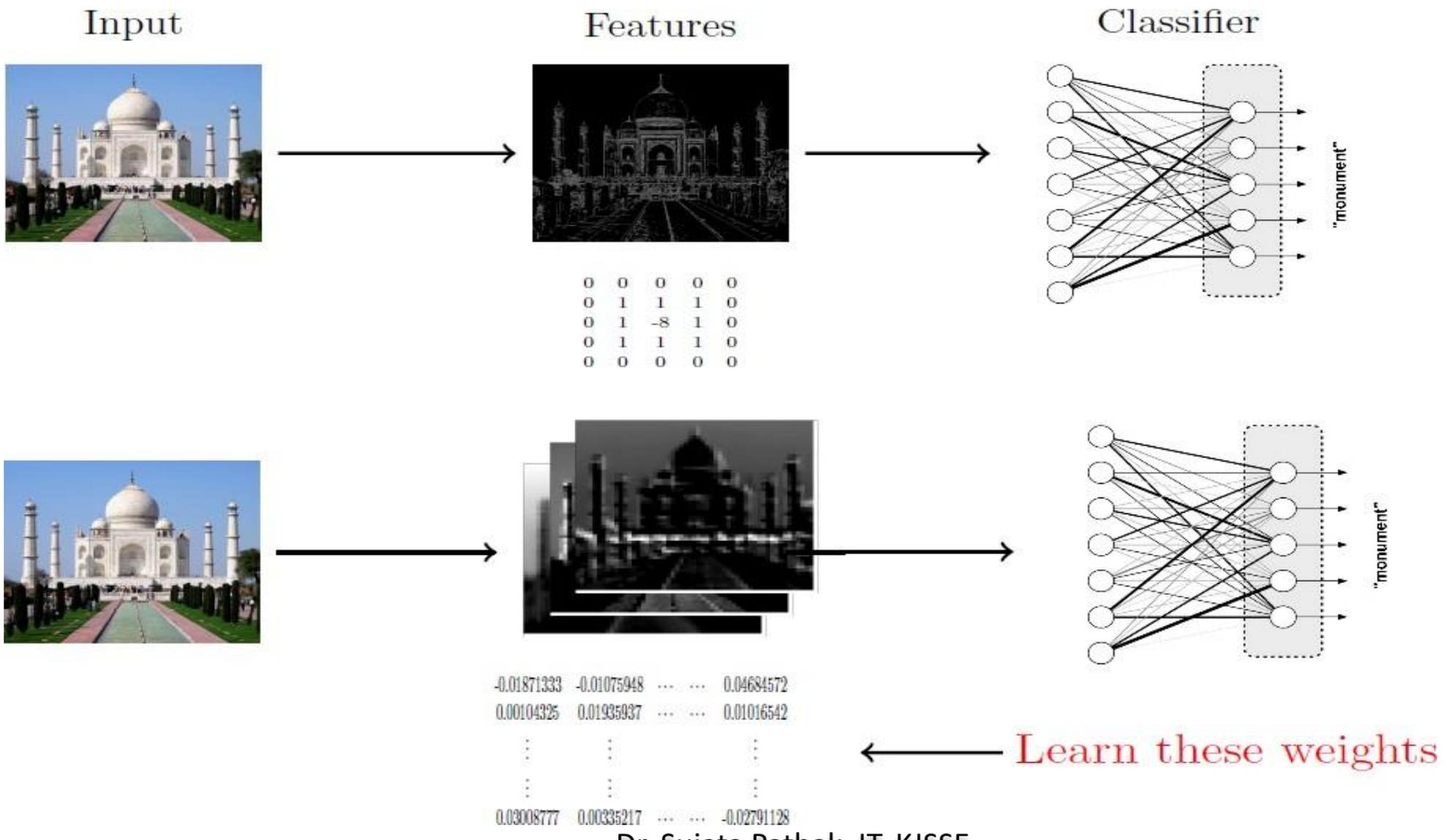
Beyond Static Feature Engineering

- Instead of using handcrafted kernels such as edge detectors can we **learn** **meaningful kernels/filters** in addition to learning the weights of the classifier?



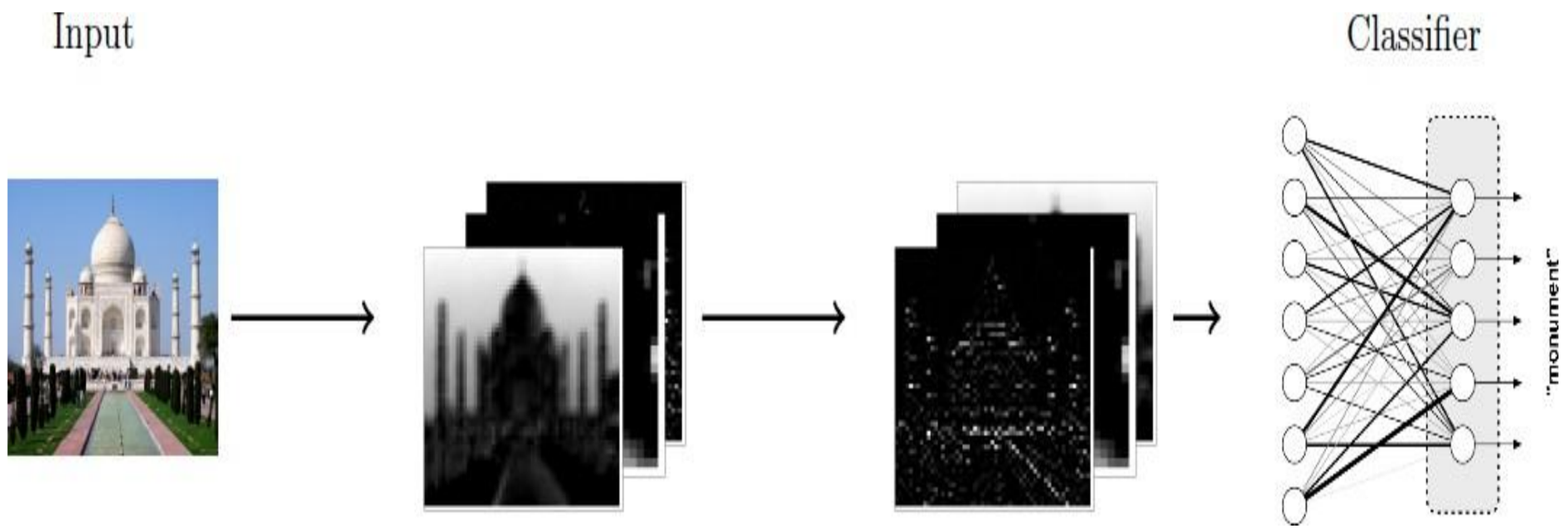
Beyond Static Feature Engineering

- **Even better:** Instead of using handcrafted kernels such as edge detectors can we **learn multiple meaningful kernels/filters** in addition to learning the weights of the classifier?



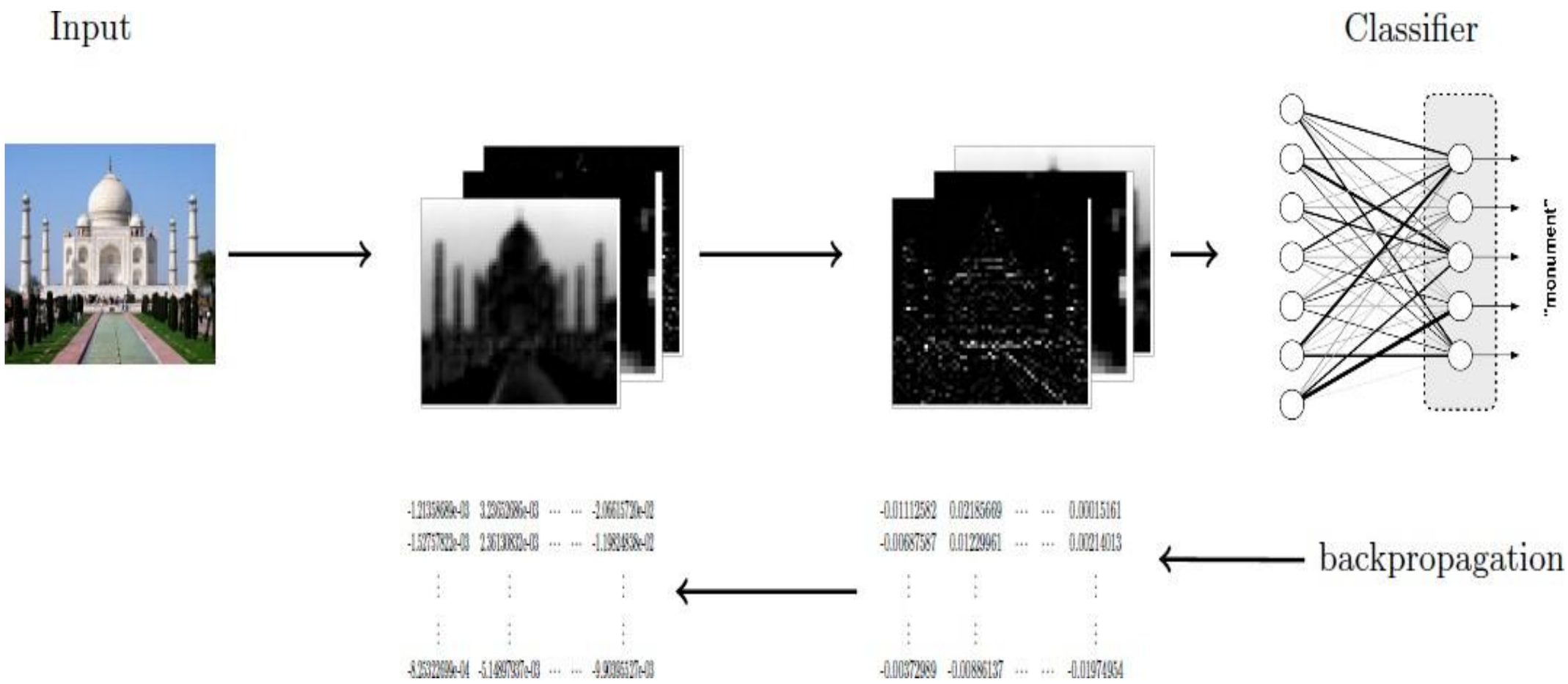
Beyond Static Feature Engineering

- Can we learn multiple layers of meaningful kernels/filters in addition to learning the weights of the classifier?



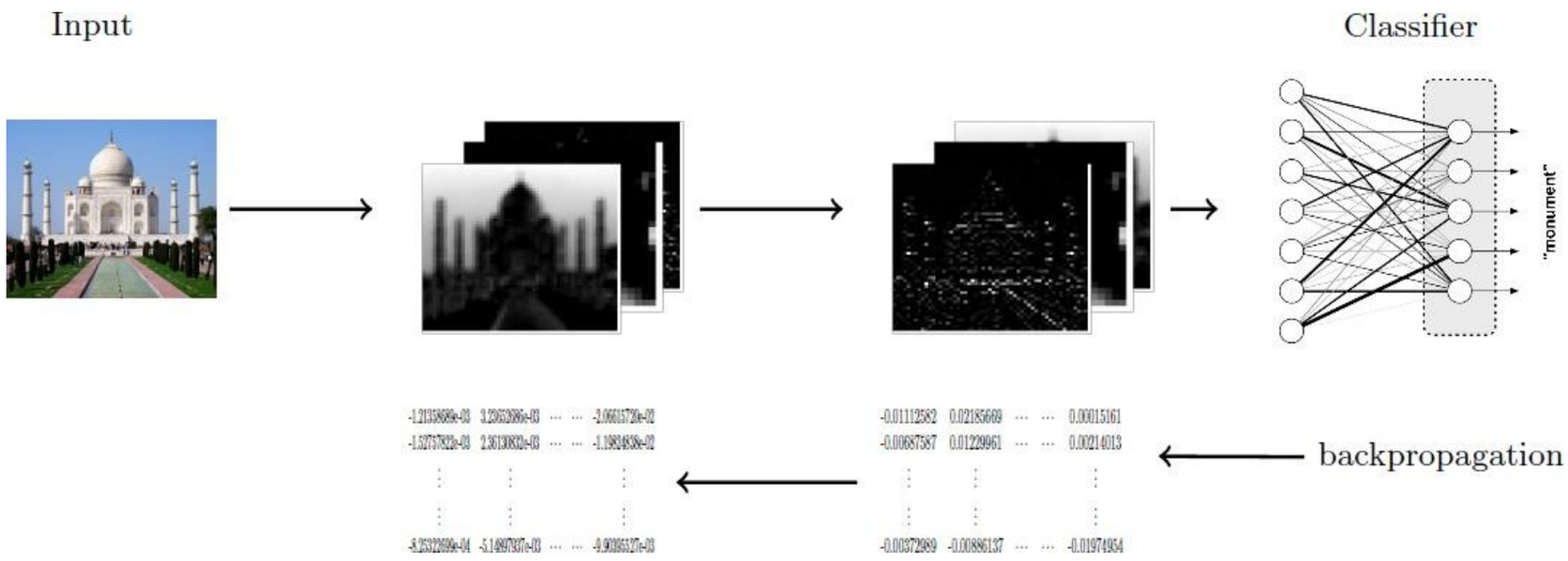
Beyond Static Feature Engineering

- Can we learn multiple layers of meaningful kernels/filters in addition to learning the weights of the classifier? **Yes, we can!**
- Simply by treating these kernels as parameters and learning them in addition to the weights of the classifier (using backpropagation)



Beyond Static Feature Engineering

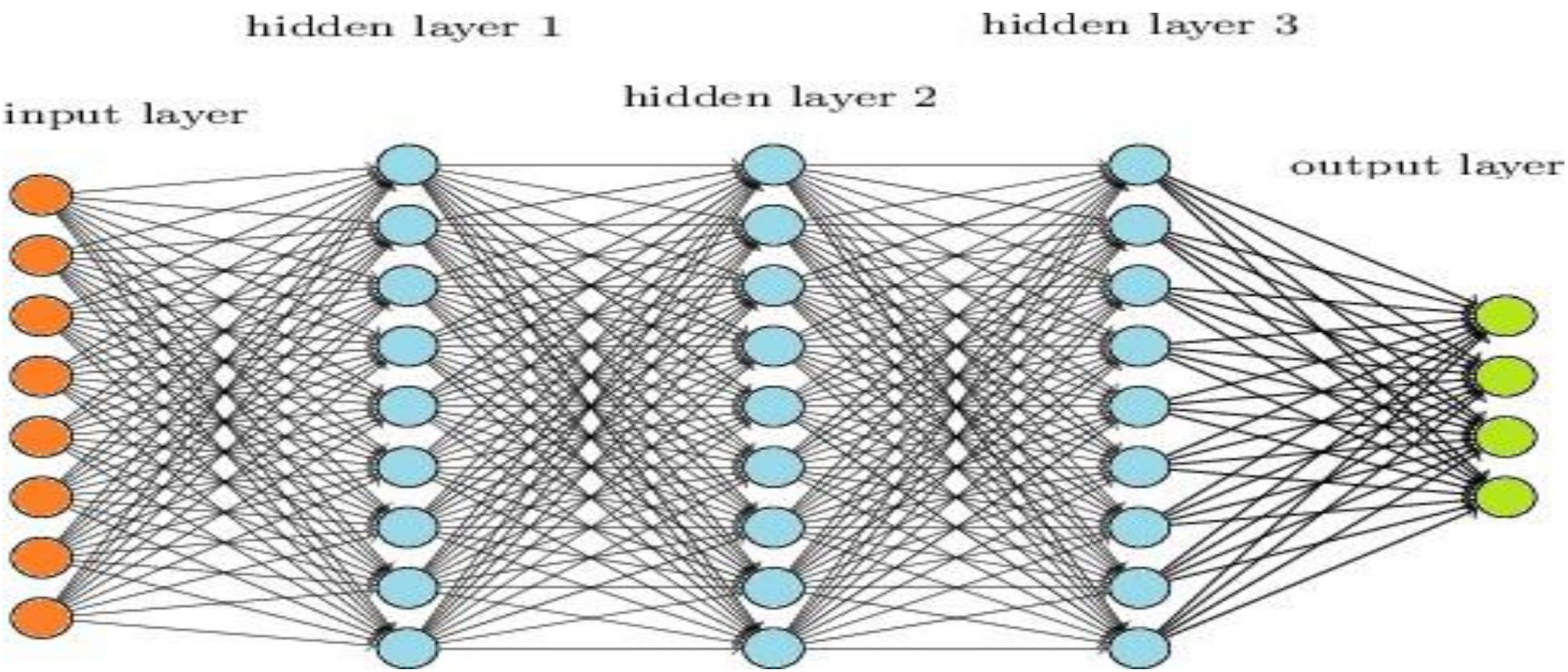
- Can we learn multiple layers of meaningful kernels/filters in addition to learning the weights of the classifier? **Yes, we can!**
- Simply by treating these kernels as parameters and learning them in addition to the weights of the classifier (using backpropagation)



- Such a network is called a **Convolutional Neural Network!**

Pause and Ponder

- Learning kernels/filters by treating them as parameters definitely is interesting
- But why not directly use flattened images with fully connected neural networks (or feedforward neural networks, FNNs) instead?



Challenges of Applying FNNs to Images



MNIST Dataset

On a reasonably *simple* dataset like MNIST, we can get about 2% error (or even better) using FNNs, but

- Ignores spatial (2-D) structure of input images – unroll each 28×28 image into a 784-D vector
  - Pixels that are spatially separate are treated the same way as pixels that are adjacent
- No obvious way for networks to learn same features (e.g. edges) at different places in the input image
- Can get computationally expensive for large images
  - For a 1MP color image with 20 neurons in the first hidden layer, how many weights in the first layer?

Challenges of Applying FNNs to Images



MNIST Dataset

Credit: Steve Renals

On a reasonably *simple* dataset like MNIST, we can get about 2% error (or even better) using FNNs, but

- Ignores spatial (2-D) structure of input images – unroll each 28×28 image into a 784-D vector
  - Pixels that are spatially separate are treated the same way as pixels that are adjacent
- No obvious way for networks to learn same features (e.g. edges) at different places in the input image
- Can get computationally expensive for large
  - images For a 1MP color image with 20 neurons in the first hidden layer, how many weights in the first layer? **60 million!**

How do Convolutional Neural Networks Solve these Challenges?

- **Local receptive fields**, in which hidden units are connected to local patches of the layer below, serve two purposes:
  - Capture local spatial relationships in pixels (which would not be captured by
  - FNNs) Greatly reduces number of parameters in the model
  - For a 1MP color image ,a filter size of $K_1 \times K_2$ in the first hidden layer, how many weights in a convolutional layer?

How do Convolutional Neural Networks Solve these Challenges?

- **Local receptive fields**, in which hidden units are connected to local patches of the layer below, serve two purposes:
  - Capture local spatial relationships in pixels (which would not be captured by FNNs)
  - Greatly reduces number of parameters in the model
  - For a 1MP color image a filter size of $K_1 \times K_2$ in the first hidden layer, how many weights in a convolutional layer? $K_1 \times K_2$, compare with 60 million for FNNs on the previous slide!

How do Convolutional Neural Networks Solve these Challenges?

- **Local receptive fields**, in which hidden units are connected to local patches of the layer below, serve two purposes:
  - Capture local spatial relationships in pixels (which would not be captured by
  - FNNs) Greatly reduces number of parameters in the model
  - For a 1MP color image a filter size of $K_1 \times K_2$ in the first hidden layer, how many weights in a convolutional layer? $K_1 \times K_2$, compare with 60 million for FNNs on the previous slide!
- **Weight sharing**, which also serves two purposes:
  - Enables translation-invariance of neural network to objects in images.

    Reduces number of parameters in the model

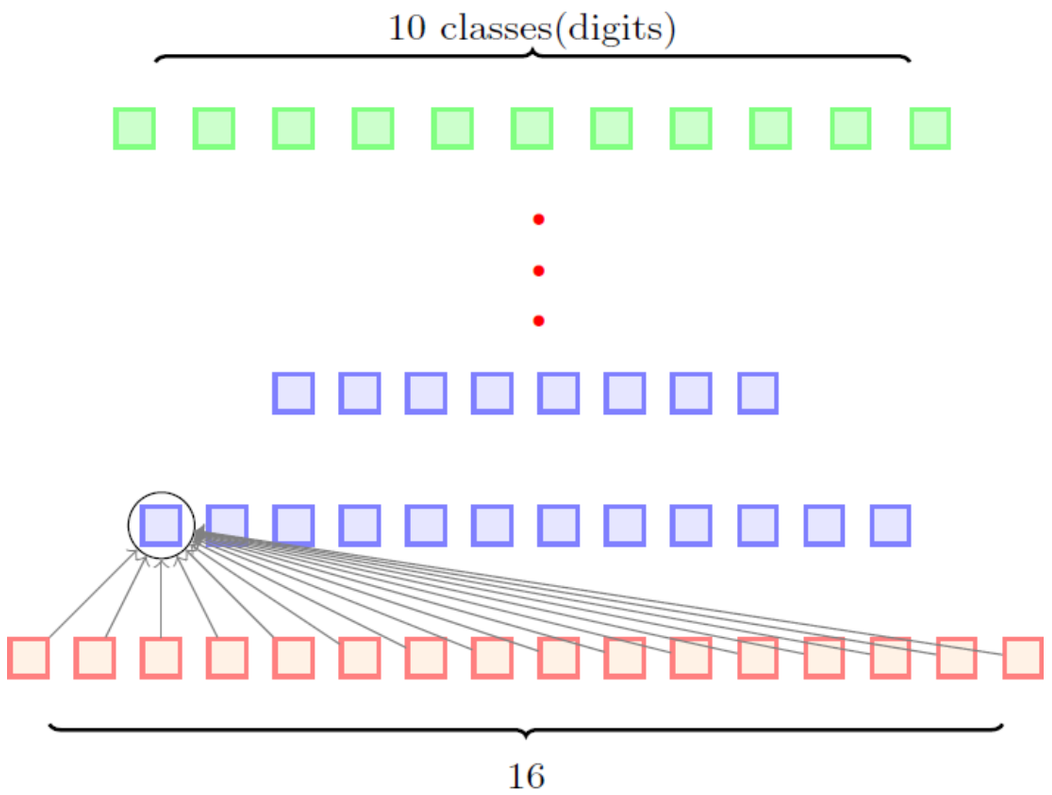
How do Convolutional Neural Networks Solve these Challenges?

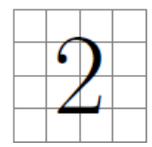
- **Local receptive fields**, in which hidden units are connected to local patches of the layer below, serve two purposes:
  - Capture local spatial relationships in pixels (which would not be captured by FNNs)
  - Greatly reduces number of parameters in the model
  - For a 1MP color image a filter size of $K_1 \times K_2$ in the first hidden layer, how many weights in a convolutional layer? $K_1 \times K_2$, compare with 60 million for FNNs on the previous slide!
- **Weight sharing**, which also serves two purposes:
  - Enables translation-invariance of neural network to objects in images
  - Reduces number of parameters in the model
- **Pooling** which condenses information from previous layer, serves two purposes:
  - Aggregates information, especially minor variations
  - Reduces size of output of a previous layer, which reduces number of computations in later layers

Credit: Steve Renals

Local Receptive Fields

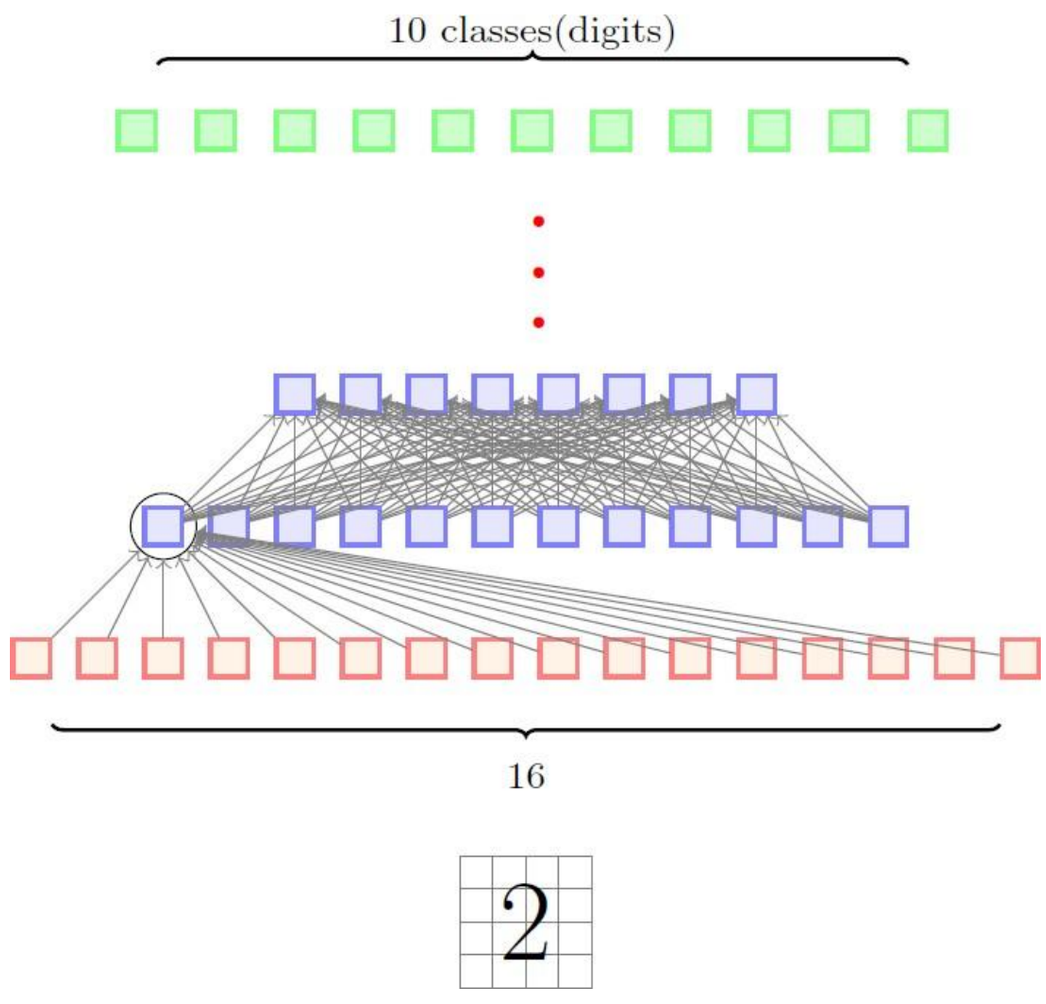
- This is what a regular feedforward neural network will look like
- There are many dense connections here





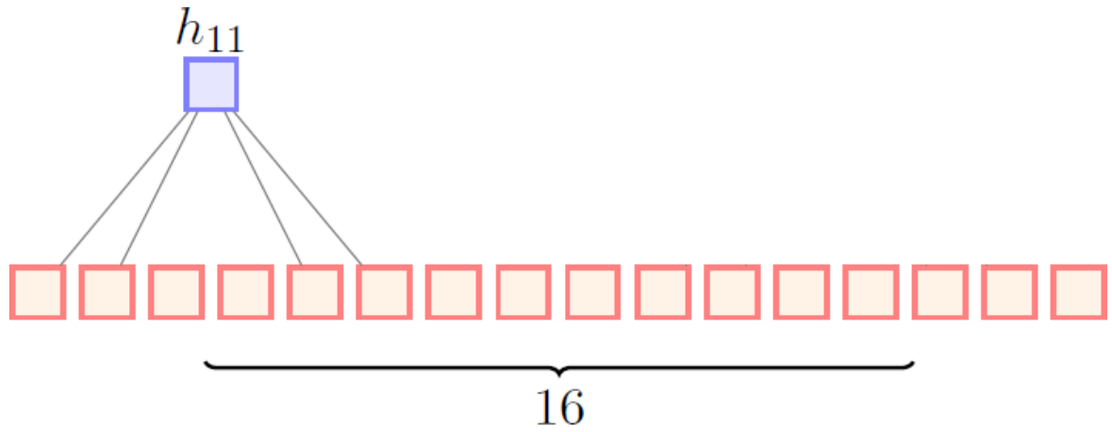
Local Receptive Fields

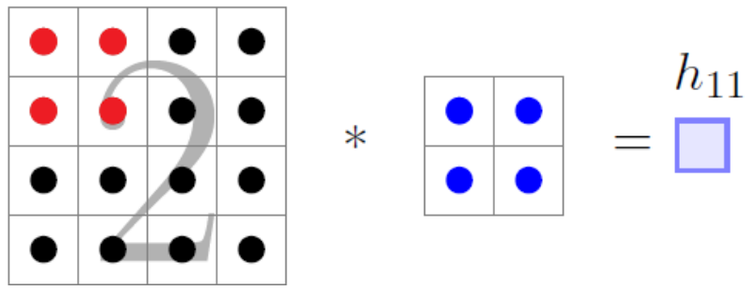
- This is what a regular feedforward neural network will look like
- There are many dense connections here
- All 16 input neurons are contributing to computation of h_{11}
- Let us contrast this to what happens in case of convolution



Local Receptive Fields

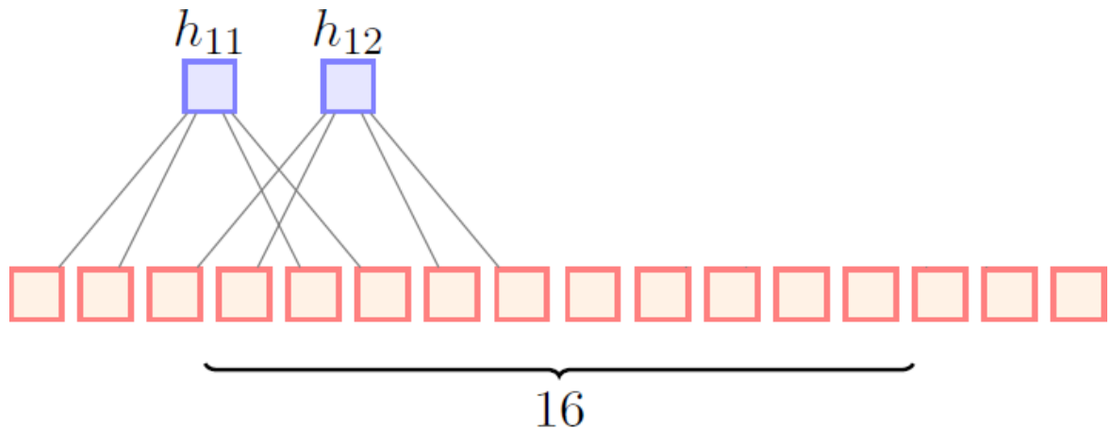
- Only a few local neurons participate in computation of h_{11}
- E.g. only pixels 1, 2, 5, 6 contribute to h_{11}

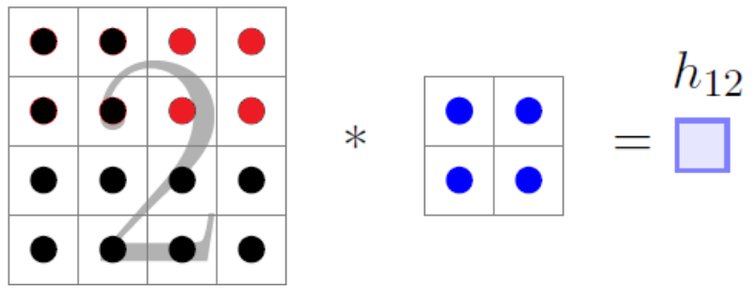




Local Receptive Fields

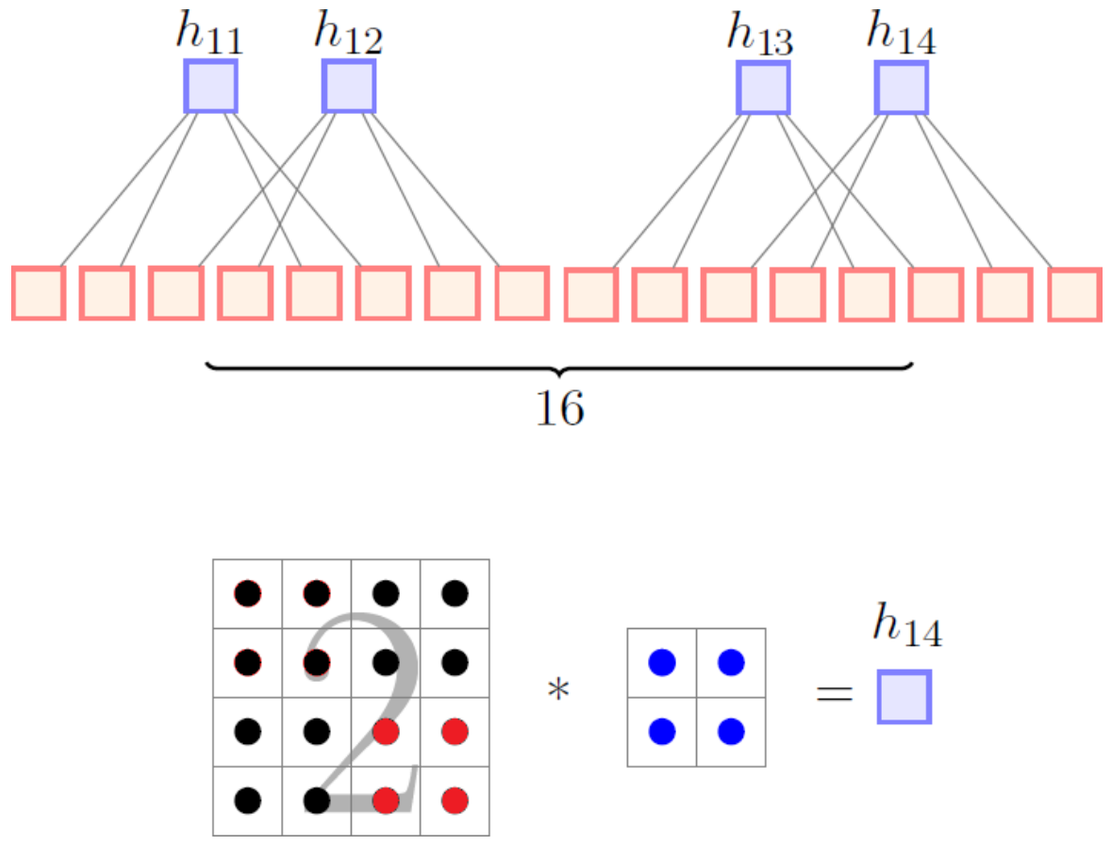
- Only a few local neurons participate in computation of h_{11}
- E.g. only pixels 1, 2, 5, 6 contribute to h_{11}





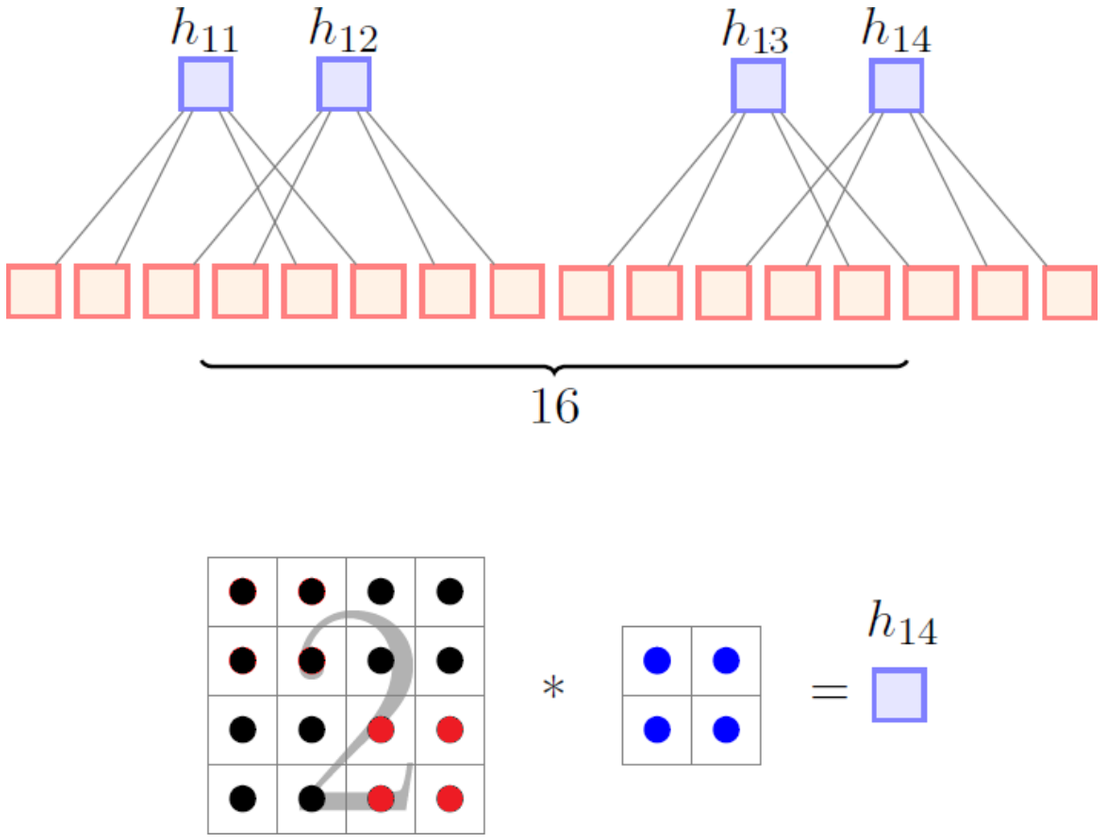
Local Receptive Fields

- Only a few local neurons participate in computation of h_{11}
- E.g. only pixels 1, 2, 5, 6 contribute to h_{11}
- Similar for other pixels



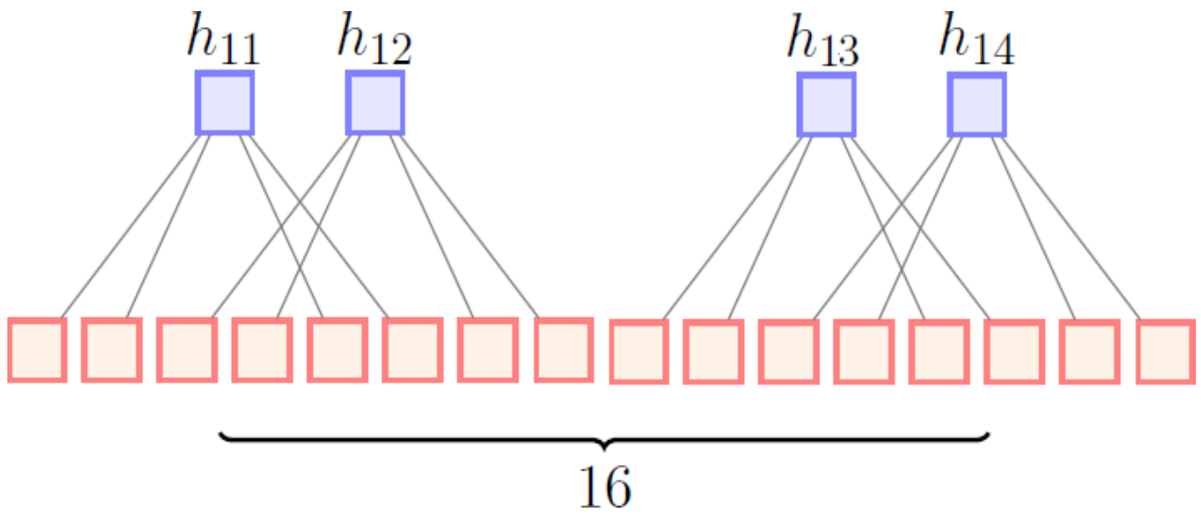
Local Receptive Fields

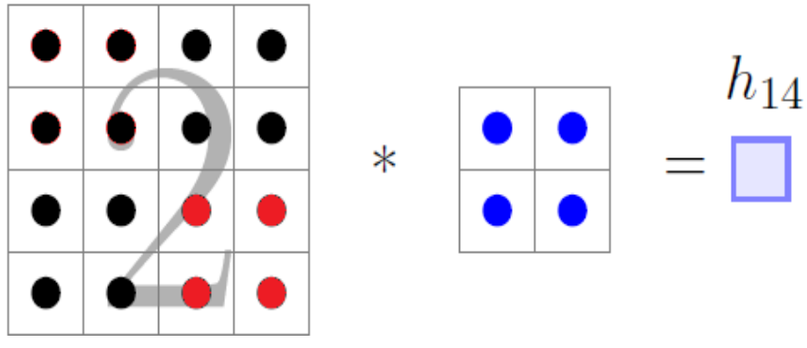
- Only a few local neurons participate in computation of h_{11}
- E.g. only pixels 1, 2, 5, 6 contribute to h_{11}
- Similar for other pixels
- The connections are much sparser
- This **sparse connectivity** reduces the number of parameters in the model



Local Receptive Fields

- Only a few local neurons participate in computation of h_{11}
- E.g. only pixels 1, 2, 5, 6 contribute to h_{11}
- Similar for other pixels
- The connections are much sparser
- This **sparse connectivity** reduces the number of parameters in the model
- We are taking advantage of the structure of the image (interactions between neighboring pixels are interesting in images)





Local Receptive Fields

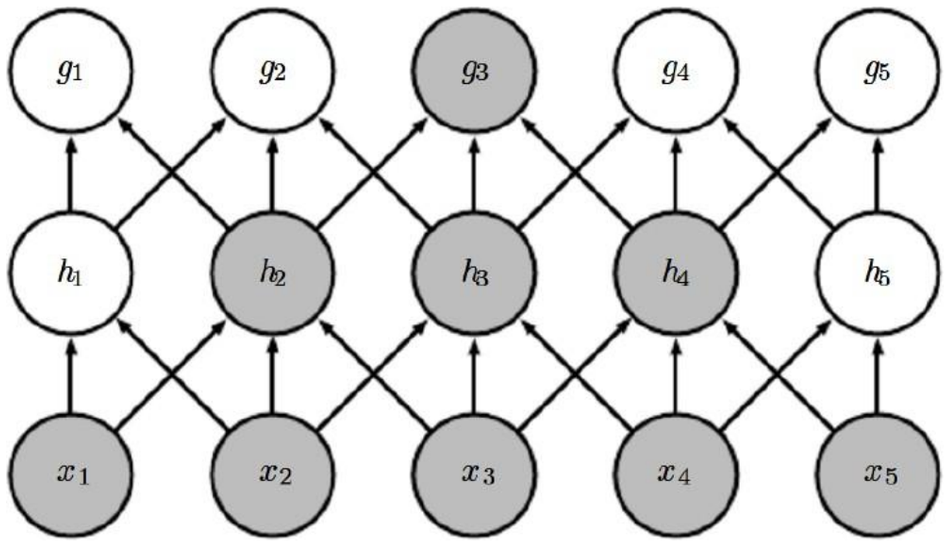
- But is sparse connectivity really a good thing?

Local Receptive Fields

- But is sparse connectivity really a good thing?
- Aren't we losing information (by losing interactions between some input pixels)

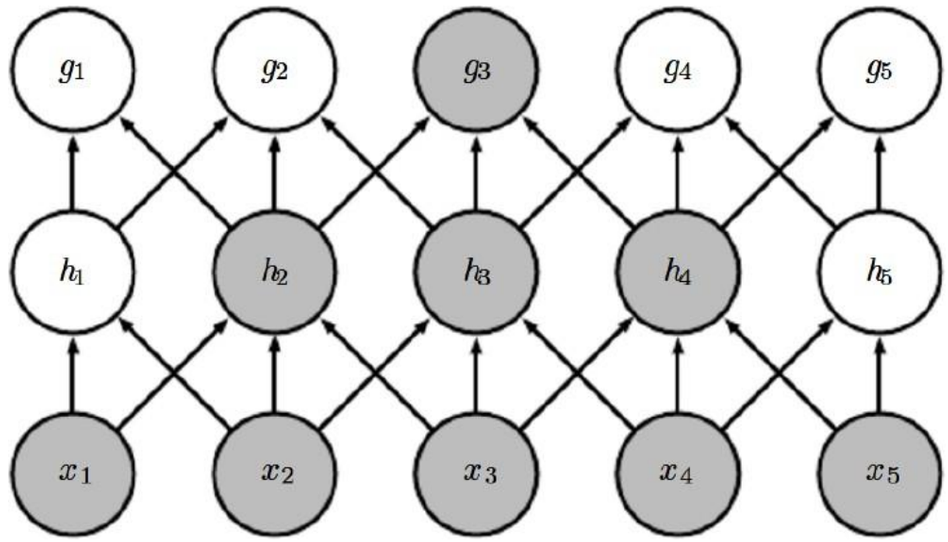
Local Receptive Fields

- But is sparse connectivity really a good thing?
- Aren't we losing information (by losing interactions between some input pixels)
- Well, not really



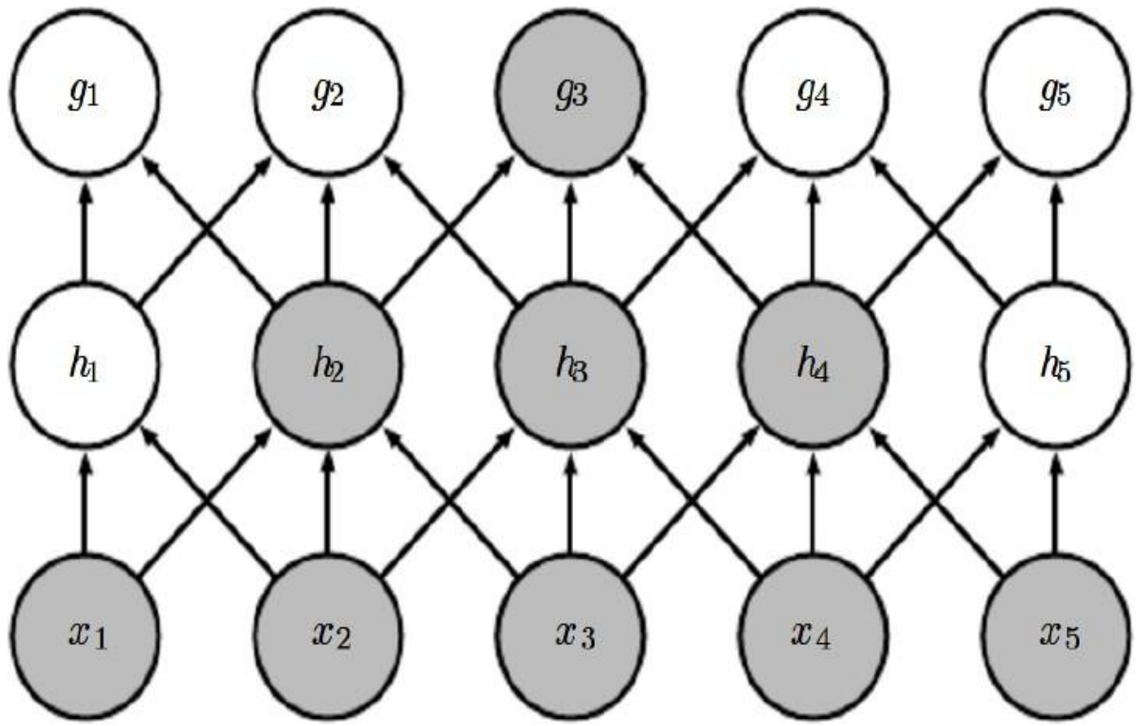
Local Receptive Fields

- But is sparse connectivity really a good thing?
- Aren't we losing information (by losing interactions between some input pixels)
- Well, not really
- The two highlighted neurons ($x_1 x_5$) do not interact in layer 1



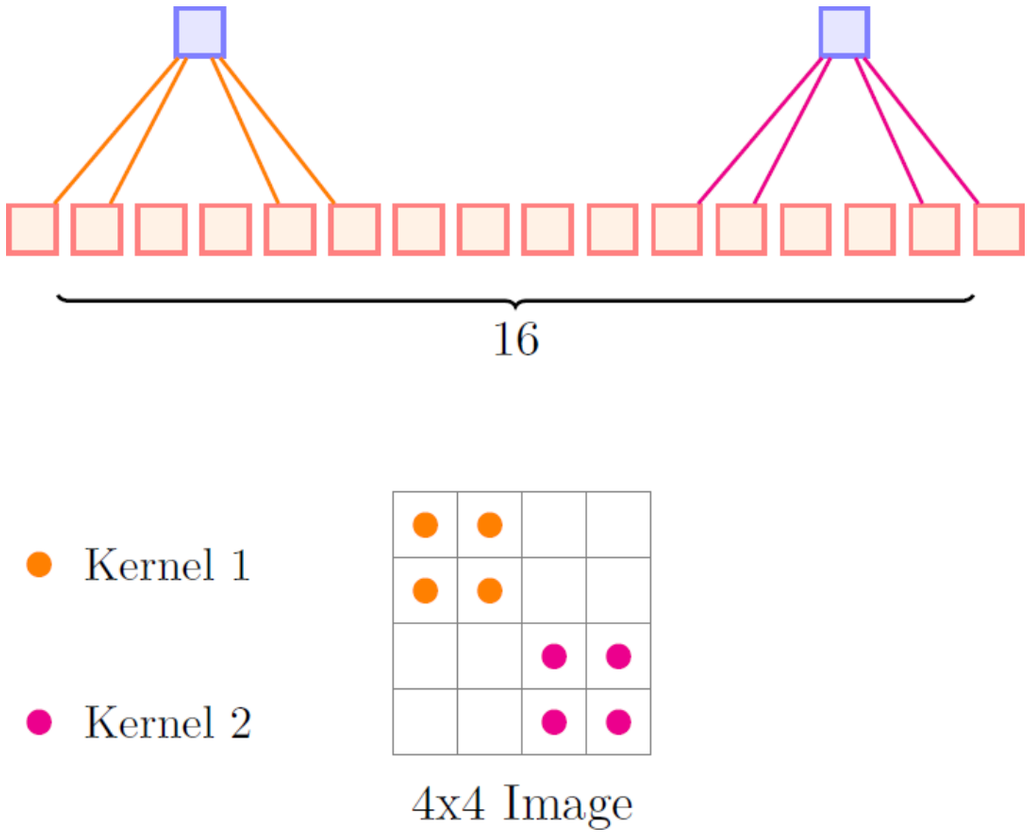
Local Receptive Fields

- But is sparse connectivity really a good thing?
- Aren't we losing information (by losing interactions between some input pixels)
- Well, not really
- The two highlighted neurons ($x_1 x_5$) do not interact in layer 1
- But they indirectly contribute to the computation of g_3 and hence interact indirectly



Weight Sharing

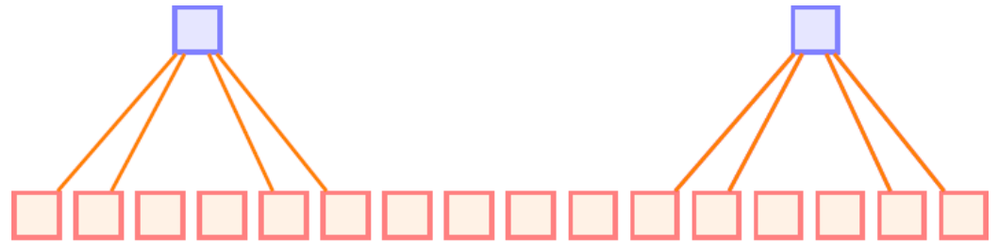
- Consider the following network; do we want the kernel weights to be different for different parts of the image?

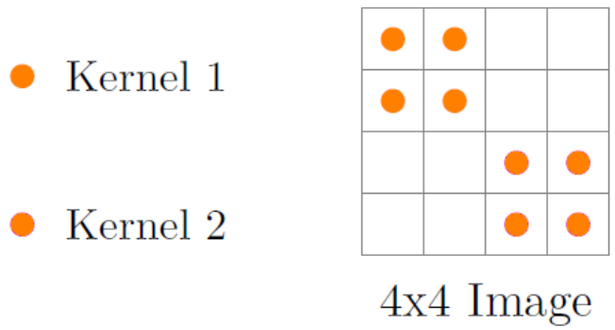


Weight Sharing

- Consider the following network; do we want the kernel weights to be different for different parts of the image?
- Not really. We would want the filter to respond to an object or an artefact in an image in the same way irrespective of where it is located in the image

$\Longrightarrow$ **translation-invariance**

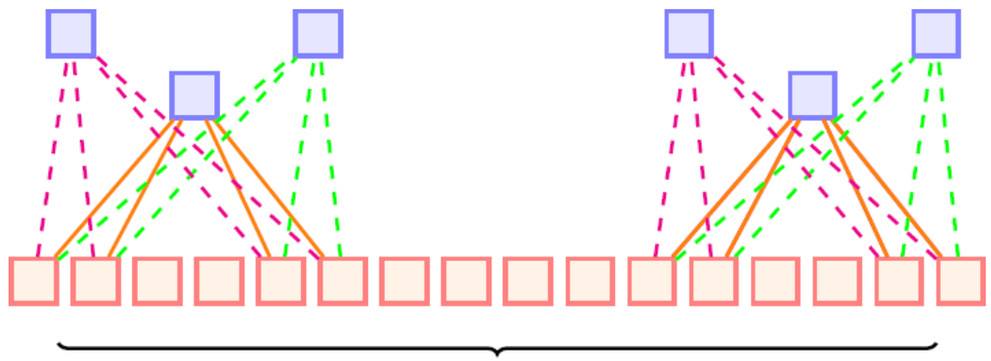




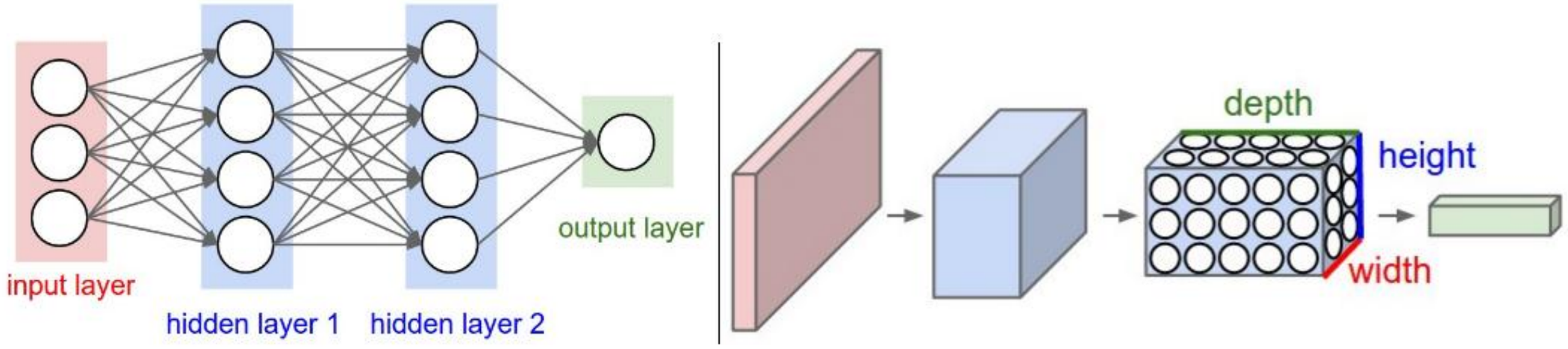
Weight Sharing

- Consider the following network; do we want the kernel weights to be different for different parts of the image?
- Not really. We would want the filter to respond to an object or an artefact in an image in the same way irrespective of where it is located in the image

  $\Longrightarrow$ **translation-invariance**
- We can have as many different kernels to capture different kinds of artifacts, but each one is intended to give the same response on all parts of the image
- This is called **weight sharing**



Dense neural network and Convolutional neural network



Left: A regular 3-layer Neural Network. Right: A ConvNet arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers. Every layer of a ConvNet transforms the 3D input volume to a 3D output volume of neuron activations. In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels).

Introduction

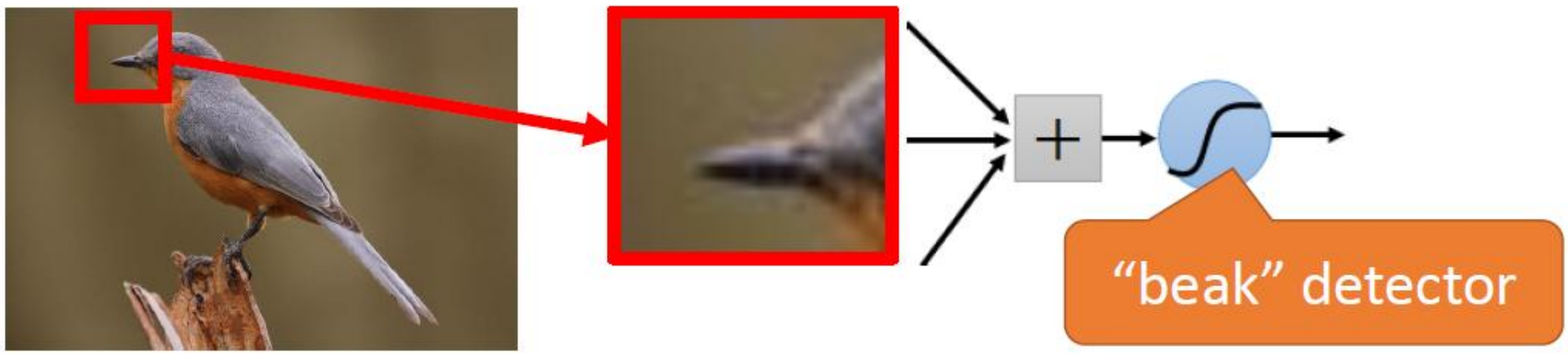


Why CNN for Image

- Some patterns are much smaller than the whole image

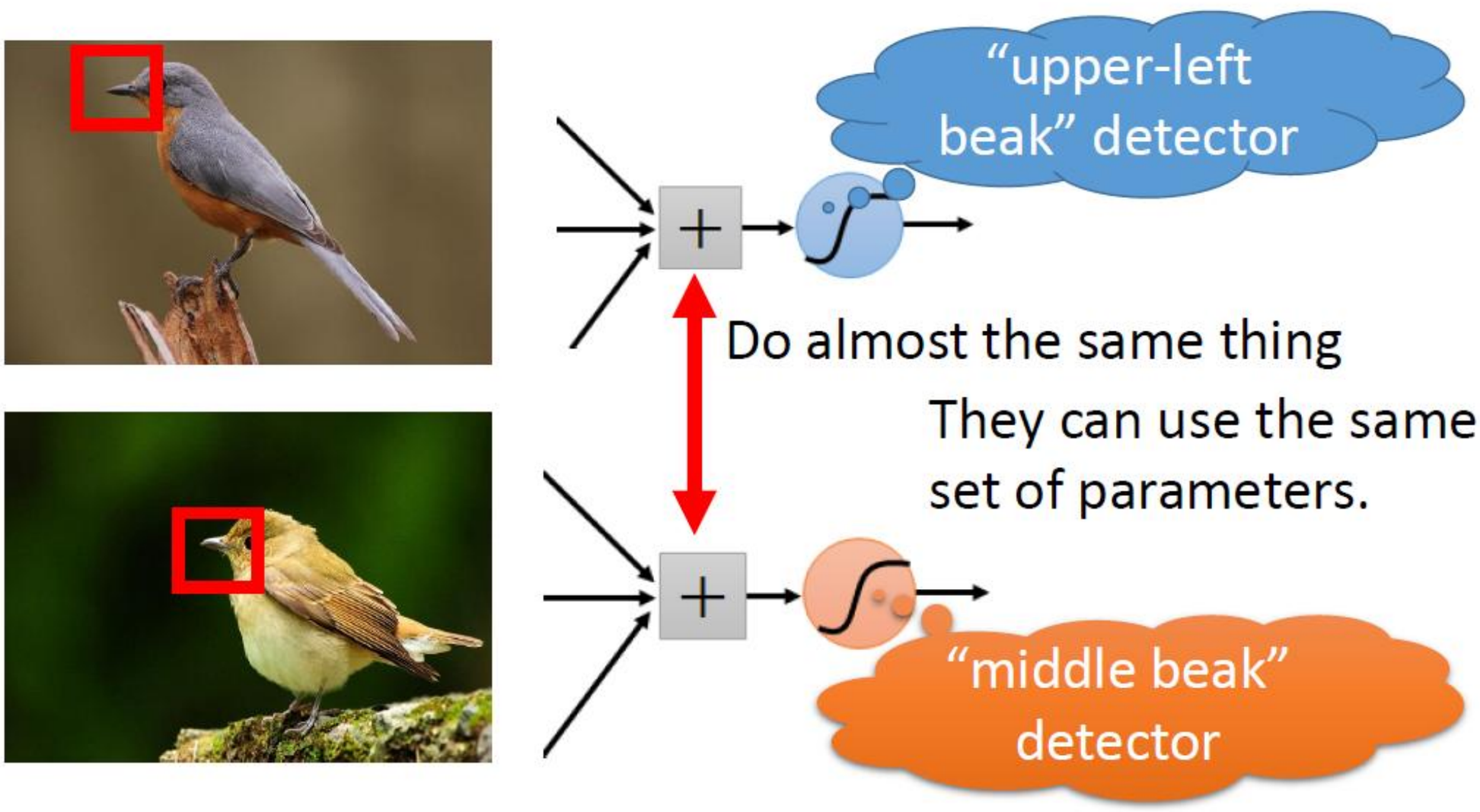
A neuron does not have to see the whole image to discover the pattern.

Connecting to small region with less parameters



Why CNN for Image

- The same patterns appear in different regions.



Why CNN for Image

- Subsampling the pixels will not change the object

bird





subsampling

bird



We can subsample the pixels to make image smaller



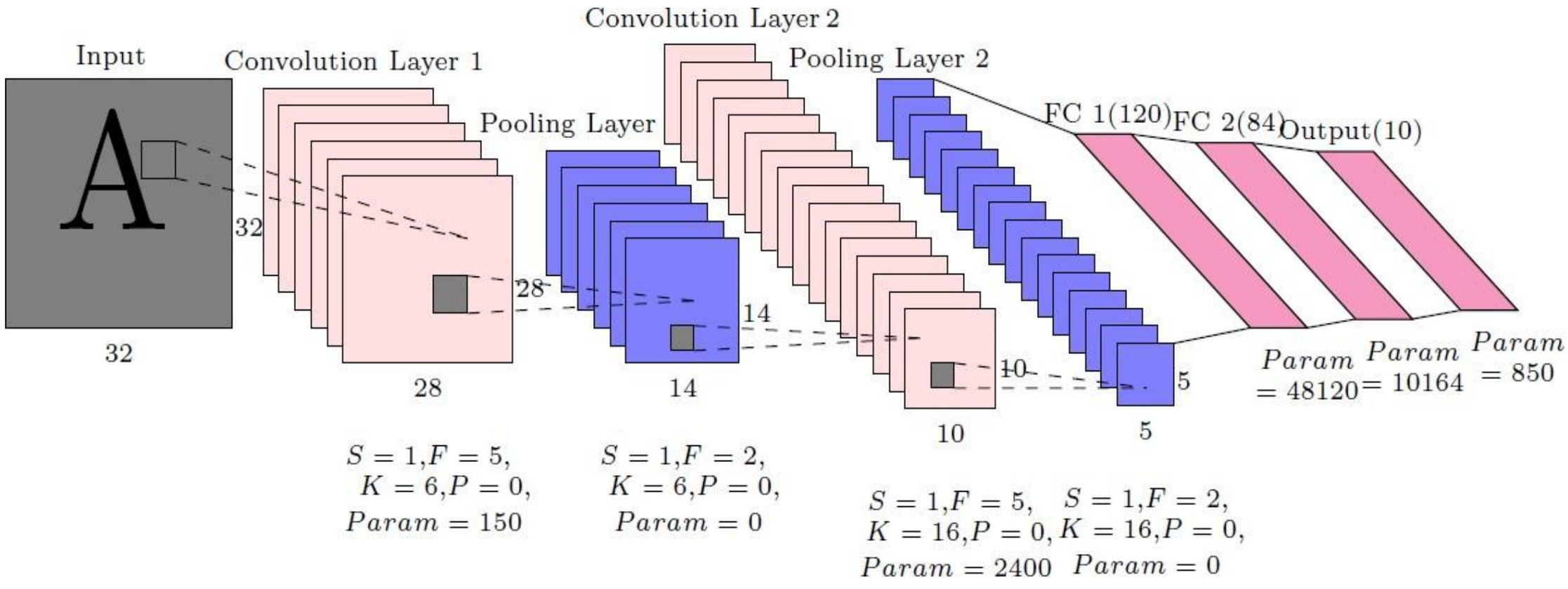
Less parameters for the network to process the image

Convolutional Neural Network

- A typical CNN looks as follows:

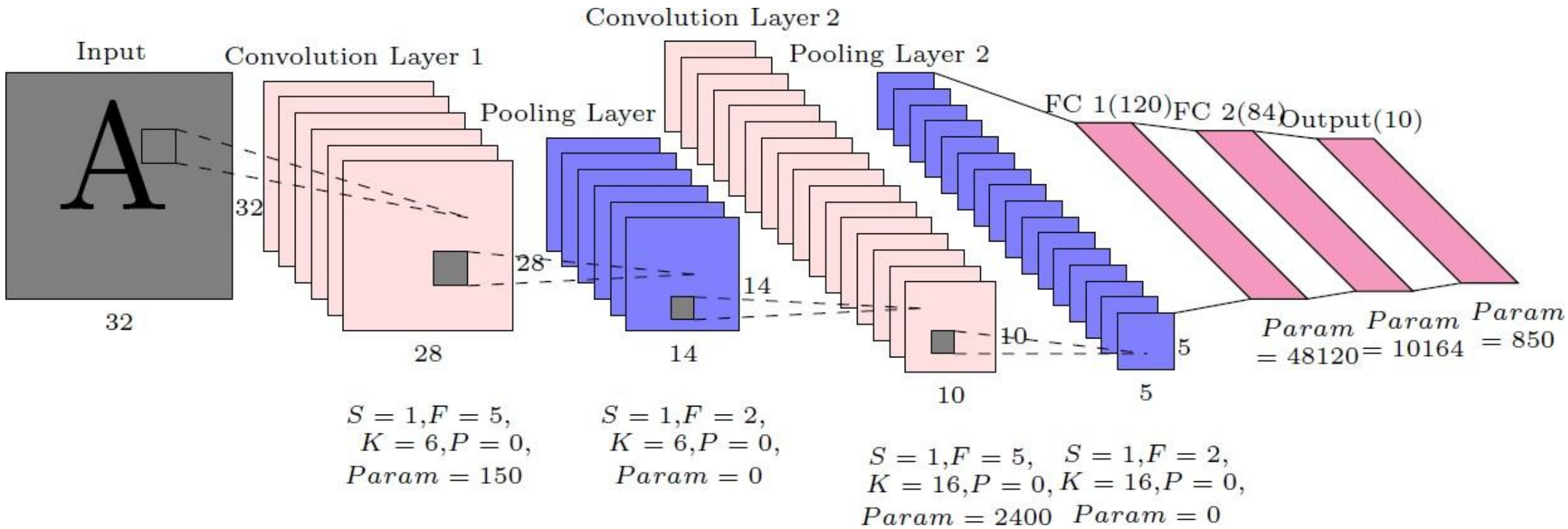
Convolutional Neural Network

- A typical CNN looks as follows:



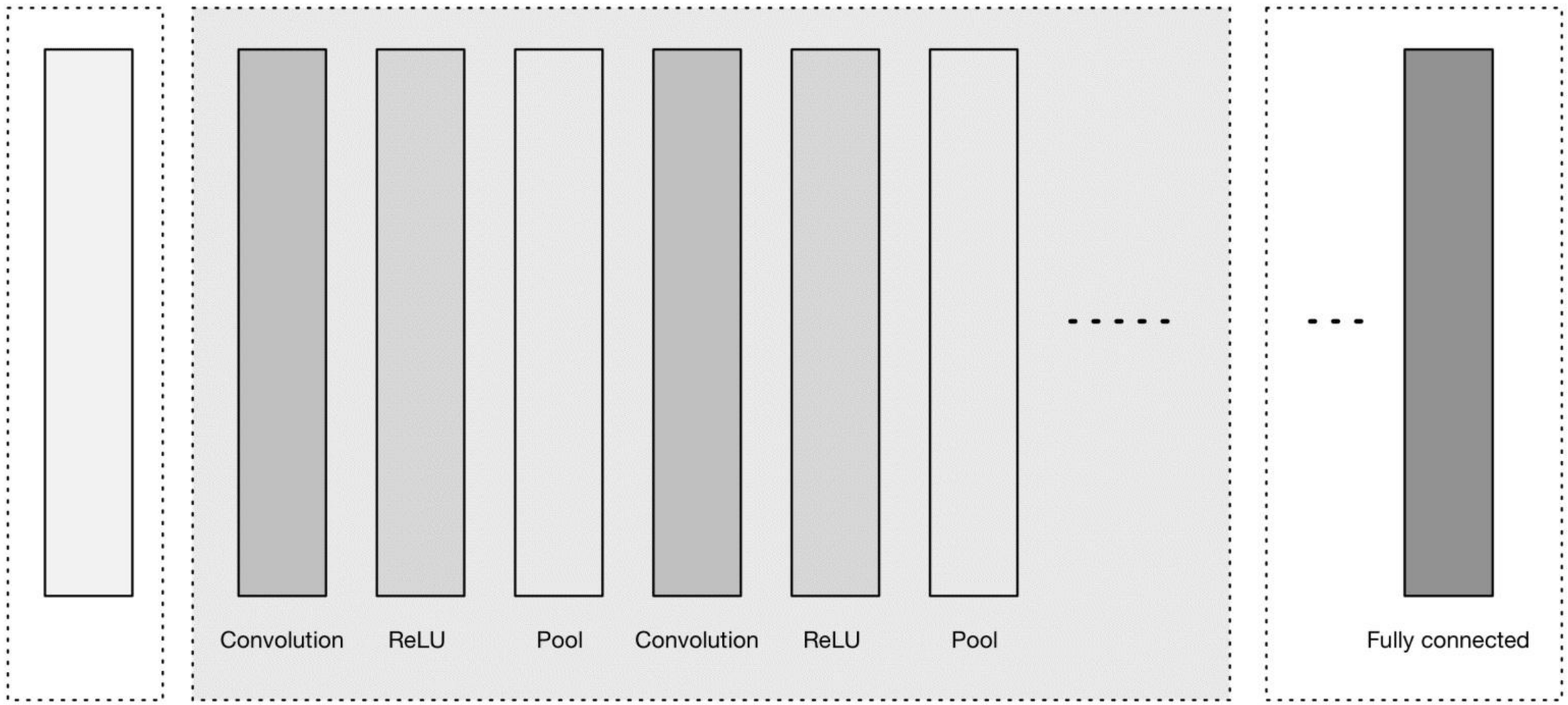
Convolutional Neural Network

- A typical CNN looks as follows:



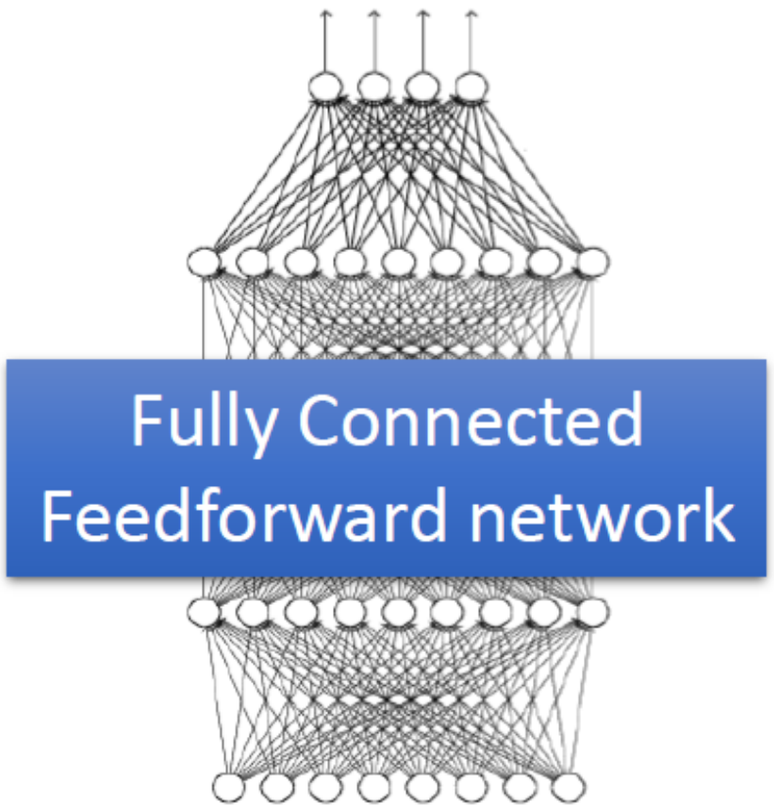
It has alternate convolution and pooling layers What do pooling layers do?

CNN Architecture Overview



The whole CNN





Convolution

Max Pooling

Convolution

Max Pooling

Can repeat many times

Flatten

The whole CNN

Property 1

- Some patterns are much smaller than the whole image

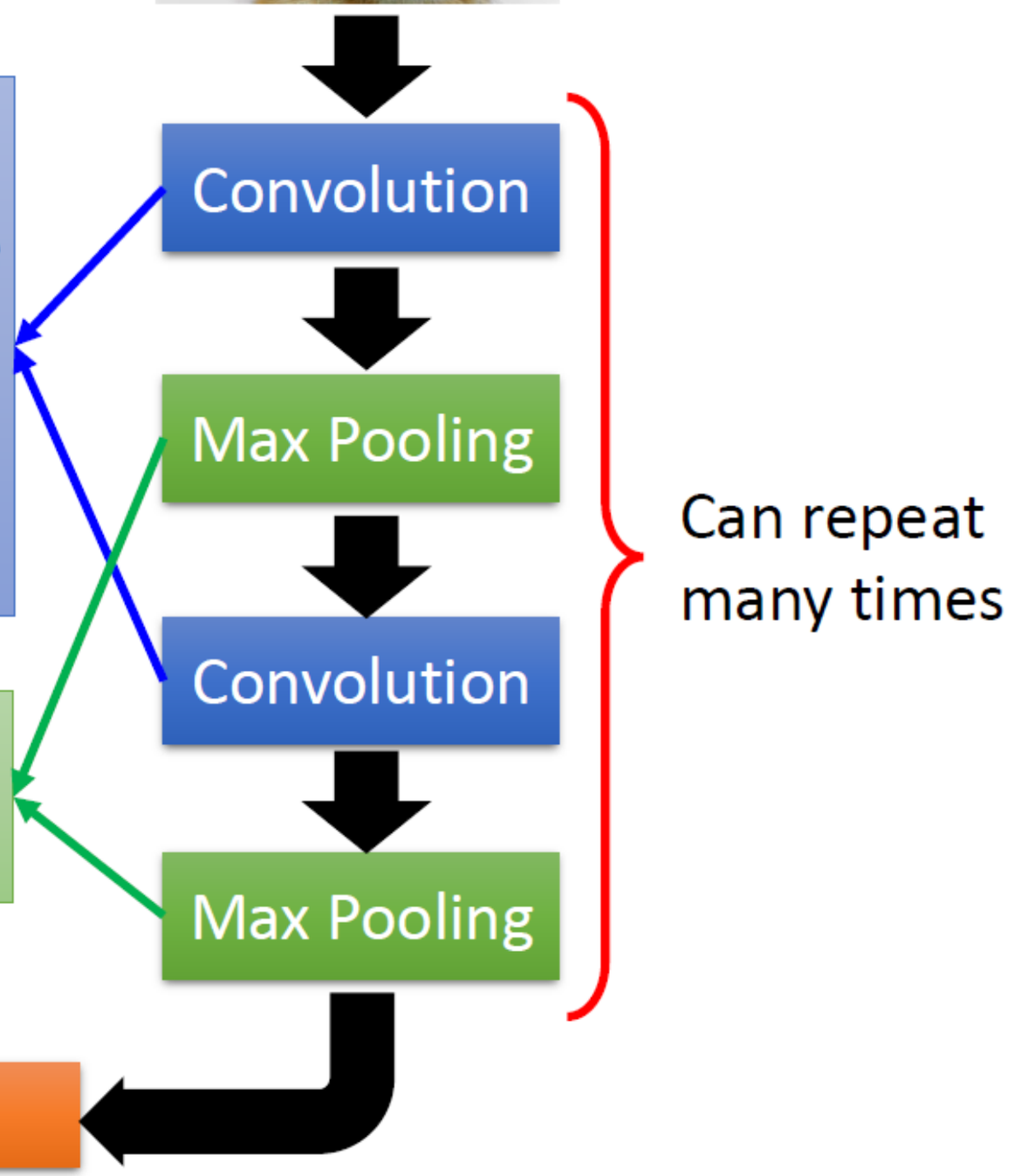
Property 2

- The same patterns appear in different regions.

Property 3

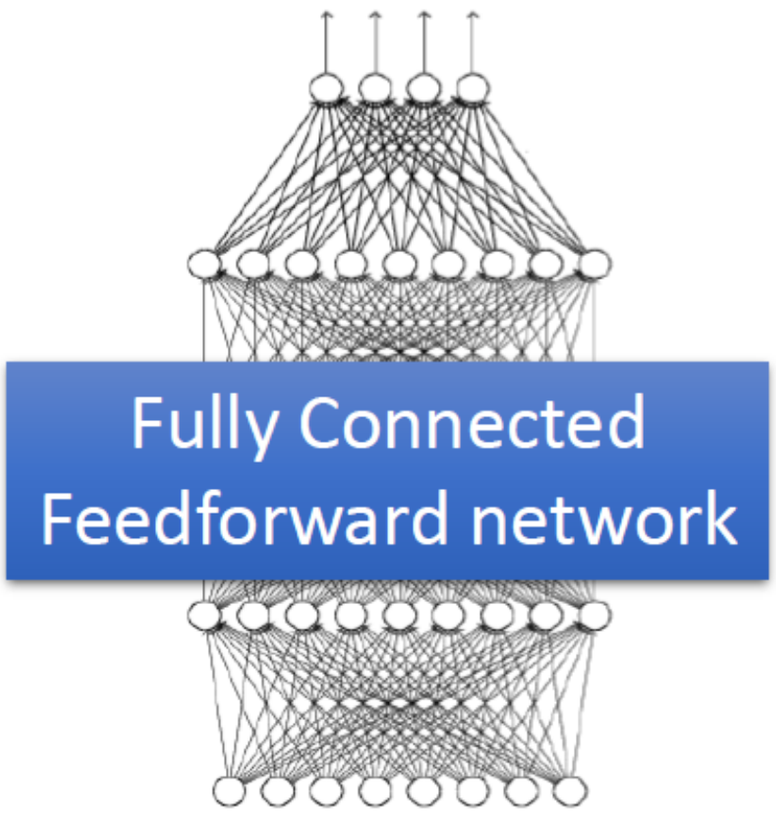
- Subsampling the pixels will not change the object

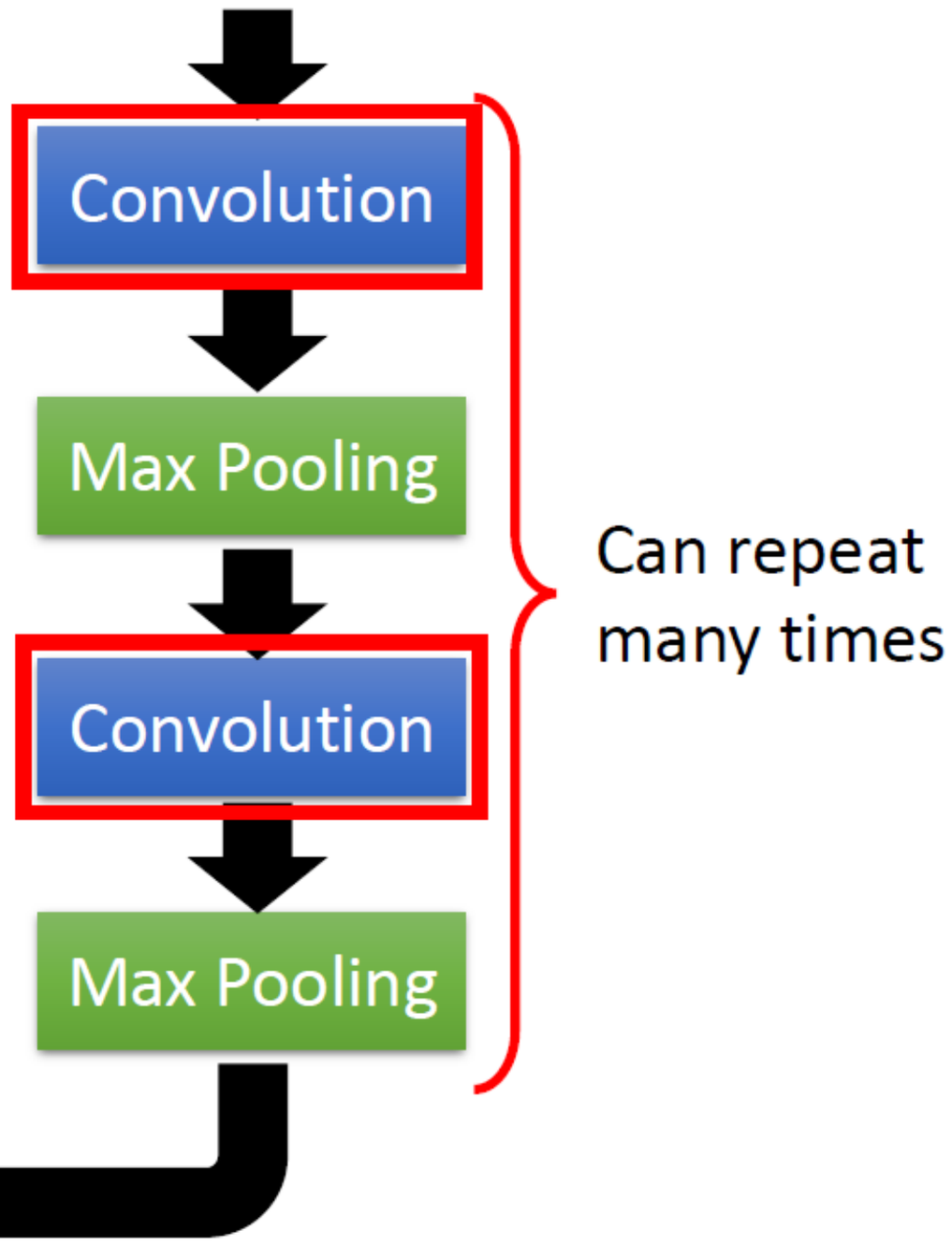




The whole CNN







CNN Architecture Overview

- Convolution-
  - Major components of convolutional layers:
    - Filters
    - Activation maps
    - Parameter sharing
    - Layer-specific hyperparameters

CNN Architecture Overview

Three major groups:

1. Input layer

2. Feature-extraction (learning) layers

3. Classification layers

CNN Architecture Overview

1. Input layer-

- Accepts three-dimensional input generally in the form spatially of the size (width × height) of the image and has a depth representing the color channels (generally three for RGB color channels).

2. Feature-extraction layers-

- have a general repeating pattern of the sequence:
  - 1. Convolution layer- Rectified Linear Unit (ReLU) activation function as a layer in the diagram
  - 2. Pooling layer

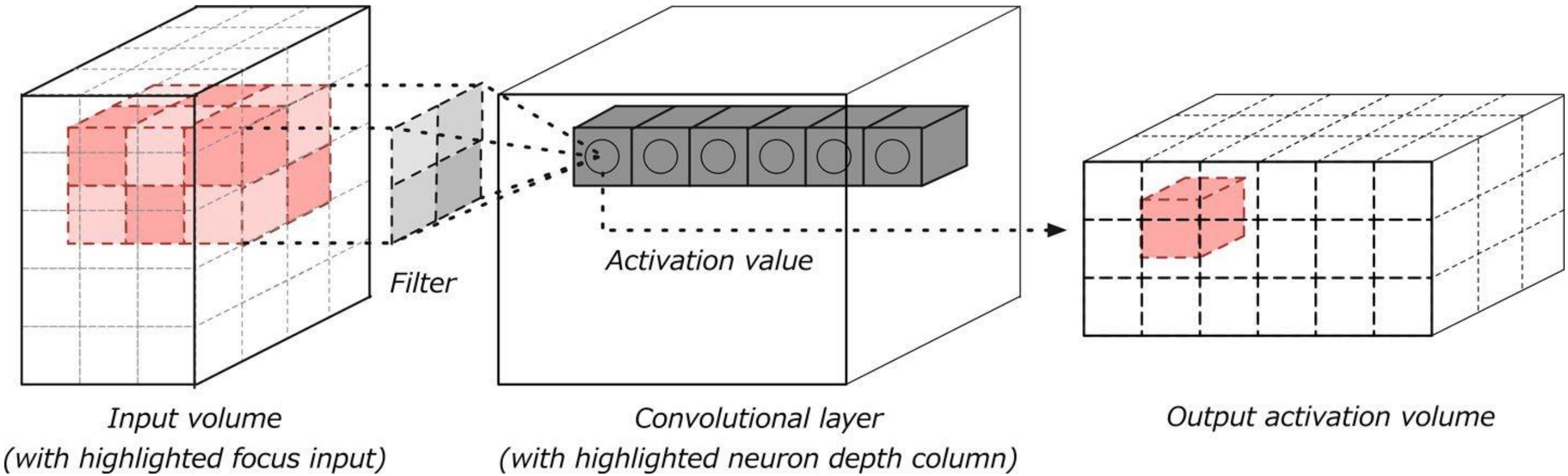
CNN Architecture Overview

3. Classification layers –
   - one or more fully connected layers to take the higher-order features and produce class probabilities or scores.
   - Fully connected to all of the neurons in the previous layer.
   - The output of these layers produces typically a two-dimensional output of the dimensions [b × N], where b is the number of examples in the mini-batch and N is the number of classes we're interested in scoring.

CNN Architecture Overview

- **Input Layers-**
- **Convolutional Layers-**
  - Core building blocks of CNN architectures.
  - Transforms the input data by using a patch of locally connecting neurons from the previous layer.
  - The layer will compute a dot product between the region of the neurons in the input layer and the weights to which they are locally connected in the output layer.
  - The resulting output generally has the same spatial dimensions (or smaller spatial dimensions) but sometimes increases the number of elements in the third dimension of the output (depth dimension).

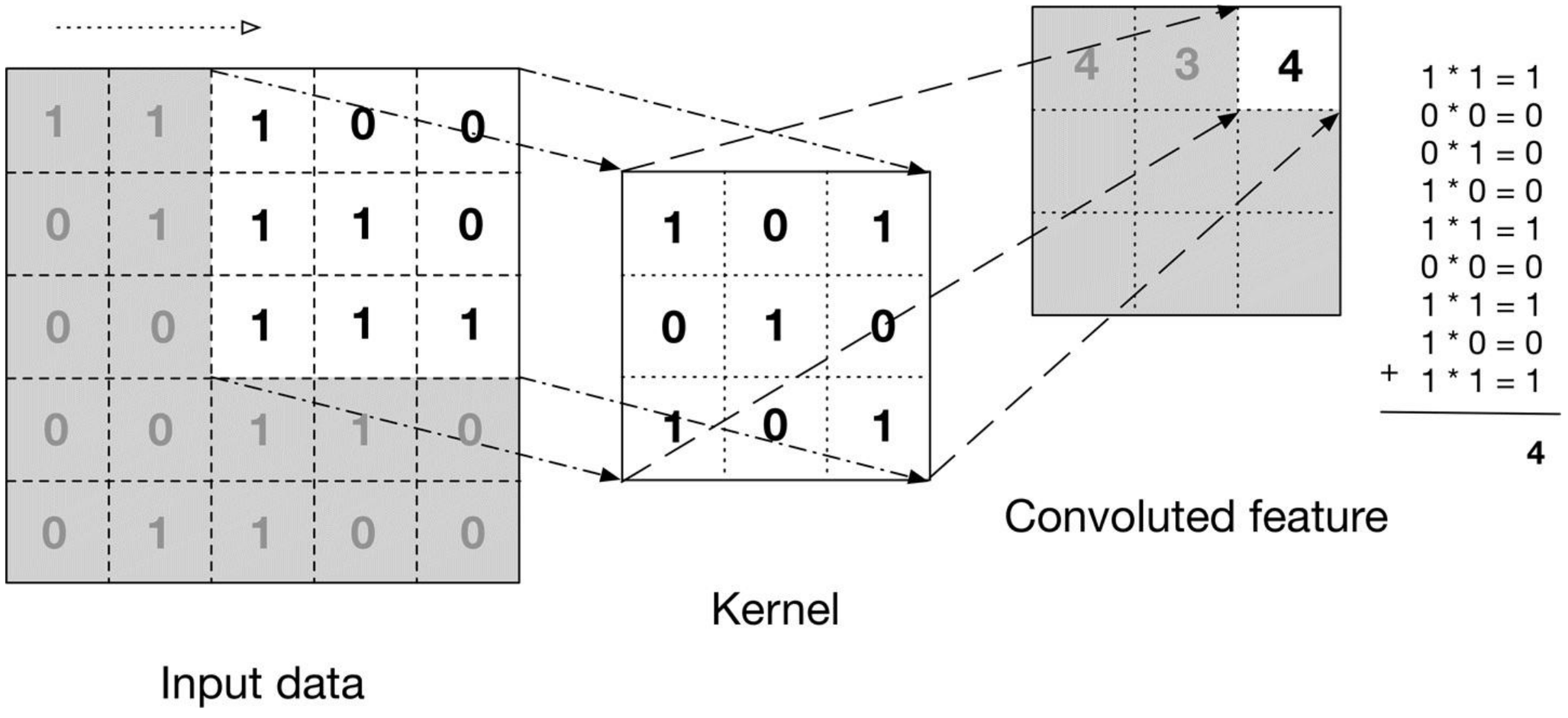
CNN Architecture Overview



Convolution layer with input and output volumes

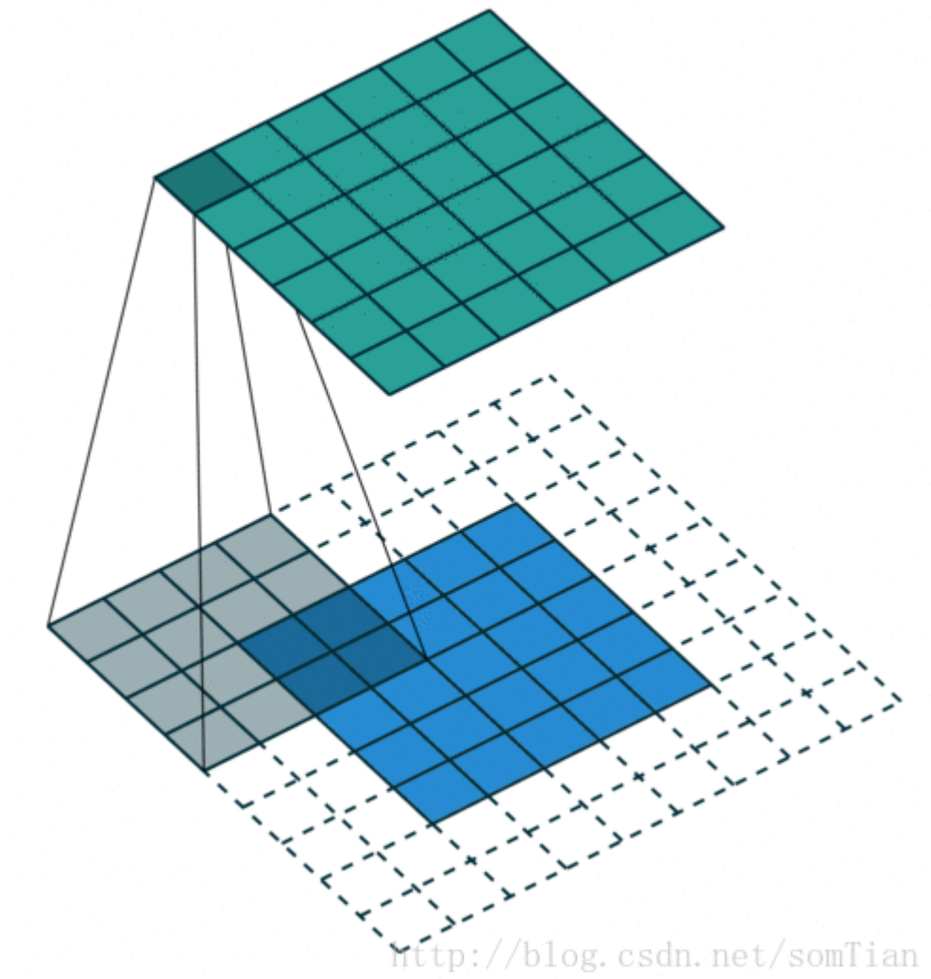
CNN Architecture Overview

- Convolution-



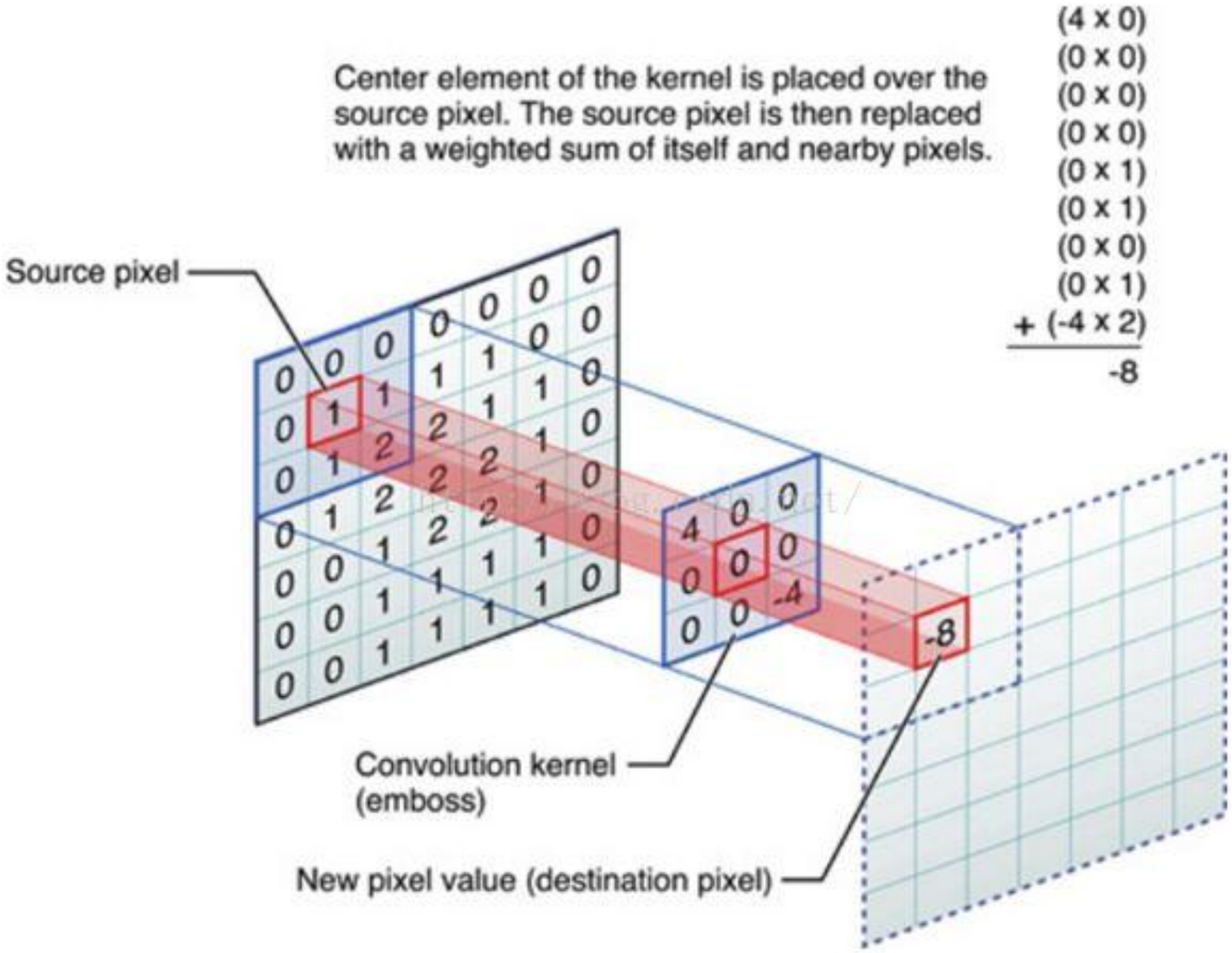
The convolution operation

Convolutional kernel



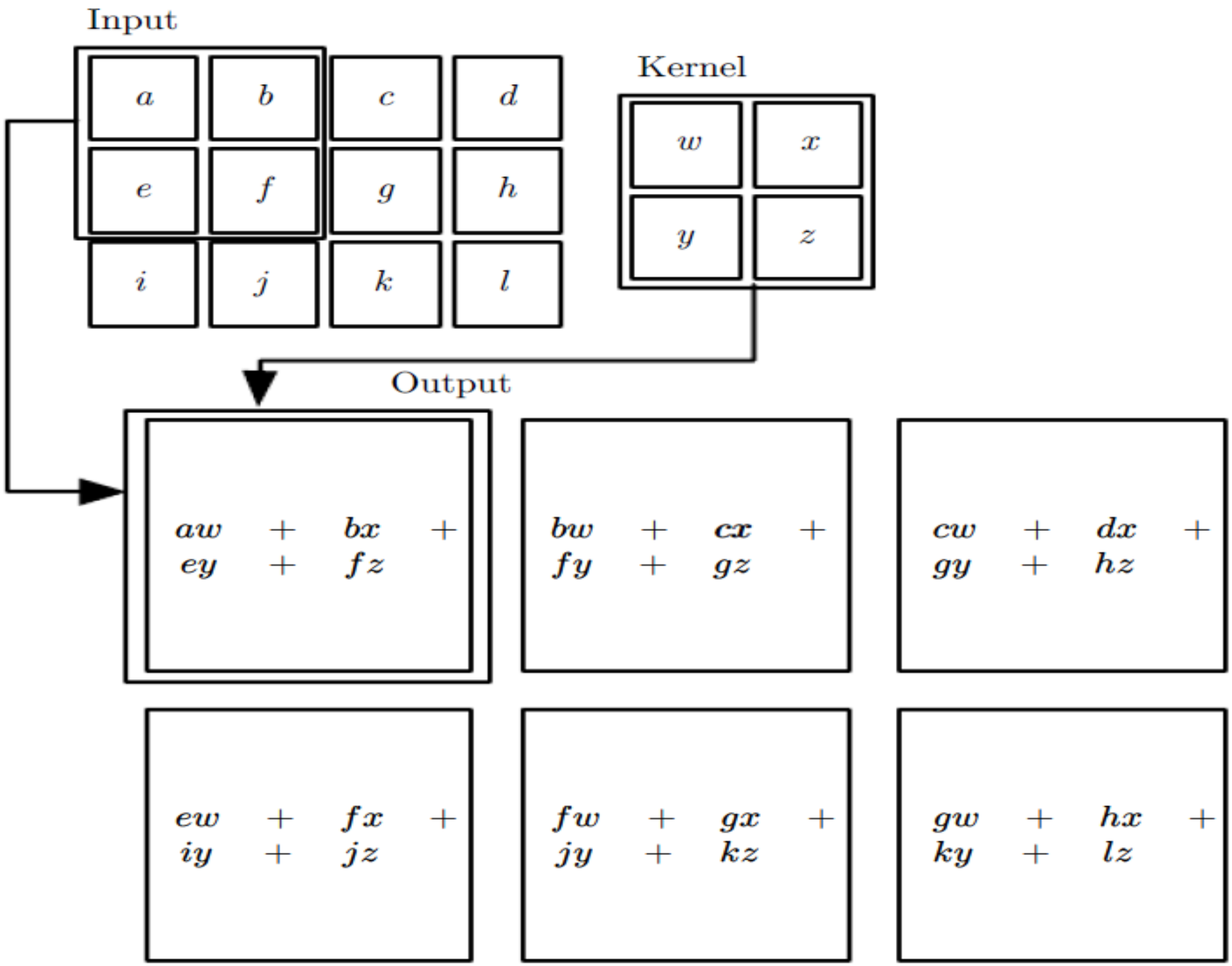
This is a gif image

Convolutional kernel



Padding on the input volume with zeros in such way that the conv layer does not alter the spatial dimensions of the input

Convolutional kernel



CNN – Convolution

Those are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Matrix

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

Matrix

⋮

Property 1

Each filter detects a small pattern (3 x 3).

CNN – Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3 -1

CNN – Convolution

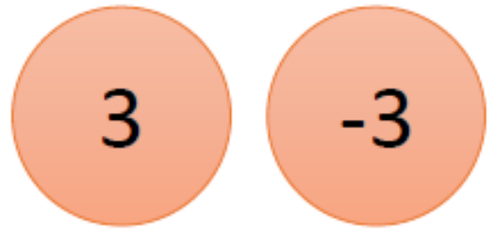
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

If stride=2

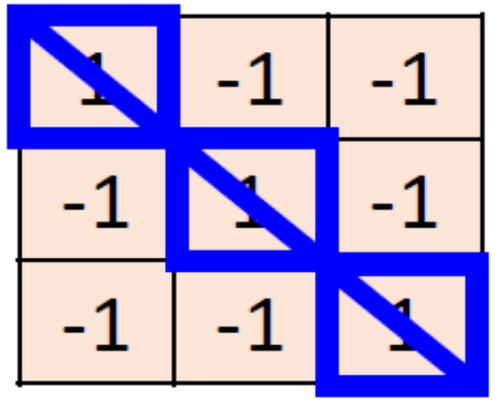
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



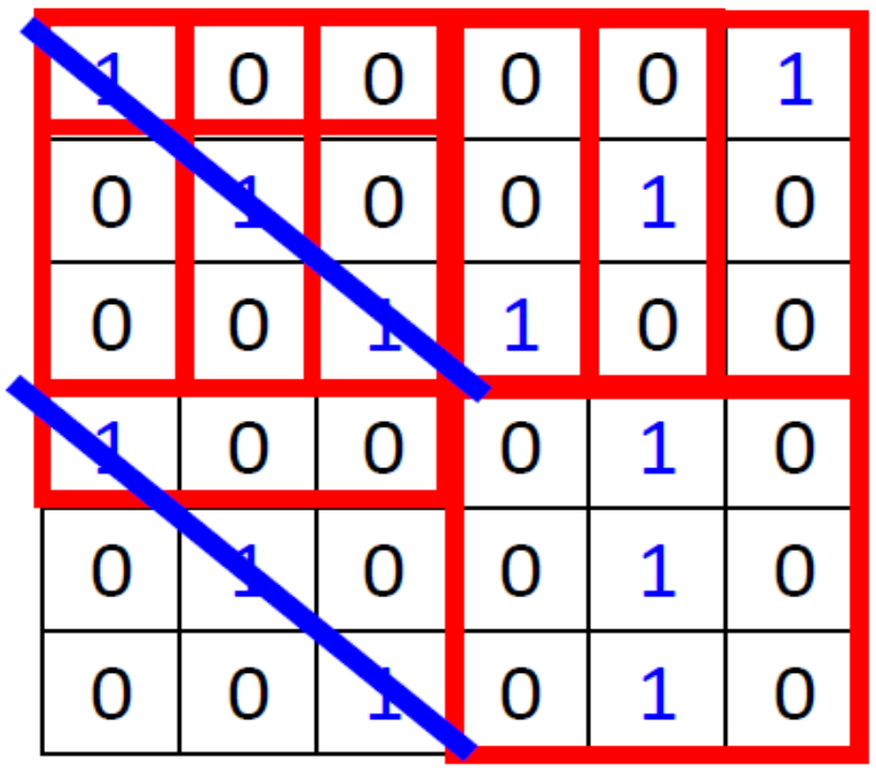
We set stride=1 below

CNN – Convolution

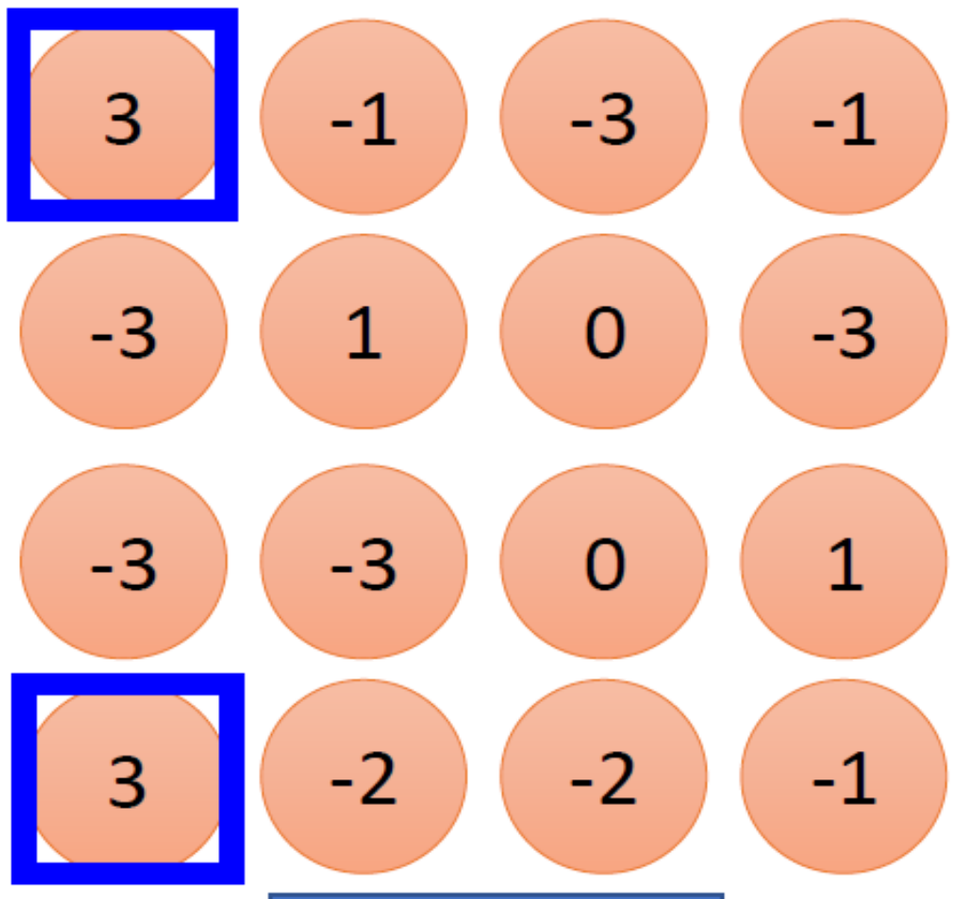


Filter 1

stride=1



6 x 6 image



Property 2

CNN – Convolution

-1	1	-1
-1	1	-1
-1	1	-1

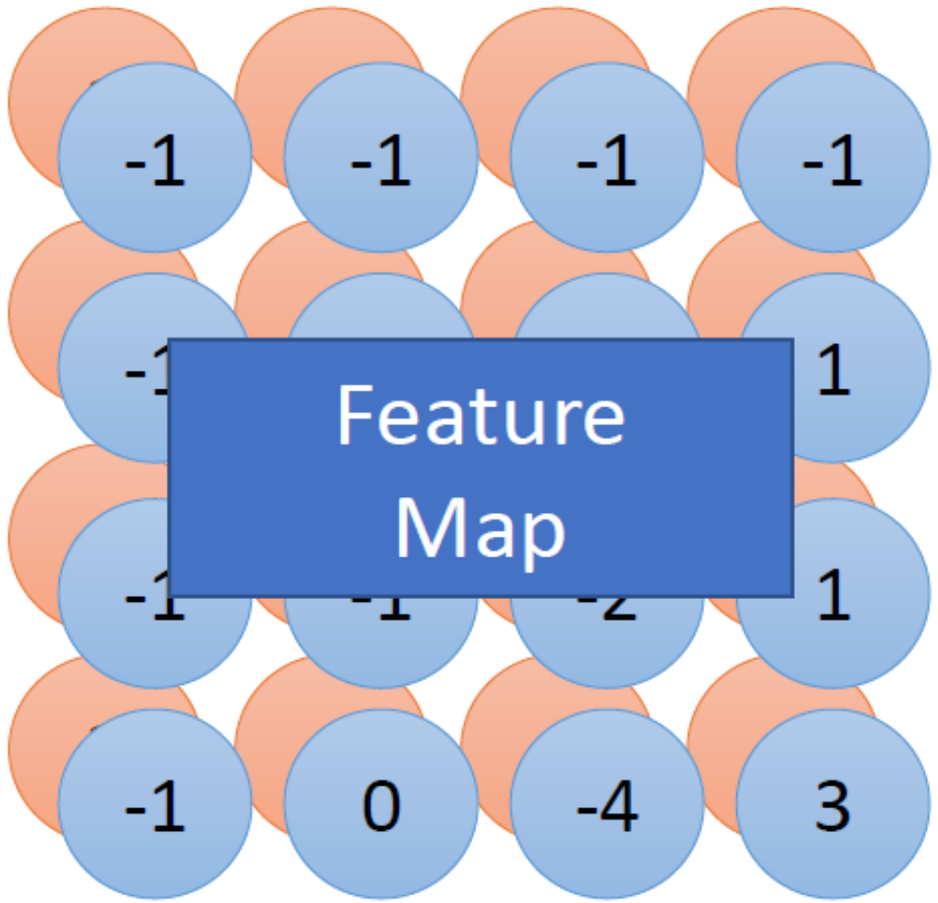
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

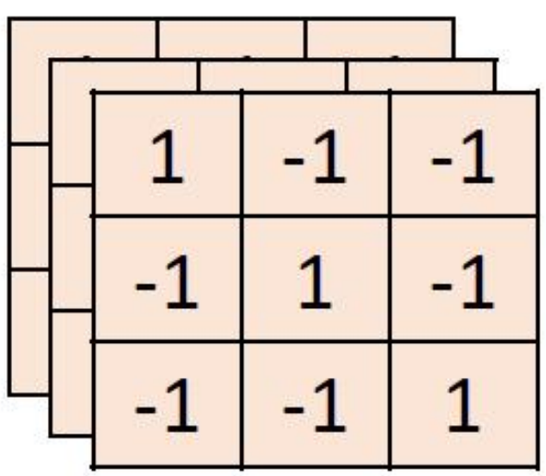
6 x 6 image

Do the same process for every filter

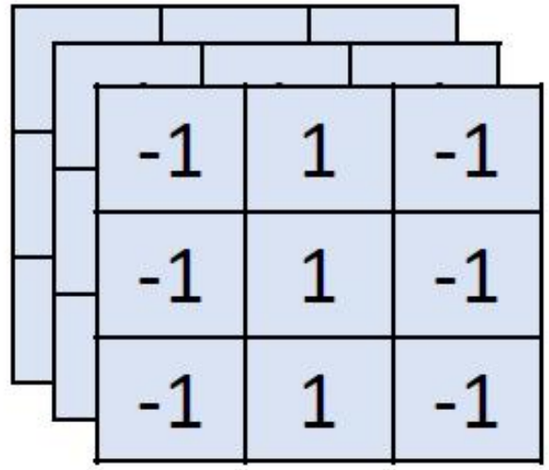


4 x 4 image

CNN – Colorful image

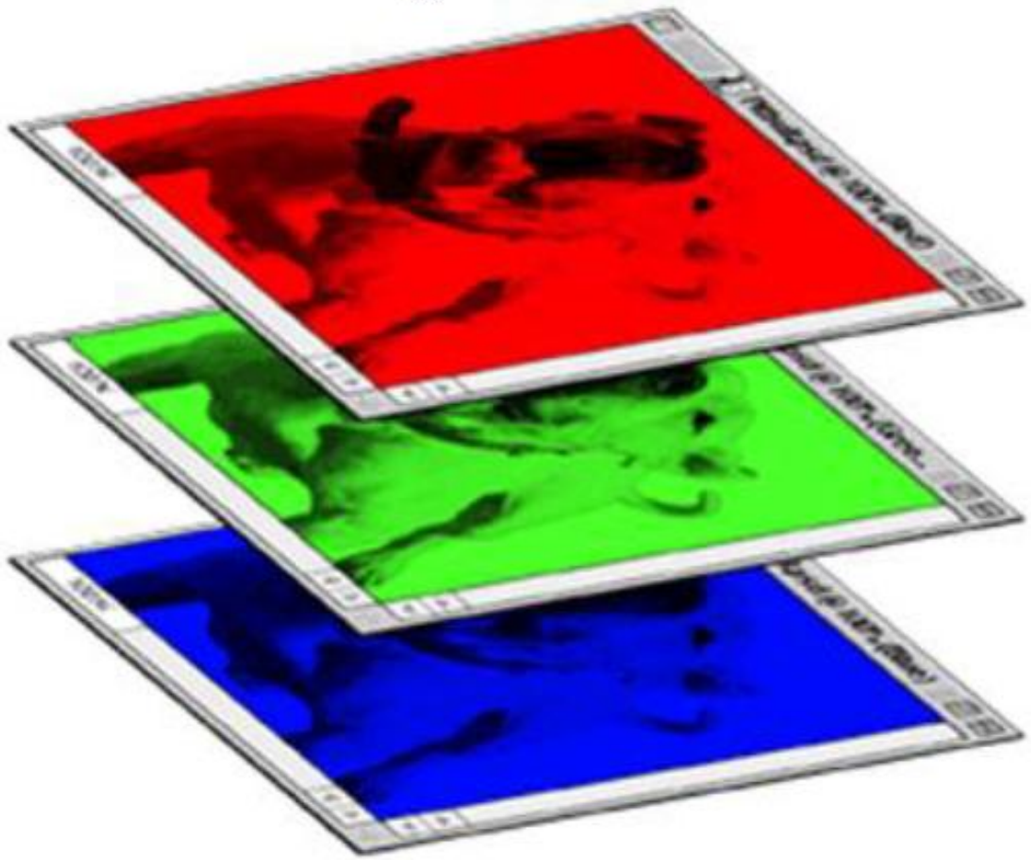


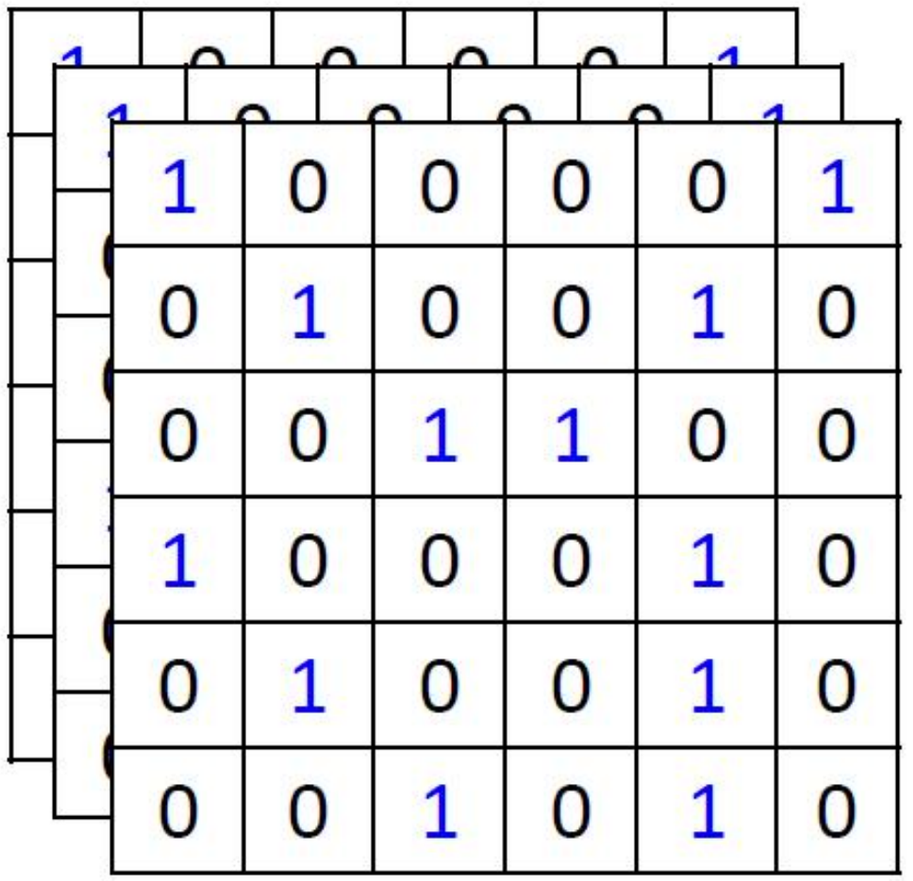
Filter 1

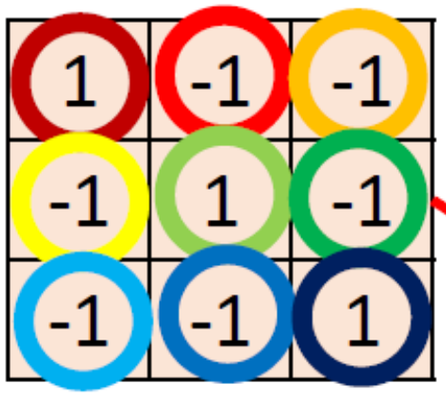


Filter 2

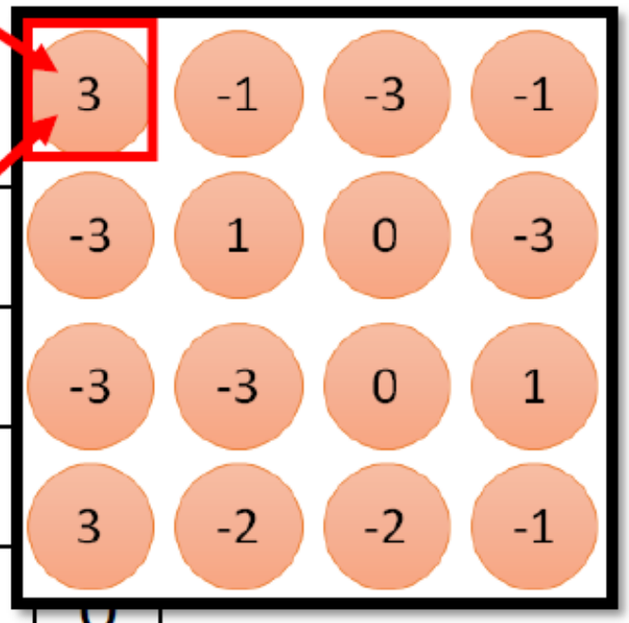
Colorful image







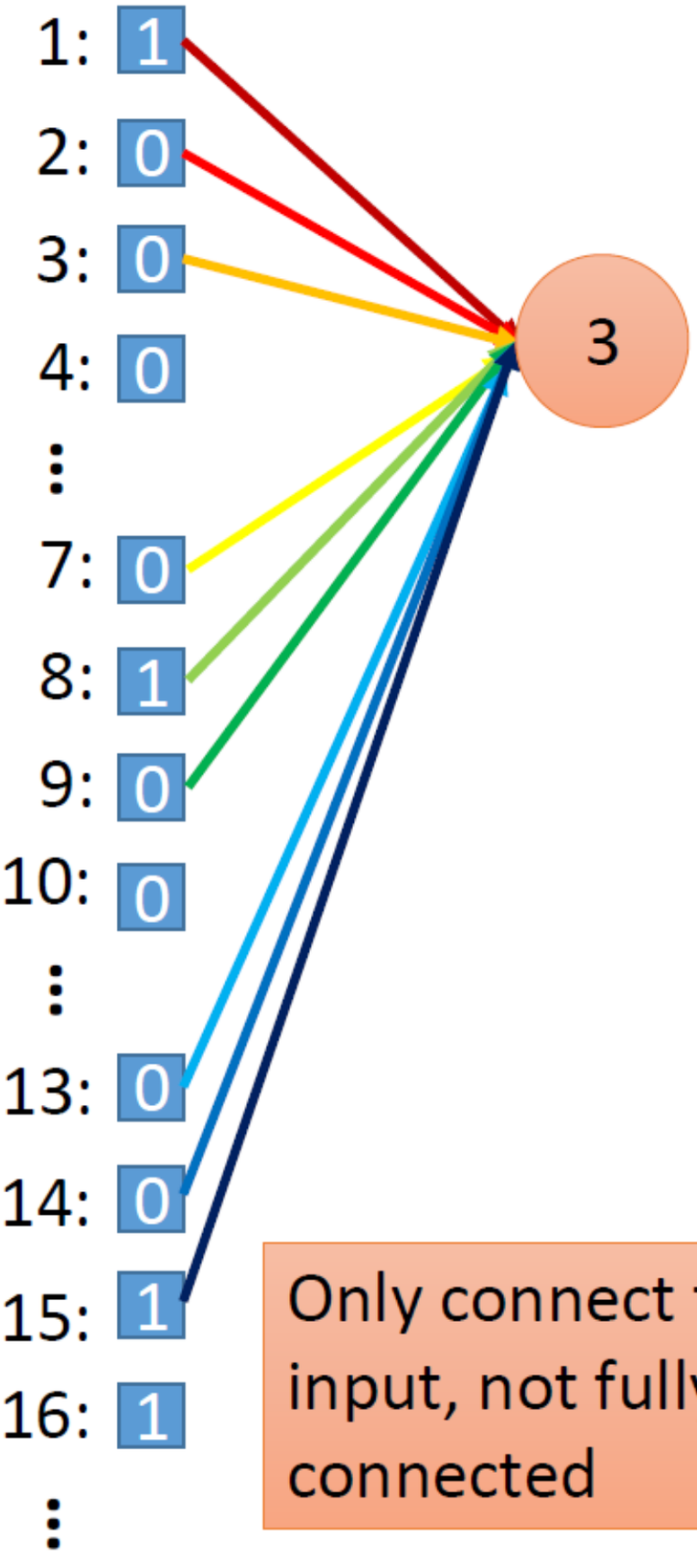
Filter 1



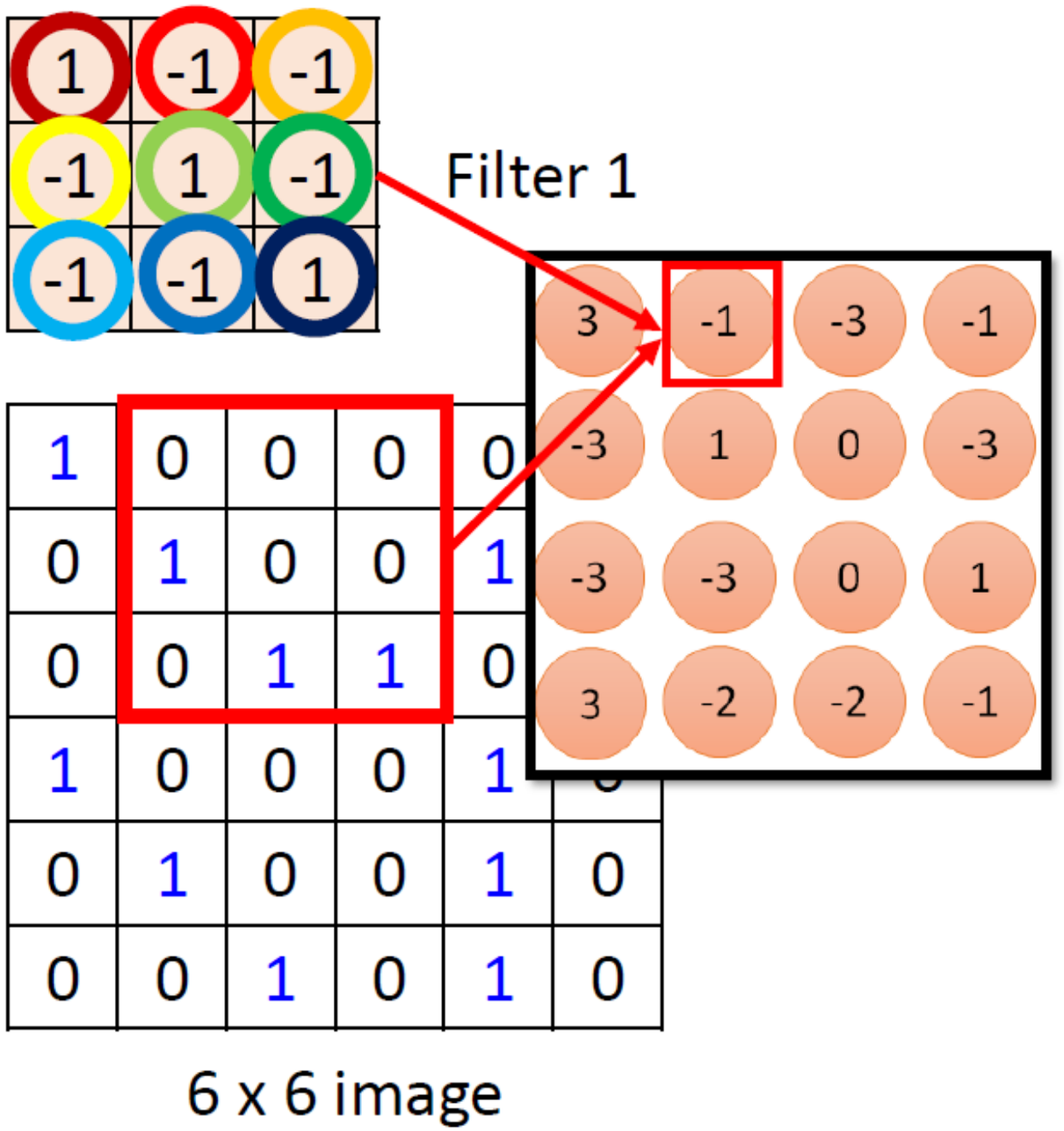
1	0	0	0	0	
0	1	0	0	1	
0	0	1	1	0	
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Less parameters!

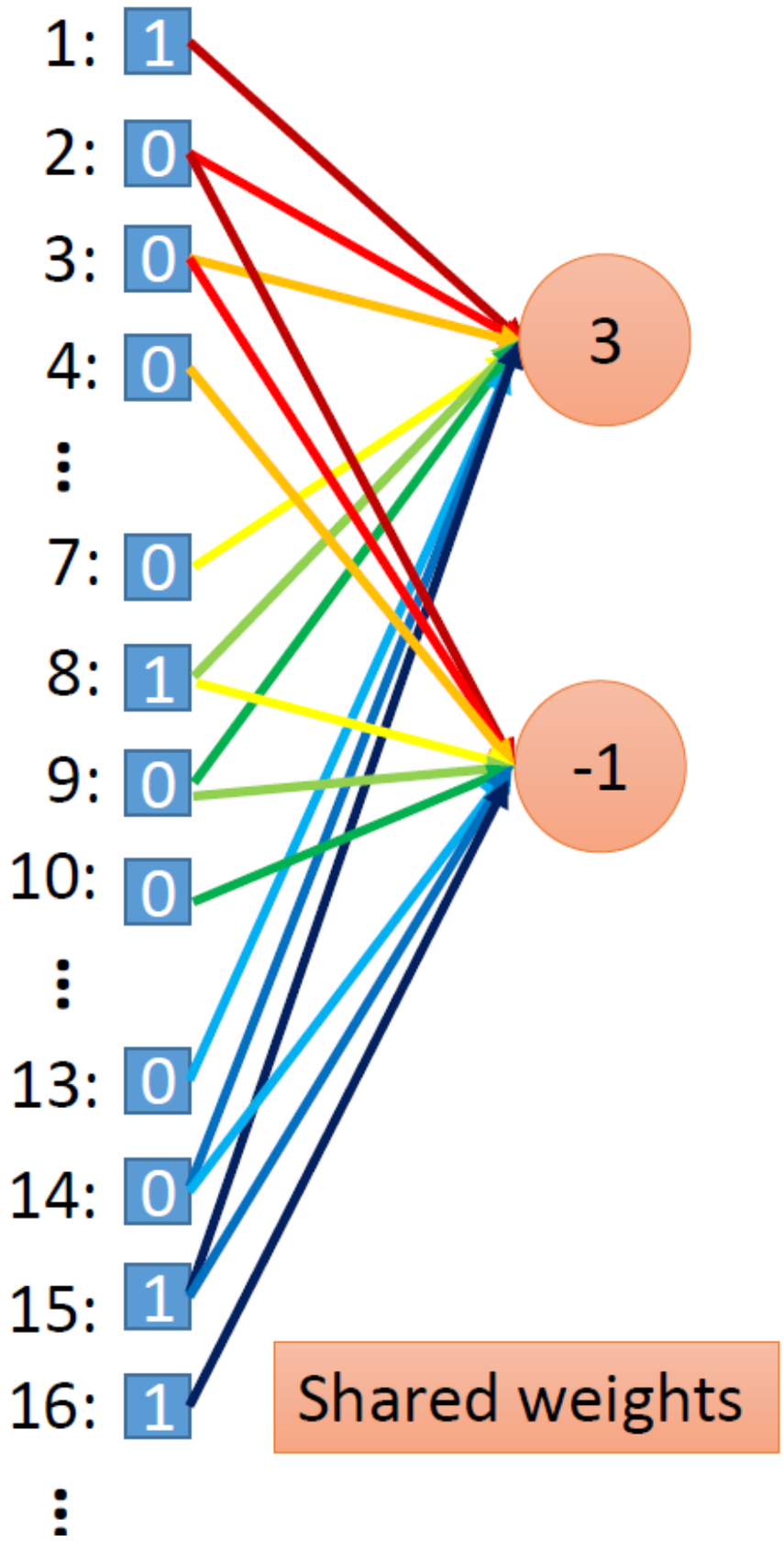


Only connect to 9 input, not fully connected



Less parameters!

Even less parameters!



Convolution

Valid and Same convolutions

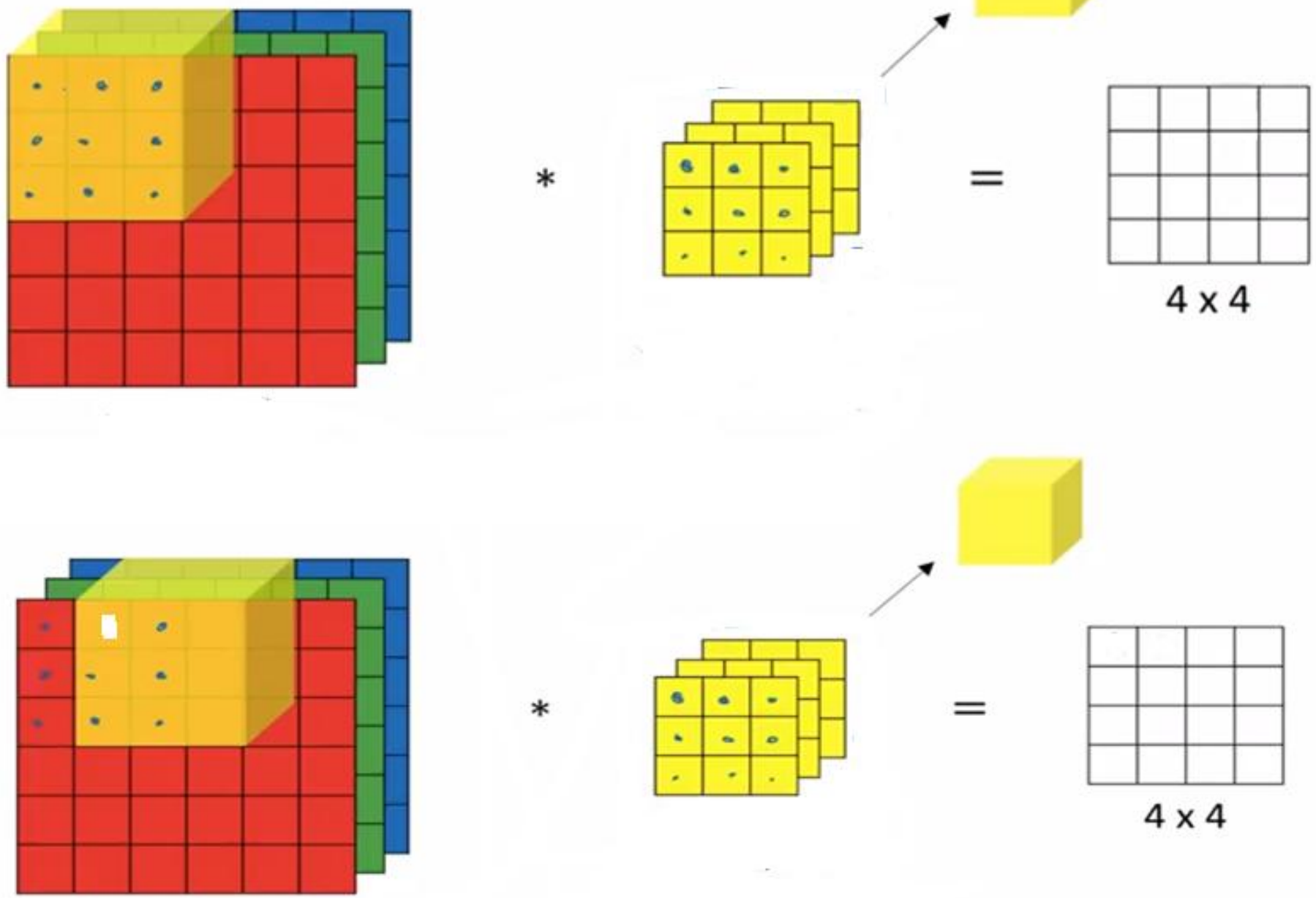
"Valid": No zero-padding is used whatsoever, and the convolution kernel is only allowed to visit positions where the entire kernel is contained entirely within the image.

- In MATLAB terminology, this is called **valid** convolution

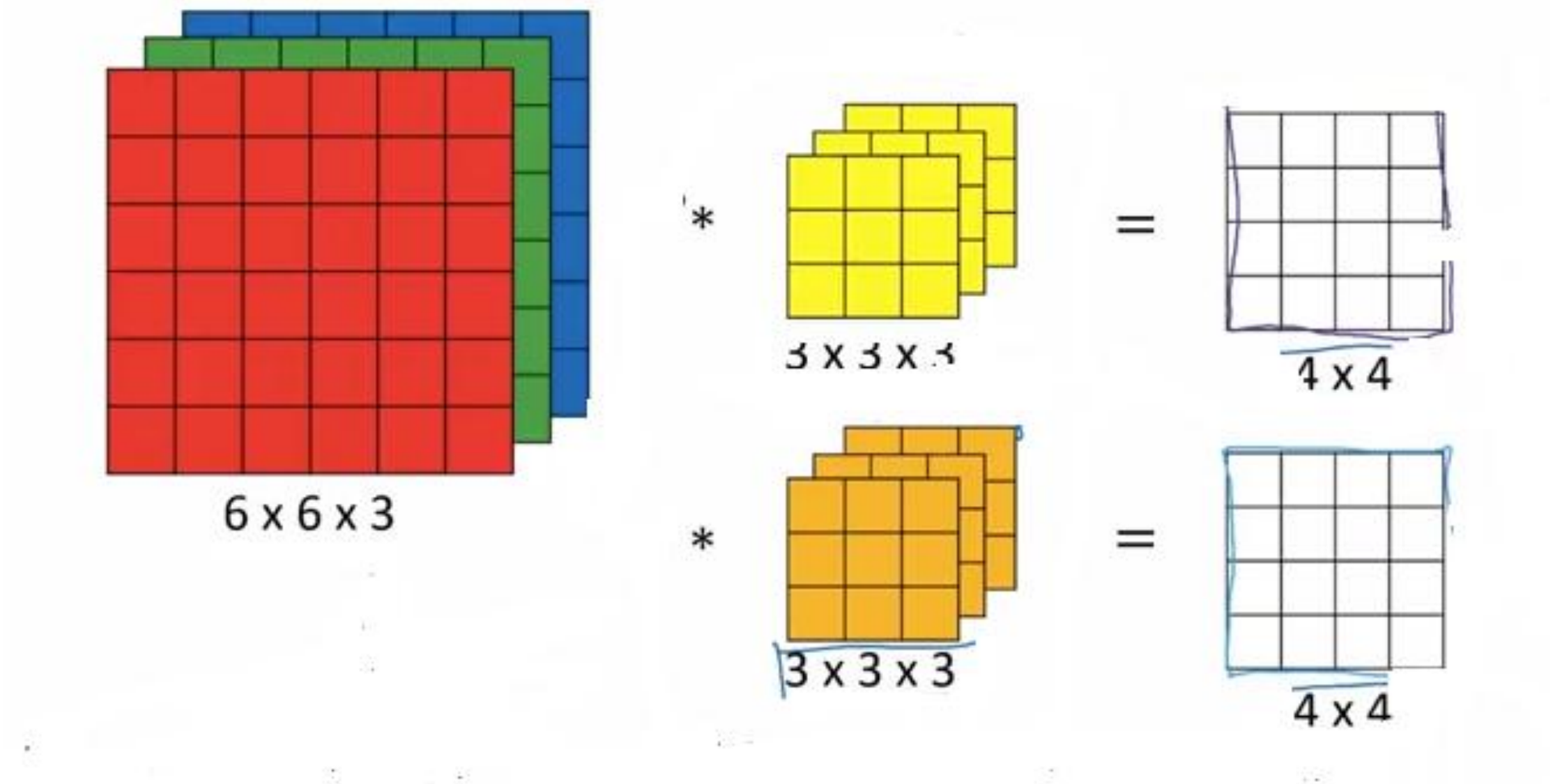
"Same": Pad so that output size is the same as the input size.

Optimal amount of zero padding (in terms of test set classification accuracy) lies somewhere between "valid" and "same" convolution.

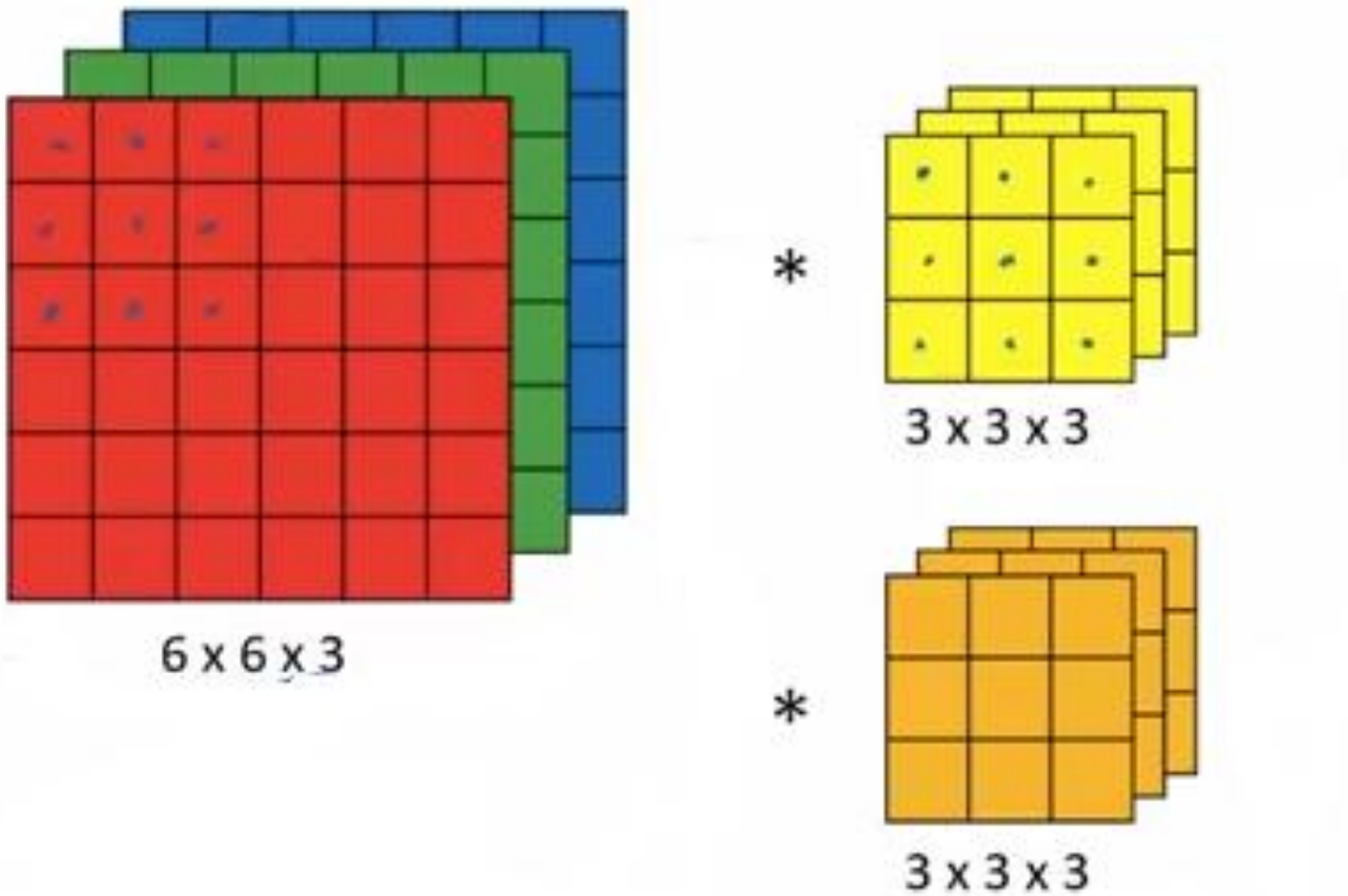
Convolutions on RGB image



Convolution-Multiple Filters



Convolution-Layer example



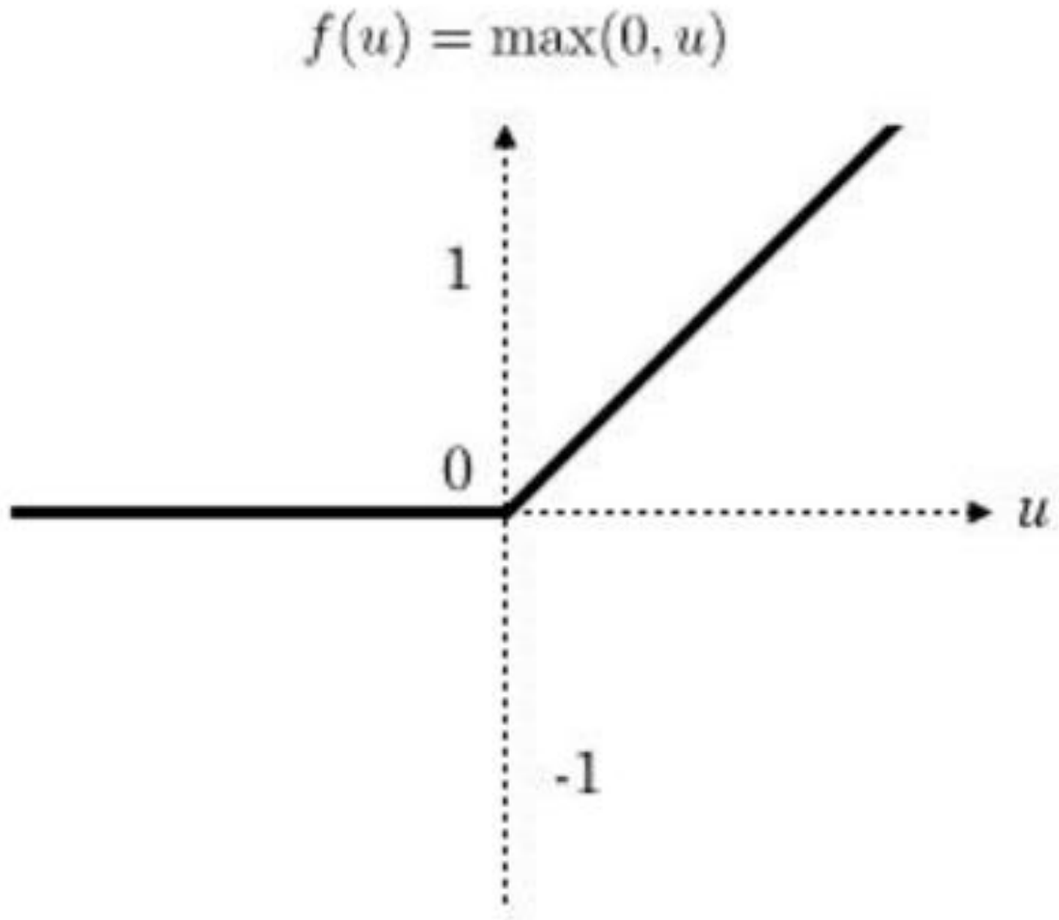
Convolution- no. of parameters in one layer

If you have 10 filters that are 3 x 3 x 3 in one layer of a neural network, how many parameters does that layer have?

Dr. Sujata Pathak, IT, KJSSE

Rectified linear unit， ReLU

rectified linear function， $f(x) = max(0, x)$



Convolution

If the 2d convolutional layer has 10 filters of 3×3 shape and the input to the convolutional layer is 24×24×3, then this actually means that the filters will have shape 3×3×3, i.e. each filter will have the 3rd dimension that is equal to the 3rd dimension of the input. So, the 3rd dimension of the kernel is not given because it can be determined from the 3rd dimension of the input.

Dr. Sujata Pathak, IT, KJSSE

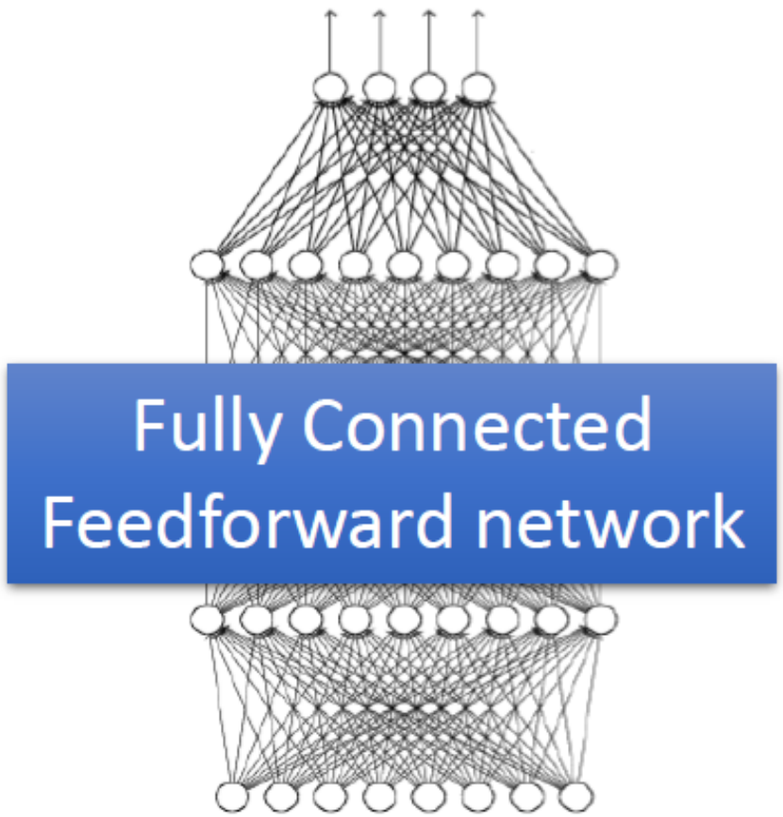
Example of ConvNet

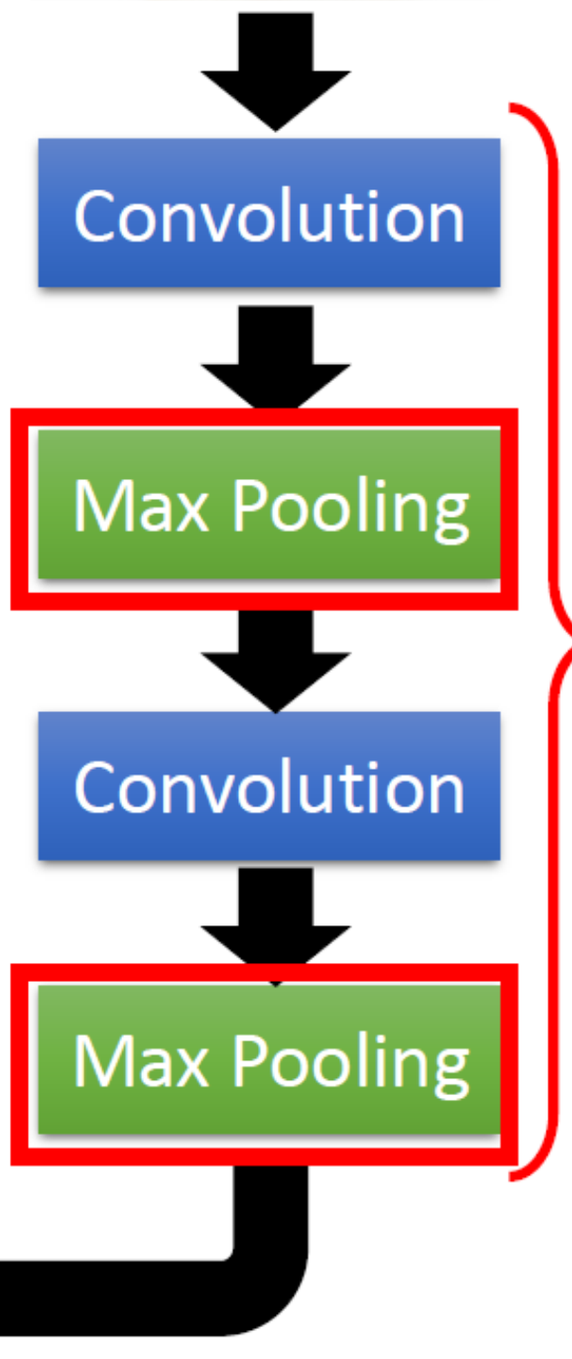
Dr. Sujata Pathak, IT, KJSSE

CNN-Pooling layer

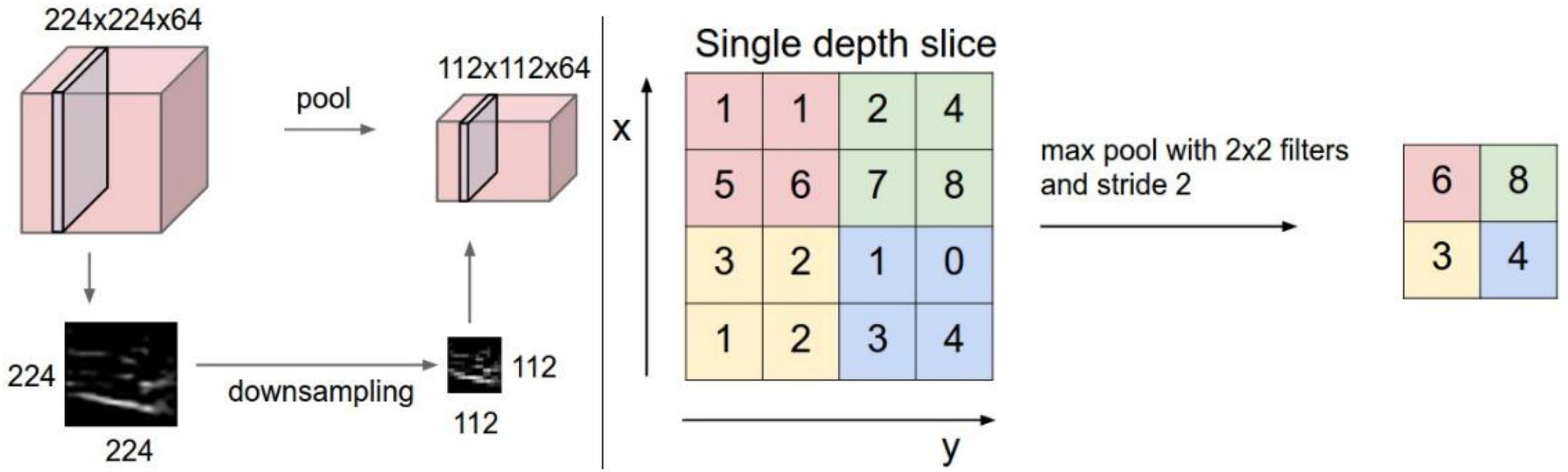
The whole CNN





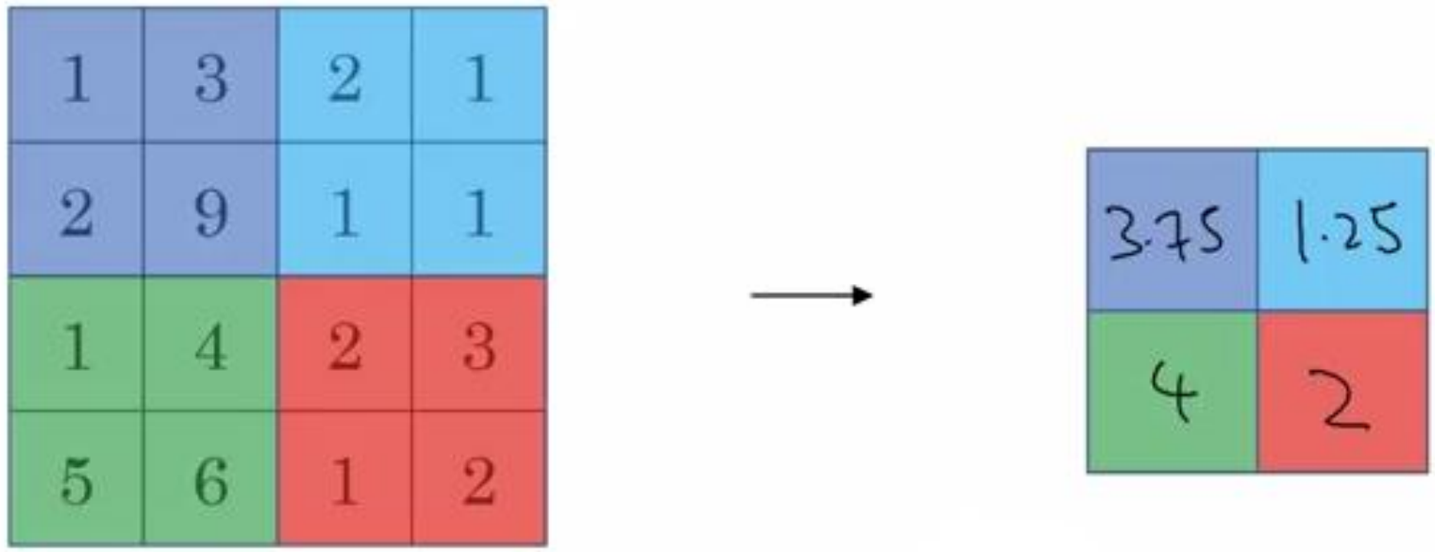


Pooling layer



Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. **Left:** In this example, the input volume of size [224x224x64] is pooled with filter size 2, stride 2 into output volume of size [112x112x64]. Notice that the volume depth is preserved. **Right:** The most common downsampling operation is max, giving rise to **max pooling**, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2x2 square).

Pooling layer-avg pooling



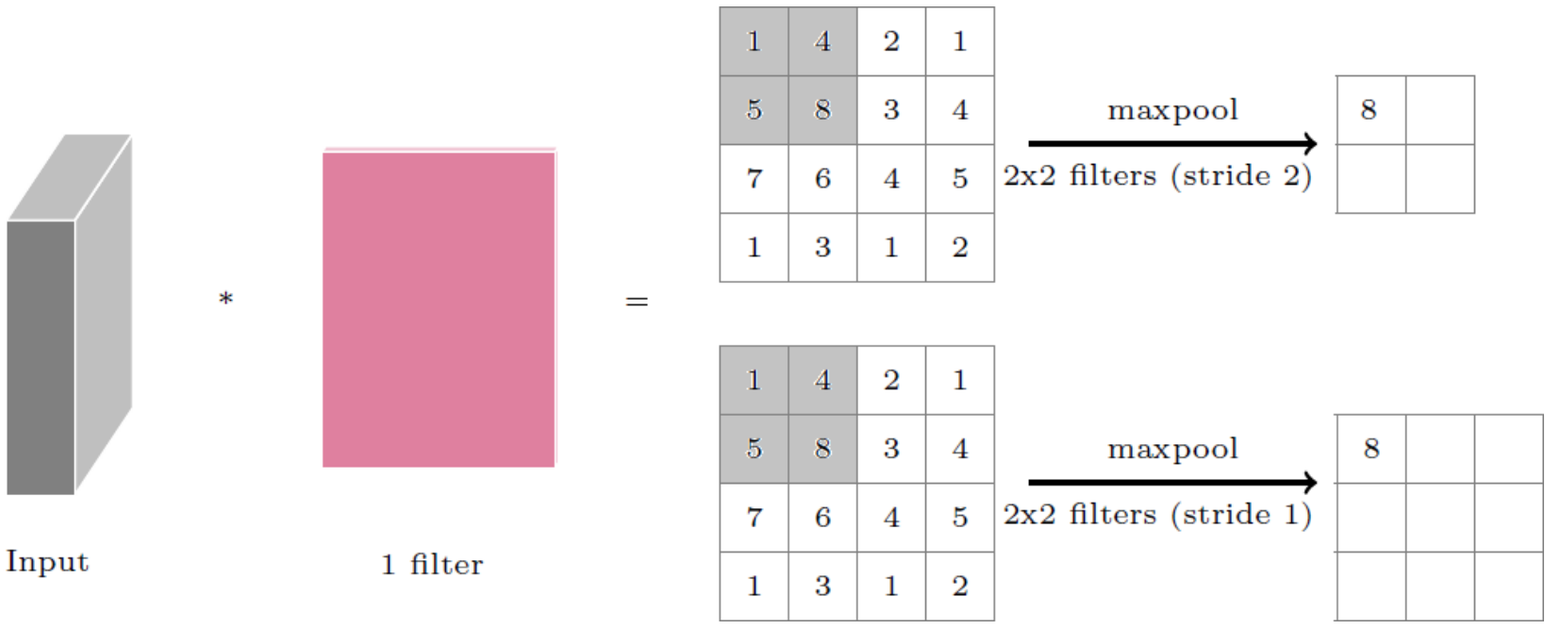
Hyperparameters:

f : filter size

s : stride

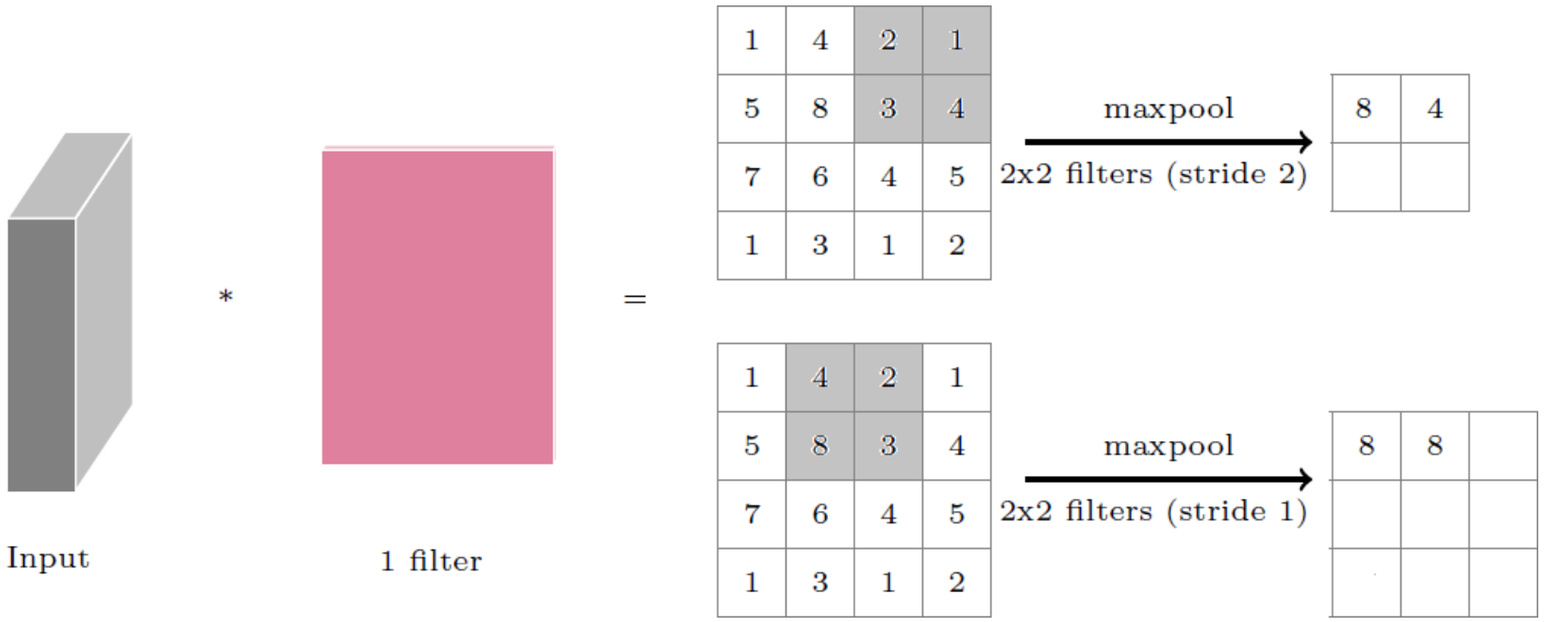
Max or average pooling

Pooling Layer



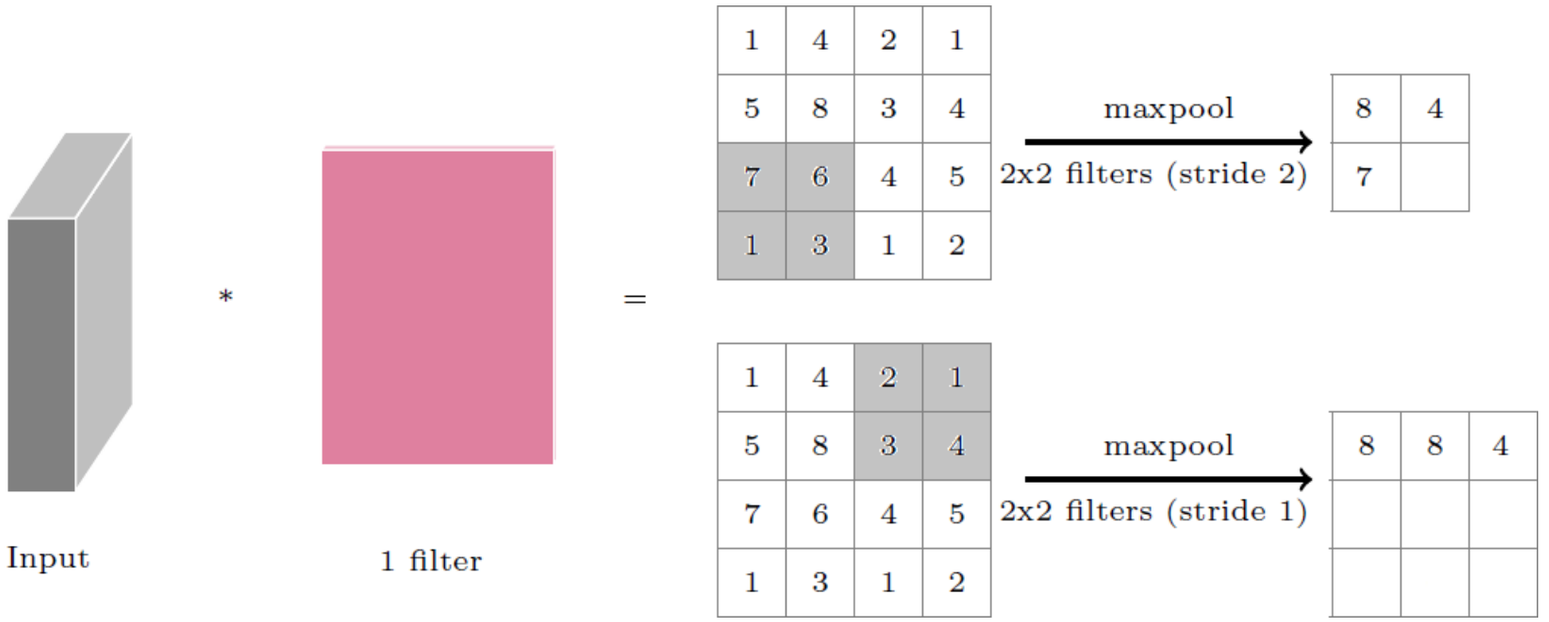
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



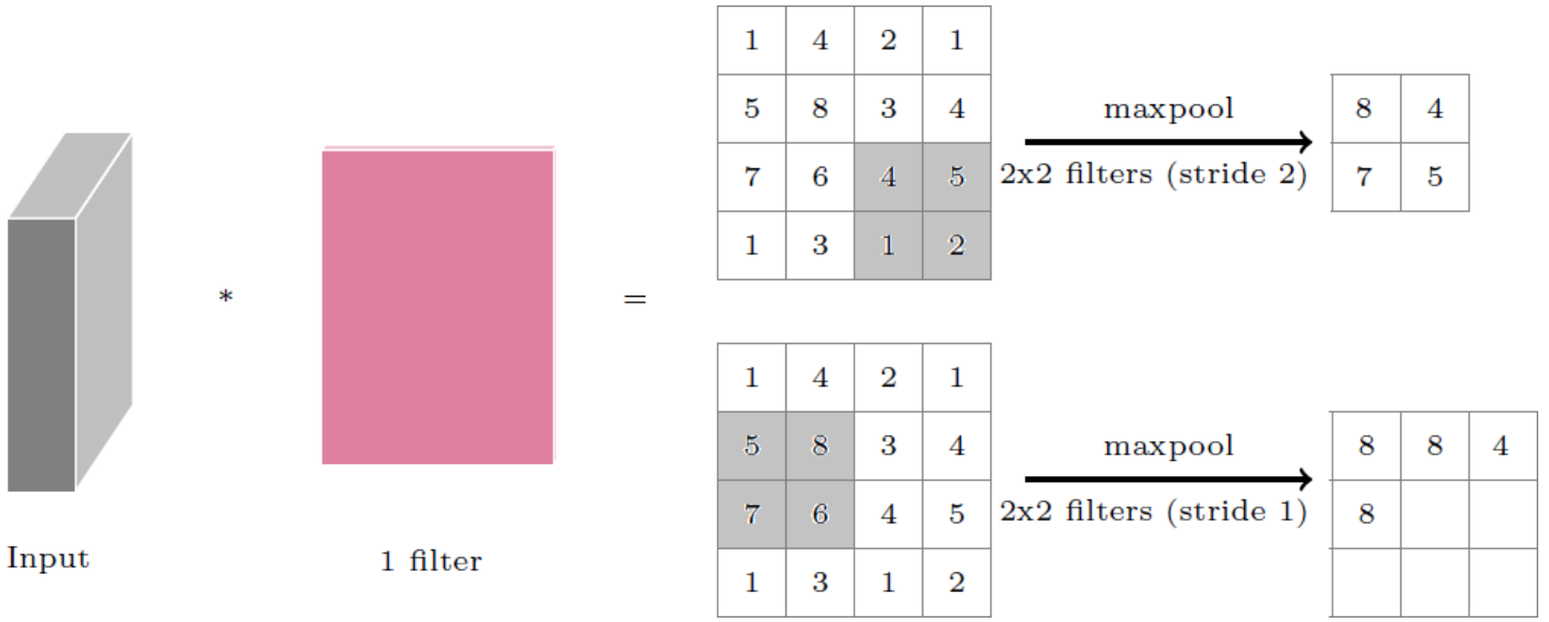
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



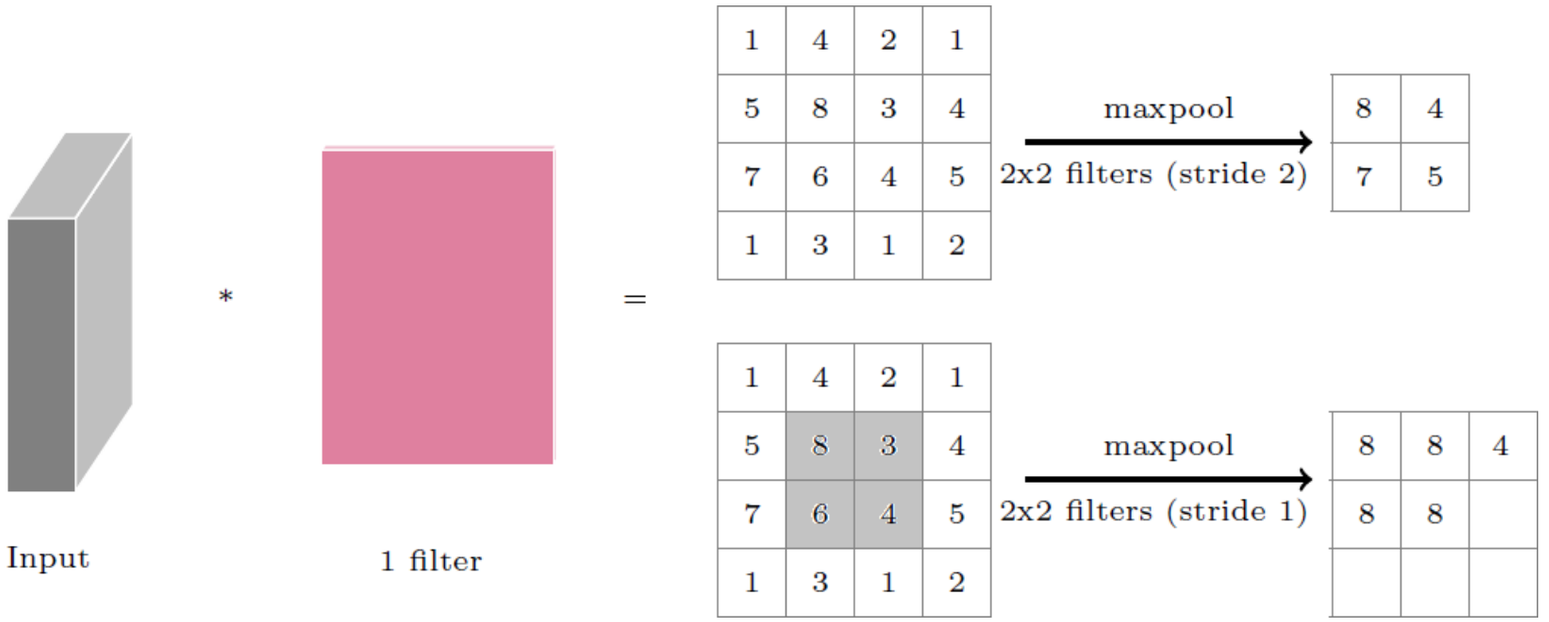
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



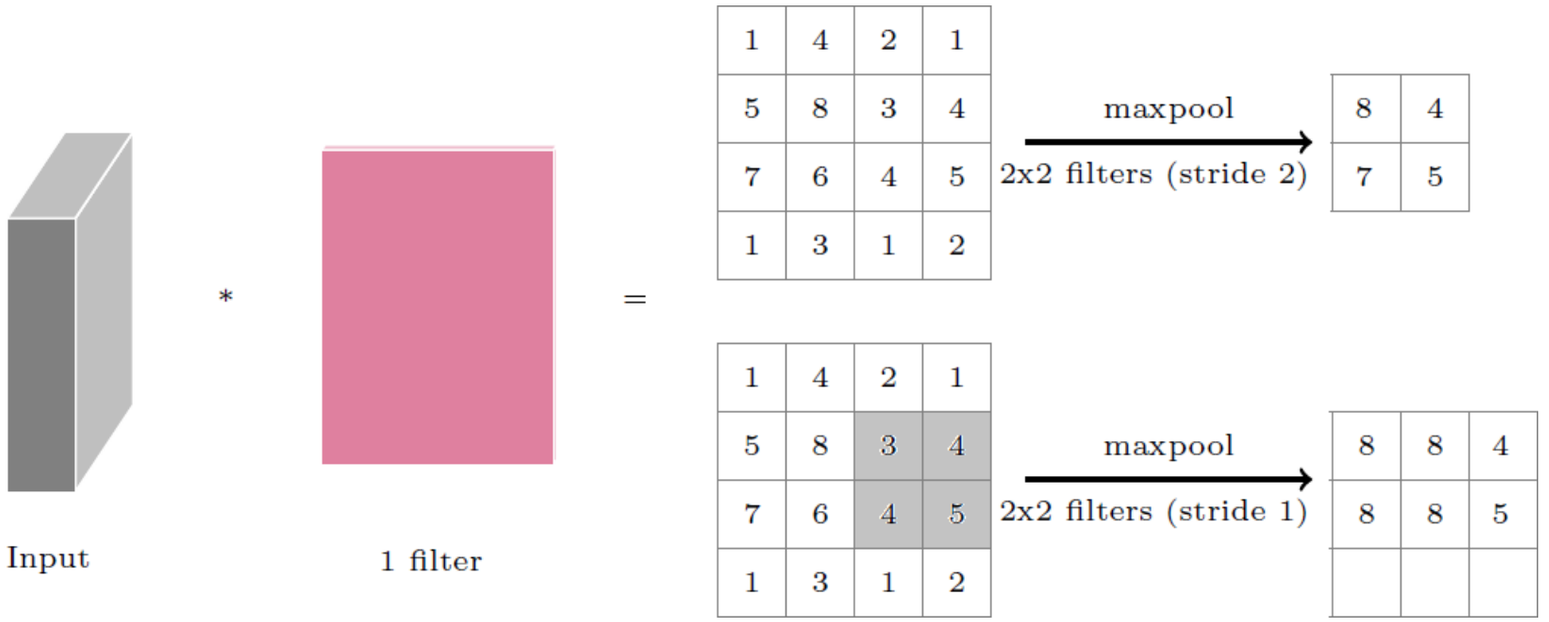
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



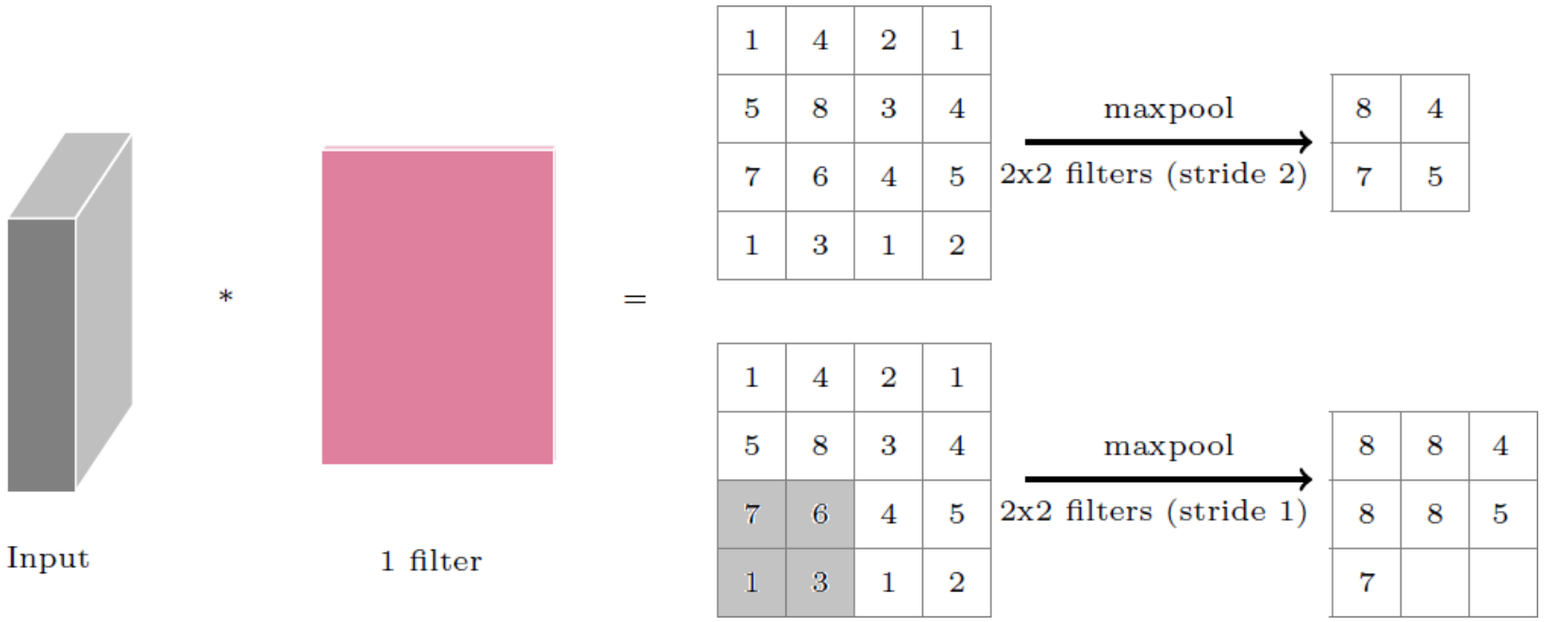
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



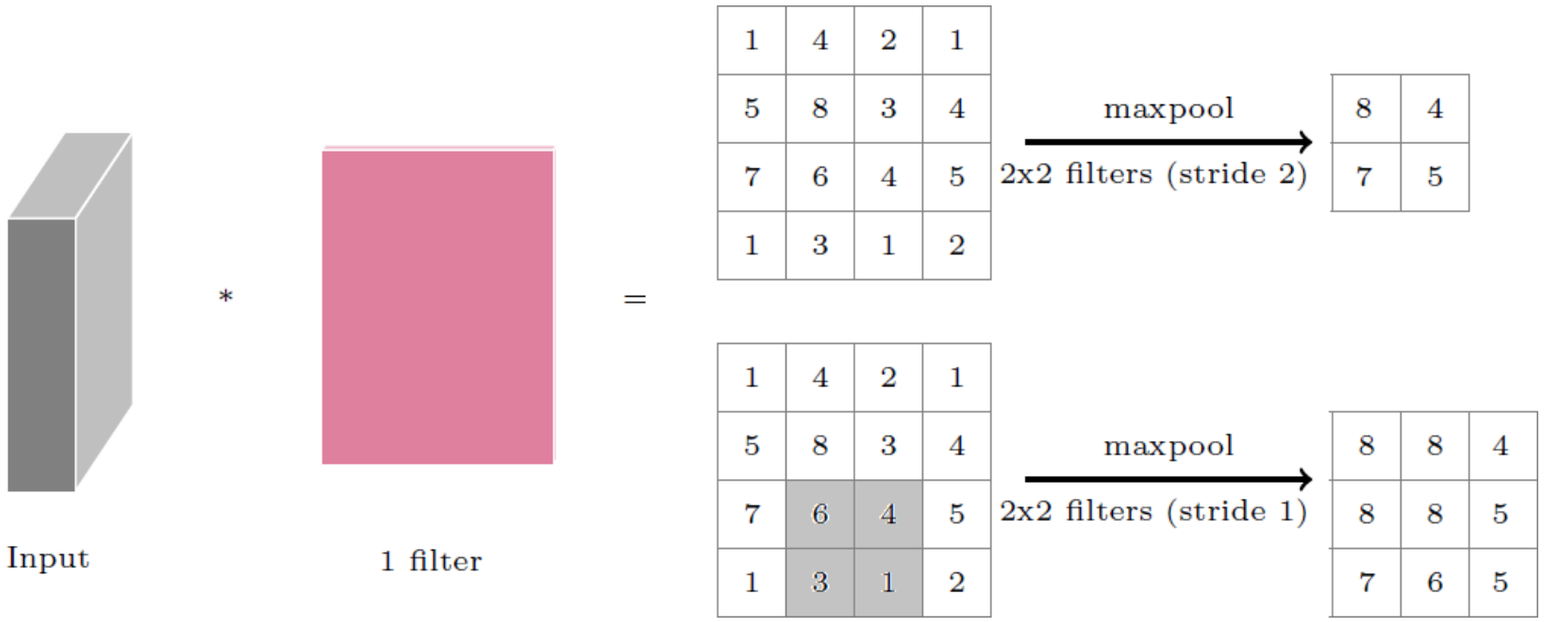
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



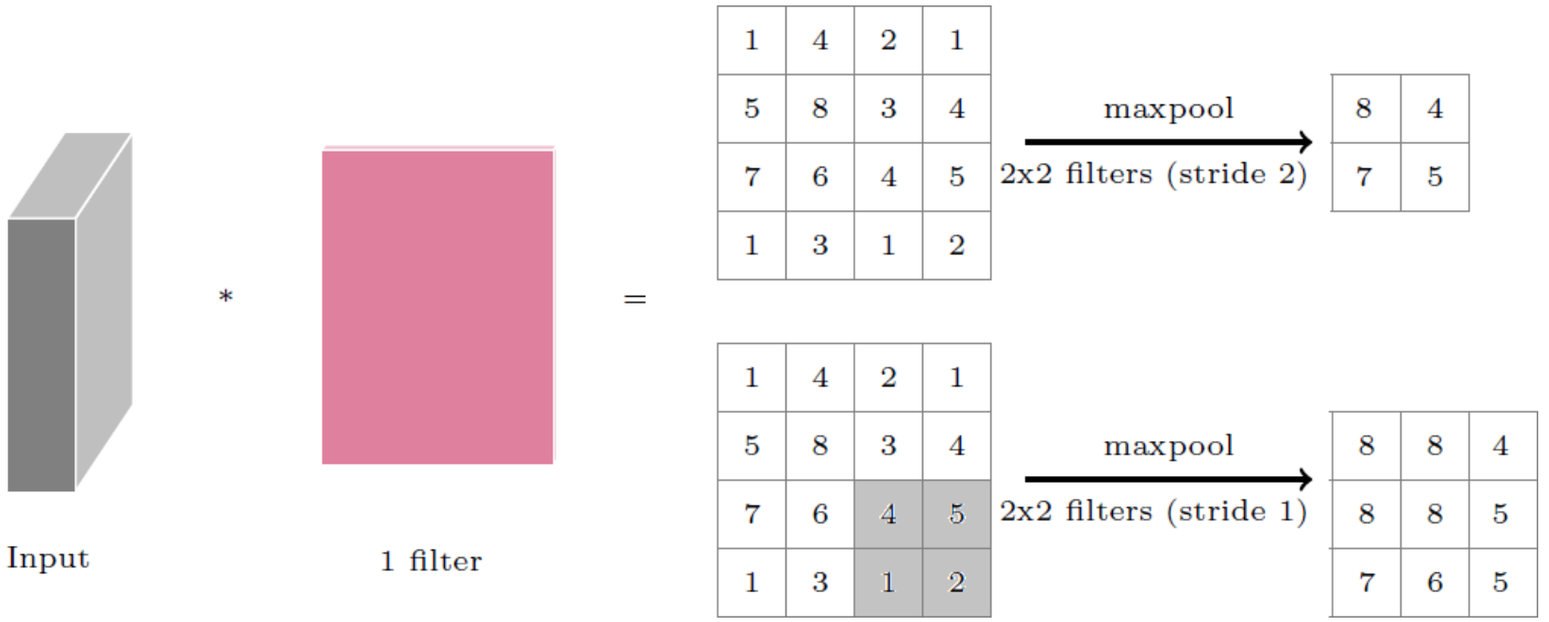
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



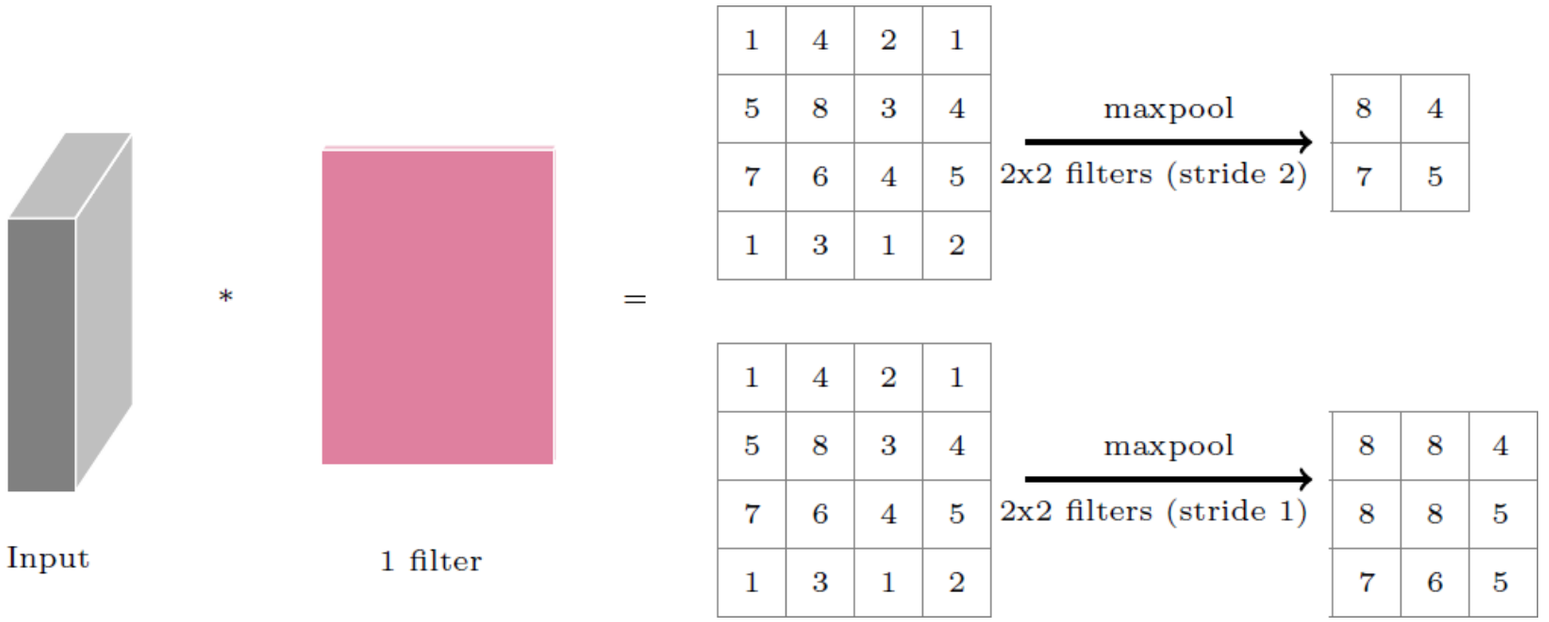
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



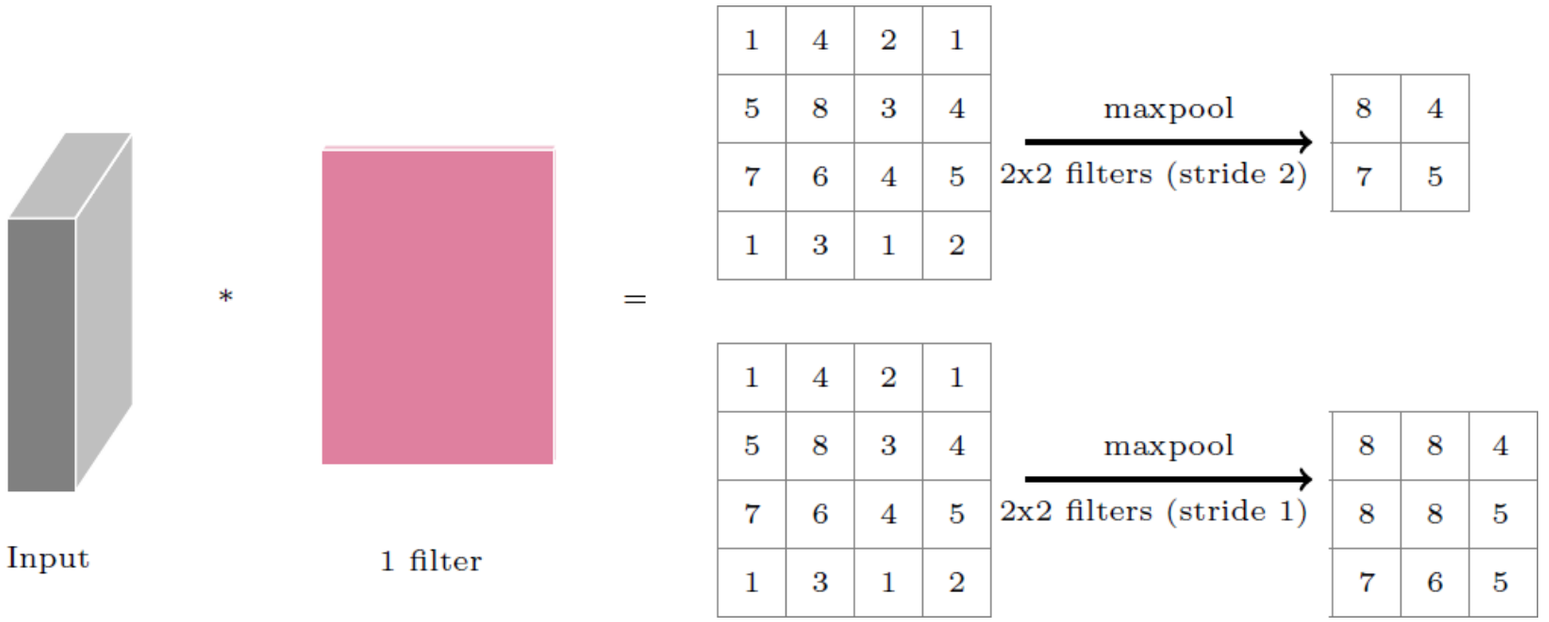
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



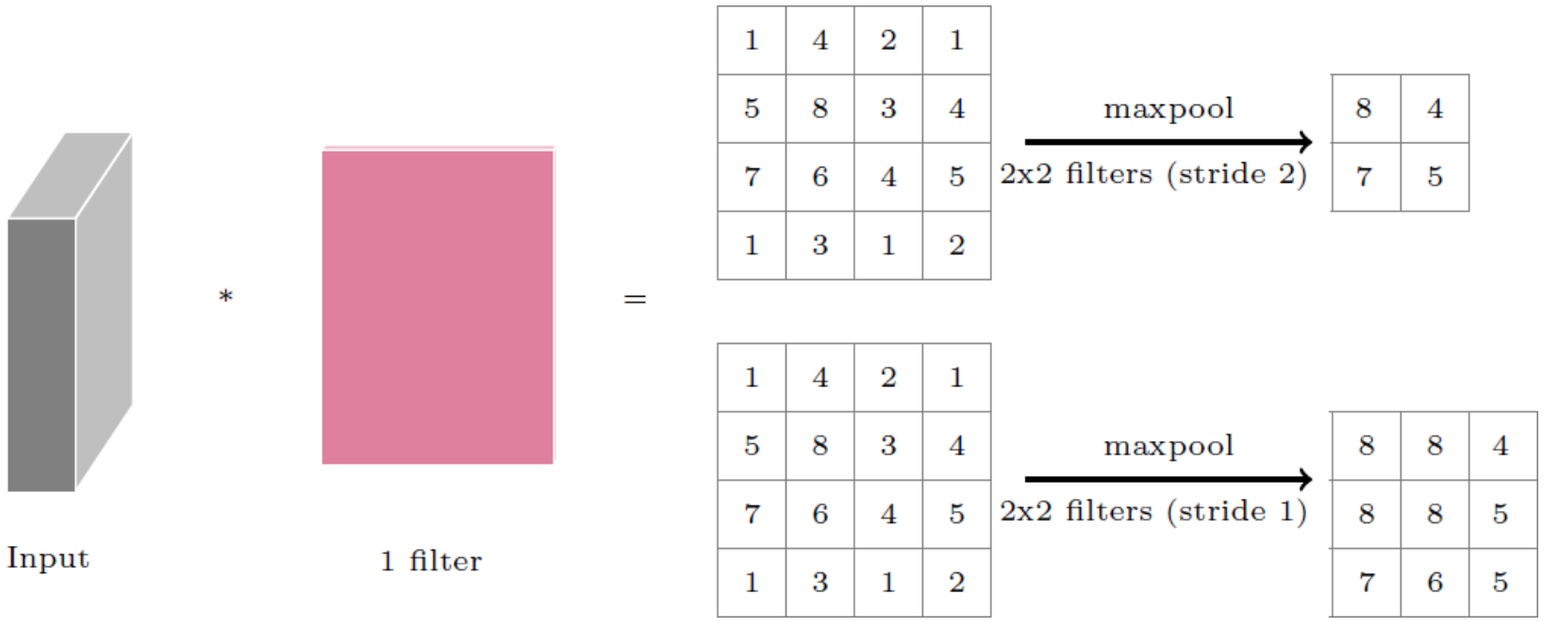
- **Pooling** is a parameter-free down sampling operation

Pooling Layer



- **Pooling** is a parameter-free down sampling operation
- Instead of **Max Pooling**, we can also do **Average Pooling**, L_2 **Pooling**, etc

Pooling Layer



- **Pooling** is a parameter-free down sampling operation
- Instead of **Max Pooling**, we can also do **Average Pooling**, L_2 **Pooling**, etc
- Other notable mentions: Mixed Pooling (combines max and average pooling), Spatial Pyramid Pooling, Spectral Pooling

CNN – Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

3	-1
-3	1

-3	-1
0	-3

-3	-3
3	-2

0	1
-2	-1

-1	-1
-1	-1

-1	-1
-2	1

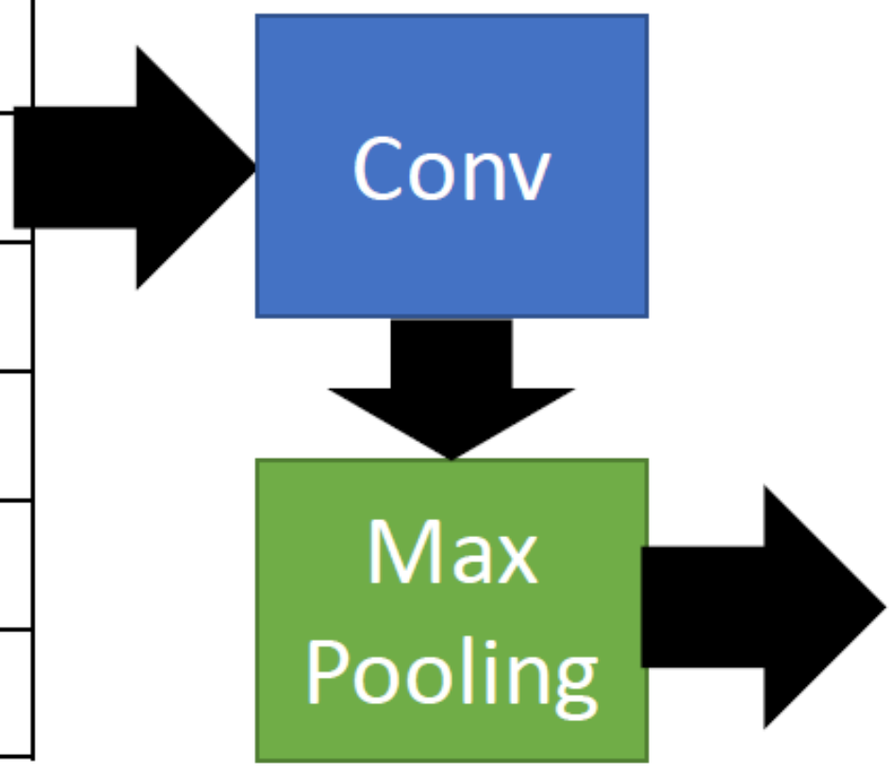
-1	-1
-1	0

-2	1
-4	3

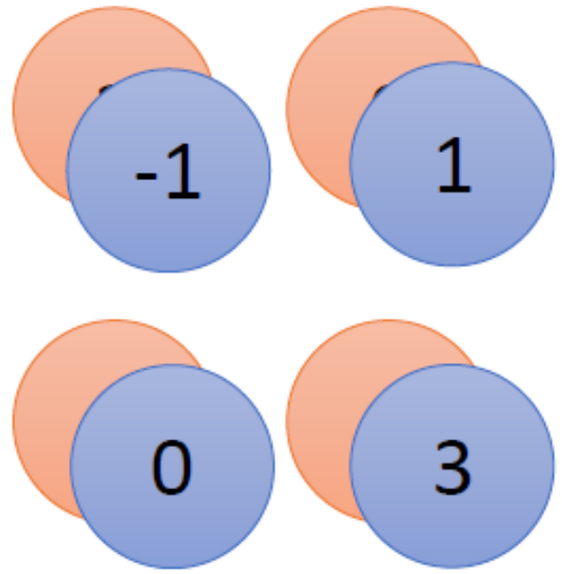
CNN – Max Pooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



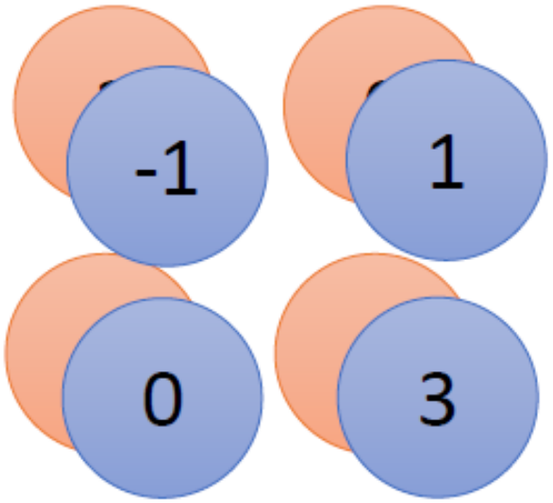
New image but smaller



2 x 2 image

Each filter is a channel

The whole CNN



A new image



Convolution

Max Pooling

Convolution

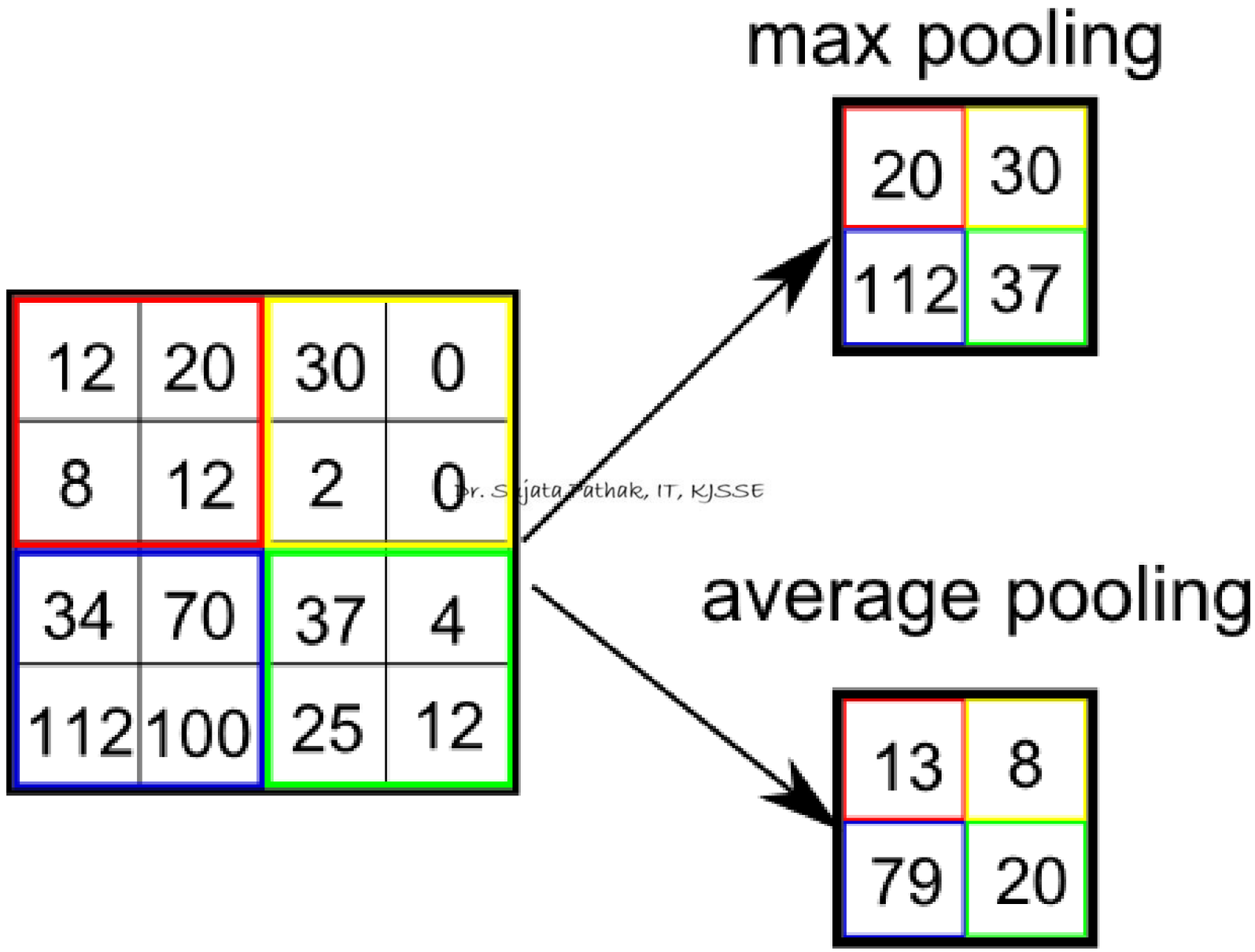
Max Pooling

Can repeat many times

Smaller than the original image

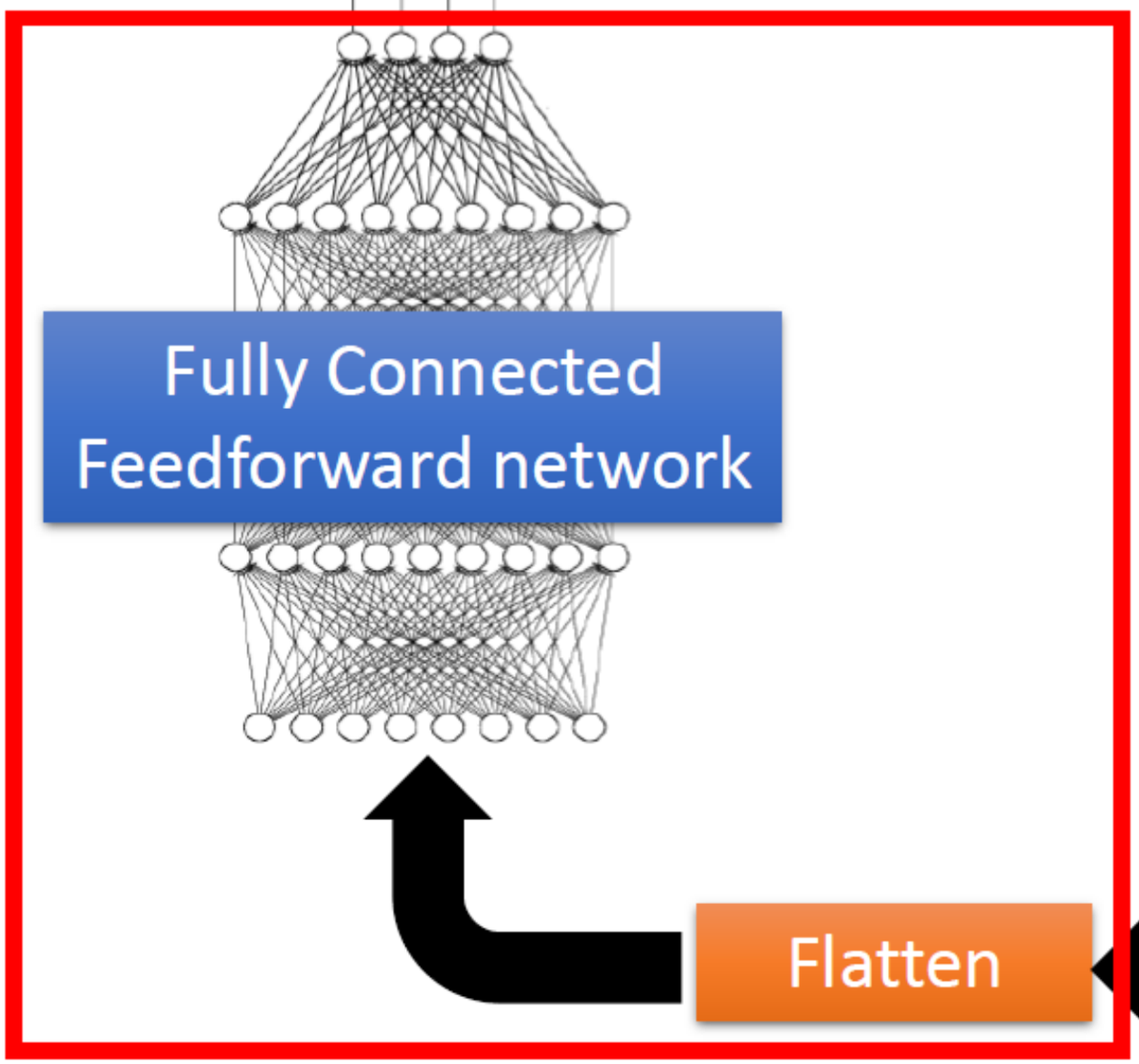
The number of the channel is the number of filters

Pooling

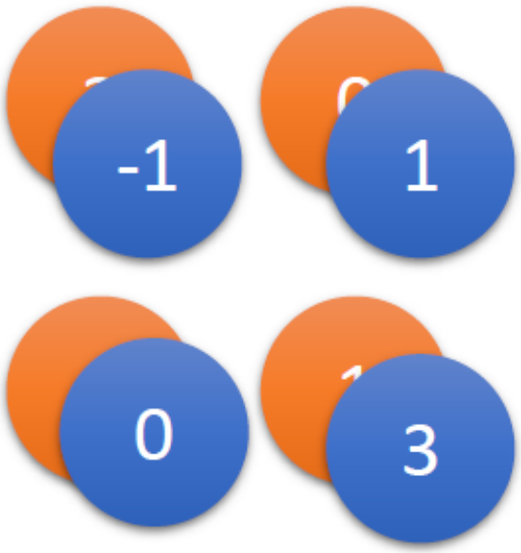

max pooling
20
30
112
37
12
20
30
0
8
12
2
0
34
70
37
4
112
100
25
12
average pooling
13
8
79
20

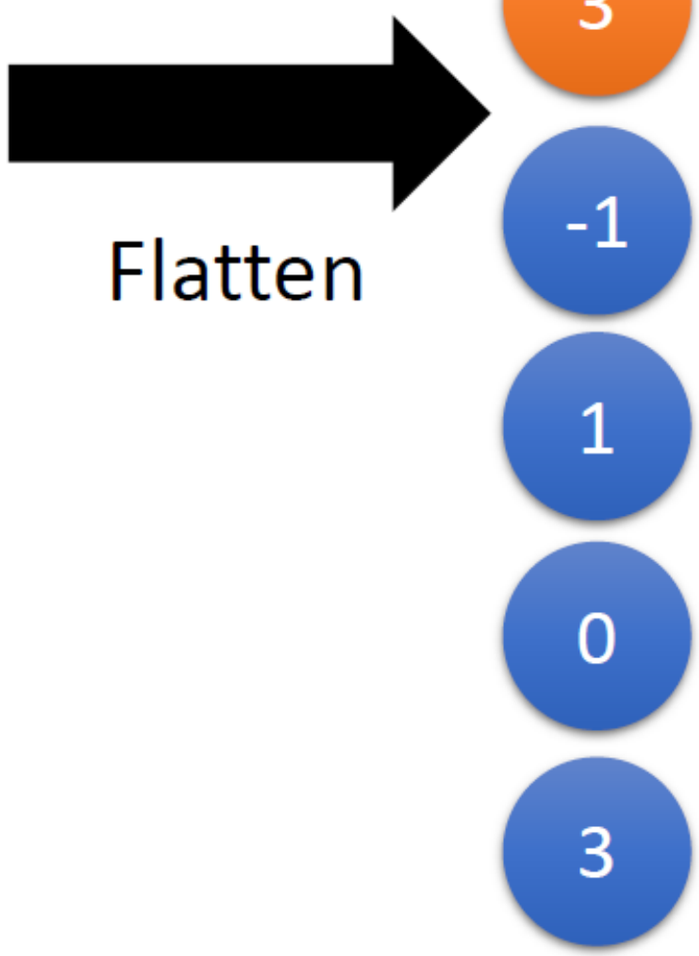
The whole CNN



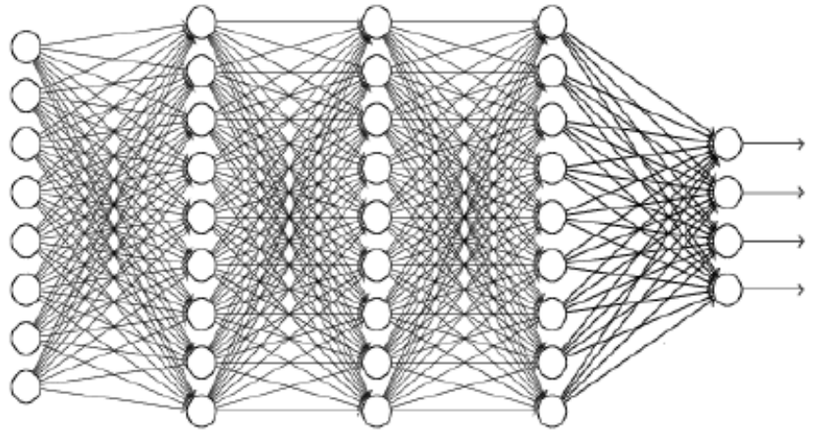


Flatten









Fully Connected Feedforward network